

Table of Contents

(Click on project title to go to respective project page/details)

Project Title	Supervisor Email
AI Student Productivity Ecosystem App	bilal.umar@vu.edu.pk
AI-Based Intelligent Child Safety Monitoring and Geofencing-Based Predictive Tracking System	shakeel@vu.edu.pk
AI-Based Medical Diagnosis System using NLP & Machine Learning	amna.bibi@vu.edu.pk
An Interactive Web Portal for Keeping records of Diabetes-Affected Patients in Pakistan”	mir.salam@vu.edu.pk
Android-Based Smart Electricity Consumption Predictor	imran.afzal@vu.edu.pk
“Answer Question” Community, Learning and Knowledge Platform	abdullah.qamar@vu.edu.pk
Campus Ease - Smart Lost & Found System for University Campuses	Ehsan.haq@vu.edu.pk
Charity Management System for Donation and Request Handling	ammara.rasheed@vu.edu.pk
Citizen Issues Management System	ahmadlodhi@vu.edu.pk
Collaborative Sharing of Bulk Items Purchasing System	umra.naeem@vu.edu.pk
Community Food Wastage Reduction Portal	hasnain@vu.edu.pk
Smart Query Routing & Email Automation System	Saima.jamil@vu.edu.pk
CyberNova Solutions	sohailaamer@vu.edu.pk
Design and Development of the FashionHub Web Application	qamar@vu.edu.pk
Digital Mandi – Smart Agricultural Marketplace Platform	tahir.jan@vu.edu.pk
EduVault Library Management System	musaddiq.hussain@vu.edu.pk
Employee Salary Management System Using Python	asad.ullah@vu.edu.pk
Project Proposal	amjad.iqbal@vu.edu.pk
Heart Disease Prediction Android App	hina.rafique@vu.edu.pk
Hive Construction Ventures Advisor System	adnanasif@vu.edu.pk
Intelligent Bug Hunting Platform Project	arif.husen@vu.edu.pk
Intelligent LMS for Automated Evaluation of OS Scheduling and Process Management Assignments	umarfarooq@vu.edu.pk
IoT-Based Smart Parking Locator with Google Maps Integration	waqar.ahmad@vu.edu.pk
Project Title: Items Auction through online System	asifhussain@vu.edu.pk
LIBRARY MANAGEMENT SYSTEM	Jibrankhan@vu.edu.pk
Madaris Management System (MMS) - Android Application	saeednasir@vu.edu.pk
Make Your Own Platter – Android-Based Food	said.nabi@vu.edu.pk

<u>Ordering System with Real-Time Alerts</u>	
<u>Medicare System</u>	hashir.khan@vu.edu.pk
<u>Medicine Recommendation System</u>	mushtaq.hussain@vu.edu.pk
<u>Mobile Accessories Stock Control app.</u>	imran.akhtar@vu.edu.pk
<u>Multi-Turn AI Chatbot with LLaMA 3</u>	neelam.alam@vu.edu.pk
<u>NGO Charity Management System</u>	qaiser.shabbir@vu.edu.pk
<u>Project Title:</u>	rehan@vu.edu.pk
<u>Online Exams App</u>	haseebakmal@vu.edu.pk
<u>Online Food Ordering and Delivery System</u>	warda.fiaz@vu.edu.pk
<u>OMRS: Online Medical Record System</u>	syed@vu.edu.pk
<u>Online Parent-Teacher Meeting System</u>	akmalkhan@vu.edu.pk
<u>Online Printing and Packaging Services System</u>	muhammad.saeed@vu.edu.p k
<u>Online Quick Pharmacy Market with AI-Assisted Medical Information</u>	khaqan@vu.edu.pk
<u>Online Utility Store Management</u>	ghulam.abbas@vu.edu.pk
<u>Price Watchdog: Autonomous Multi-Platform E-commerce Agent</u>	fazitahir@vu.edu.pk
<u>Prison Management System</u>	shamsu.rehman@vu.edu.pk
<u>Project Title</u>	huma.mumtaz@vu.edu.pk
<u>Real-Time University Transit Tracker System</u>	zaid.ismail@vu.edu.pk
<u>RetailFlow: A Cloud-Integrated Mobile E-Commerce Ecosystem</u>	irshad.nasir@vu.edu.pk
<u>Pakistan Population Data Analysis System</u>	mohsin@vu.edu.pk
<u>Shopify-Based E-Commerce Store Development for Skincare Products</u>	amber.shahzadi@vu.edu.pk
<u>PROJECT TITLE: SKILLSWAP: A PEER-TO-PEER COMMUNITY SKILL EXCHANGE PLATFORM</u>	safid.ullah@vu.edu.pk
<u>SkillSwap – Peer-to-Peer Learning & Skill Exchange Platform</u>	nadiatabassum@vu.edu.pk
<u>Smart Cafeteria Management & Pre-Order System</u>	manahil.hassan@vu.edu.pk
<u>Smart Counselling & Appointment System</u>	komal.saleem@vu.edu.pk
<u>Smart Fleet & Logistics Management System</u>	manwar@vu.edu.pk
<u>Smart Fuel Optimization & Resource Management System</u>	kamran.queshi@vu.edu.pk
<u>Smart Home Maintenance Services</u>	hassan.ali@vu.edu.pk
<u>Smart Package & Subscription Reminder App</u>	fizzah@vu.edu.pk
<u>Smart Plant Care System with AI-Based Monitoring and Mobile App</u>	waqas.ahmad@vu.edu.pk
<u>Smart Resume Screener & Job Board</u>	Muhammad.ilyas@vu.edu.pk
<u>Smart Student Performance & Prediction System</u>	humairanaeem@vu.edu.pk
<u>Smart University Complaint System</u>	mohammad.raheel@vu.edu.p k
<u>SmartGrader – AI Powered Short Answer Evaluation</u>	bilal.saleem@vu.edu.pk

<u>System</u>	
<u>Smart Household and Home Services Management System</u>	asmabatool@vu.edu.pk
<u>Remote Computer Network Monitoring</u>	asimmehmood@vu.edu.pk
<u>Stroke Prediction System using Linear Regression</u>	Tayyaba.sehar@vu.edu.pk
<u>StudyBuddy: Smart Study Planner & Reminder System</u>	rizwana.noor@vu.edu.pk
<u>Vehicle Service Management System using MVC-based Framework</u>	israr.ahmad@vu.edu.pk
<u>Web-based Stock, Inventory and Sales Management System</u>	abdur.rafay@vu.edu.pk
<u>Fake News Detection System Using Python and Django</u>	m.kaleem@vu.edu.pk
<u>Ship Classification from SAR Images Using the NASTaR Dataset</u>	noor.rahman@vu.edu.pk
<u>Online Task Scheduling in Edge Computing using Reinforcement Learning Techniques</u>	zulfiqarmrd@vu.edu.pk
<u>Automated Fire Detection System</u>	madiha.hussain@vu.edu.pk
<u>Automated Helmet detection system using deep learning</u>	laraib.sana@vu.edu.pk
<u>Brain Tumor Segmentation using Res-Net with Flask-Based Visualization</u>	shafaq.nisar@vu.edu.pk
<u>DeepDerm with Grad-CAM Visualization</u>	sonia.salman@vu.edu.pk
<u>Automated Detection, Multi-Stage Severity Grading and Explanation of Diabetic Retinopathy Disease</u>	m.kamran@vu.edu.pk
<u>Fake News Detection using Machine Learning</u>	umairali@vu.edu.pk
<u>Fork After Withholding (FAW) Attack Detection Using Machine Learning</u>	fouziajumani@vu.edu.pk
<u>GRAZE SMASHER PRODUCT HUNTING TOOL</u>	wasif.mairaj@vu.edu.pk
<u>Image Analysis and Tagging Web Application</u>	nidaanwar@vu.edu.pk
<u>Intelligent Diabetes Risk Predictor</u>	komal.khawer@vu.edu.pk
<u>Multilingual AI-Based Framework for Real-Time Semantic Plagiarism and Code Similarity Detection</u>	mehboob.ali@vu.edu.pk
<u>Smart Food Inspection System for Meat Quality Analysis Using Artificial Intelligence</u>	saima.munawar@vu.edu.pk

AI Student Productivity Ecosystem App

Project Domain / Category

Mobile Application

Abstract / Introduction

The proposed system is an AI-powered student productivity application designed to help learners manage study material, improve understanding, and generate personalized learning content. The application integrates multiple AI services to provide smart assistance such as note processing, quiz creation, and interactive tutoring in a single mobile platform.

The system aims to reduce study effort and improve learning efficiency by combining traditional study tools with modern AI capabilities. It will be developed as a scalable Android application and published on the Play Store, targeting students from school to university level.

Functional Requirements:

- The system shall allow users to register and log in using email or social authentication.
- The system shall allow users to upload or scan handwritten/printed notes using camera input.
- The system shall extract text from images using OCR functionality.
- The system shall generate AI-based summaries of uploaded notes using external AI APIs.
- The system shall provide an AI chatbot that answers student questions based on uploaded content.
- The system shall generate quizzes and MCQs automatically from study material.
- The system shall create flashcards for quick revision from uploaded notes.
- The system shall allow users to save and organize notes into folders or subjects.
- The system shall provide a personalized study planner with reminders and schedules.
- The system shall track user learning progress and display performance analytics.
- The system shall support voice input for asking questions and converting speech to text.
- The system shall allow offline access to previously saved notes and summaries.
- The system shall integrate free AI APIs (e.g., Hugging Face or Google AI Studio) for generating responses and content.
- The system shall allow sharing of notes, quizzes, and flashcards with other users.

- The system shall send notifications for daily study reminders and revision tasks.

Tools:

Android Studio
Flutter / Kotlin
Firebase
Hugging Face Inference API
Google AI Studio (Gemini API)
Firestore
Firebase Authentication
Firebase Cloud Messaging
Git
GitHub
Postman
Figma
ML Kit Text Recognition
Node.js

Supervisor:

Name: Bilal Bin Umar
Email ID: bilal.umar@vu.edu.pk
Teams ID: u.bilalumar@live.com

AI-Based Intelligent Child Safety Monitoring and Geofencing-Based Predictive Tracking System

Project Domain / Category

Mobile Application / Location-Based Services / Artificial Intelligence (AI)

Abstract / Introduction

This project proposes an AI-Based Intelligent Child Safety Monitoring and Geofencing-Based Predictive Tracking System. It enables parents to monitor real-time location, receive safety alerts, and analyze movement patterns using AI. The system integrates geofencing, predictive tracking, and anomaly detection to enhance child safety.

Functional Requirements:

FR1: User registration and child profile management.

The system will allow parents to create secure accounts using email or phone authentication. It will also enable adding and managing multiple child profiles with basic details like name, age, and device linkage.

FR2: Real-time GPS tracking using Google Maps API.

The application will continuously track the child's live location and display it on an interactive map. Parents can monitor movement in real time with accurate latitude and longitude data.

FR3: Geofencing for safe zones.

Parents can define virtual boundaries such as home, school, or playground on the map. The system will treat these areas as safe zones for monitoring purposes.

FR4: Alerts for zone entry/exit.

The app will automatically notify parents when a child enters or leaves a defined safe zone. These alerts will help parents stay informed about the child's movement.

FR5: SOS emergency alerts.

A dedicated SOS button will allow the child to send an emergency alert to parents instantly. The alert will include real-time location and notification for quick response.

FR6: Location history.

The system will store past location data of the child for tracking movement patterns. Parents can review daily or weekly travel history when needed.

FR7: Firebase notifications.

Push notifications will be sent using Firebase Cloud Messaging for all important alerts. This ensures real-time communication between the system and parents.

FR8: AI anomaly detection.

The system will analyze movement patterns and detect unusual behavior such as staying too long in unknown places. It will generate alerts based on these anomalies.

FR9: Predictive route analysis.

The app will learn routine paths like home to school and predict expected routes. If the child deviates from the usual path, parents will be notified.

FR10: Voice emergency detection.

The system can detect emergency keywords like help or danger using voice recognition. It will automatically trigger alerts without manual input.

FR11: AI chatbot support.

A simple AI chatbot will assist parents by answering queries about location, history, or alerts. It will provide quick summaries and basic insights.

FR12: User-friendly UI.

The application will have a simple and easy-to-use interface for both parents and children. It will ensure smooth navigation and accessibility for all users.

Tools

Android Studio, Firebase, Google Maps API, SQLite, Retrofit, AI APIs

Supervisor

Name: Shakeel Saeed

Email ID: shakeel@vu.edu.pk

MS Teams ID: shakeelsaeedvu@outlook.com

AI-Based Medical Diagnosis System using NLP & Machine Learning

Project Domain / Category

Web Development, NLP, ML

Abstract

This project introduces an AI-Based Medical Diagnosis System that leverages Natural Language Processing (NLP) and Machine Learning (ML) to assist users in identifying potential diseases based on their symptoms. Users can input symptoms in natural language, which the system processes to extract structured medical information such as symptom type, severity, and duration. These details are then converted into feature vectors and analyzed using ML classification models, including Decision Trees, Naive Bayes, or Random Forests, to generate probable diagnoses with confidence scores. To improve accuracy and interpretability, the system employs a hybrid approach, combining data-driven ML predictions with rule-based logic derived from medical knowledge.

A distinguishing aspect of this system is its explainable outputs, providing users with clear reasoning behind each diagnosis, highlighting influential symptoms and key decision factors. It also offers basic recommendations and maintains a history of past diagnoses for reference. Overall, the project demonstrates the integration of modern AI techniques into healthcare support systems, emphasizing transparency, usability, and structured outputs. Its design makes it particularly suitable for academic purposes, as evaluators can assess its reasoning and predictions efficiently without requiring full system execution.

Functional Requirements

User Roles

- **Patient/User**
 - Enter symptoms in text
 - View diagnosis and recommendations
 - Access history
- **Admin**
 - Manage datasets
 - Monitor model performance
 - Manage symptom dictionary
- **Doctor Role**
 - Validate predictions
 - Add expert rules/feedback

User Management

- The system shall allow:
 - User registration, login, logout
 - Password encryption
- The system shall support:
 - Role-based access control

User Interface

- The system shall display the homepage upon opening the website or application.
- The homepage shall contain a header with the logo, navigation menu (Home, About, Submit Symptoms, History, Admin Login), and optional user profile icon if logged in.
- The homepage shall include a main section with a prominent text input box for users to type symptoms.
- The homepage shall provide placeholder text in the input box guiding users: "Type your symptoms here (e.g., fever, headache, body pain)".
- The homepage shall have optional dropdowns or selection fields for:
 - Severity (Low, Medium, High)
 - Duration (Hours, Days, Weeks)

Symptom Submission

- The homepage shall have a "Diagnose Me" button that submits the user input to the system.
- When the user submits symptoms, the homepage shall validate that the input is not empty.
- If the input is empty, the system shall display a warning message prompting the user to enter symptoms.
- Upon valid submission, the homepage shall redirect the user to the Results Page showing predictions, confidence scores, and explanation.

Natural Language Input Module (NLP Engine)

The system shall accept:

- Free-text input (English)
- The system shall:
- Perform:
 - Tokenization
 - Stop-word removal
 - Lemmatization/stemming
- Extract:
 - Symptoms
 - Duration
 - Severity indicators
- The system shall map extracted text to standard medical terms
- Example:
 - Input: "I have high temperature and severe headache"
 - Output: [fever, headache]
- The system shall:
 - Handle spelling mistakes
 - Suggest corrections

Symptom Normalization & Dictionary

- The system shall maintain:
 - Symptom ontology/dictionary

- The system shall:
 - Map synonyms:
 - “temperature” → fever
 - “stomach pain” → abdominal pain
- Admin shall:
 - Add/edit/remove synonyms

Feature Engineering Module

- The system shall convert symptoms into:
 - Numerical feature vectors
- The system shall include:
 - Binary encoding (present/absent)
 - Converts presence/absence of symptoms into **0 or 1**.
 - Example:
 - fever → 1 (present)
 - cough → 0 (absent)
 - Purpose: Lets ML models distinguish which symptoms are reported.
 - **Severity Weighting**
 - Assigns numeric values to symptom severity.
 - Low → 1, Medium → 2, High → 3
 - Purpose: ML models consider not just presence but **how severe the symptom is**.
 - **Duration Encoding**
 - Converts symptom duration into numeric values:
 - Short → 1, Medium → 2, Long → 3
 - Purpose: Longer-lasting symptoms might have **higher significance** in diagnosis.
 - **Vectorization**
 - Combines all features into a single **feature vector** for the ML model.
 - Example:
 - [fever=1, headache=1, cough=0, severity=3, duration=2]
 - Provides a **structured input** that the ML algorithms can process efficiently

Machine Learning Model Module

- The system shall:
 - Train model on symptom-disease dataset
- Supported models:
 - Decision Tree
 - Naive Bayes
 - Random Forest
- The system shall:
 - Split dataset (train/test)
 - Evaluate accuracy, precision, recall
- The system shall:

- Save trained model

Prediction Engine

- The system shall:
 - Accept processed input
 - Generate top N disease predictions
- The system shall:
 - Output probability scores

Hybrid Logic Layer

- The system shall:
 - Apply rule-based adjustments:
 - Critical symptom override
 - Severity-based prioritization

Example:

If “chest pain” → raise priority

Explainable AI Module (CORE REQUIREMENT ★)

- The system shall:
 - Display:
 - Key symptoms influencing prediction
 - Feature importance
- The system shall generate explanation:

Prediction: Flu (82%)

Reason:

 - Fever → high importance
 - Headache → medium importance
 - Duration → supports viral infection

Output & Visualization Module

- The system shall display:
 - Ranked diseases
 - Confidence scores
 - Extracted symptoms
- The system shall include:
 - Graphs/charts for probabilities

Recommendation Module

- The system shall:
 - Suggest:
 - Precautions
 - Basic remedies
- The system shall include:
 - “Consult doctor” warning for critical cases

History & Reporting

- The system shall:
 - Store all diagnoses
- The system shall generate:
 - Downloadable reports (PDF)
- Report must include:

- Input text
- Extracted symptoms
- Predictions
- Explanation

Admin Dashboard

- The system shall:
 - Upload/manage datasets
 - View:
 - Model accuracy
 - Usage statistics

Error Handling

- The system shall:
 - Handle ambiguous input
 - Prompt user for clarification
 - The system shall:
 - Prevent empty submissions

Tools

Frontend: React.js / Vue.js, HTML, CSS, JavaScript

Backend: Python (Flask / Django)

AI/ML: scikit-learn, TensorFlow, NLP (spaCy / Transformers)

Database: MySQL / MongoDB

Supervisor

Name: Amna Bibi

Email ID: amna.bibi@vu.edu.pk

Teams ID: aamna.bibi26@outlook.com

An Interactive Web Portal for Keeping records of Diabetes-Affected Patients in Pakistan”

Project Domain: **Web Application**

Abstract:

This project presents the design and development of an online web portal aimed at maintaining comprehensive records of diabetes-affected patients in Pakistan. The system is intended to provide a centralized, secure, and user-friendly platform for healthcare professionals to store, access, and manage patient data efficiently. By digitizing medical records, the portal minimizes manual paperwork, reduces the risk of data loss, and enhances the accuracy of patient information. It also facilitates real-time data updates, enabling doctors, clinics, and hospitals to monitor patient history, treatment plans, and progress more effectively.

Furthermore, the proposed web portal aims to support better healthcare decision-making and improve patient outcomes by enabling data analysis and reporting features. Patients can benefit from improved continuity of care, while healthcare providers gain insights into trends and patterns related to diabetes across different regions of Pakistan. The system emphasizes data privacy and security, ensuring that sensitive medical information is protected. Overall, this project contributes to strengthening the healthcare infrastructure by leveraging digital technology to address the growing challenge of diabetes management in the country.

On one side, the portal will serve as a one-stop platform to support better healthcare decision-making and on other side the social welfare organizations both in government as well as in non-government organizations can use this portal for supporting individuals who cannot fully care for themselves.

Functional Requirements:

End-User:

- The end-user visits the web portal through domain name.
- The user will first sign-up and then Sign-in.
- The user searches a particular record on the basis of unique Id.
- The user can view his/her medical history, treatment plan, care precautions and next appointment date.
- The user can leave feedback for the concerned doctor.

Doctor/Health care Unit:

- The doctor/healthcare unit will register to the portal first by providing the requisite information on form for approval.
- After Admin approval, the doctor/healthcare unit can login and upload the relevant information about patient like patient’s medical history, number of

examinations and dates, treatment plan, care precautions and progress remarks etc.

- The doctor can also update the patient's treatments plan, next examination date, care precautions etc.
- Provide relevant information for contact or appointment.

Admin:

- The admin will register to the portal by providing the requisite information and login to the system.
- The admin will check the Doctor profile/healthcare unit and will grant approval/rejection accordingly.
- The admin will give feedback to the doctor/healthcare unit, if needed.
- The admin will generate various reports at different levels showing the trend and pattern of diabetes in country or some in regions.
- Manage content (upload/edit) if required.

Tools:

For Front-End:

Html, CSS, JavaScript, bootstrap

For Backend:

PhP, My Sql

Supervisor Name: MIR SALAM KHAN

Email ID: mir.salam@vu.edu.pk

Teams ID: mir.salam@outlook.com

Android-Based Smart Electricity Consumption Predictor

Project Domain / Category:

Machine Learning

Abstract / Introduction:

This project develops an Android-based application to predict household electricity consumption using a hybrid approach that combines a pre-trained machine learning model with user historical data. The system collects appliance details, such as power ratings and daily usage hours, and incorporates past electricity consumption to improve prediction accuracy. Predictions are used to estimate electricity bills based on local tariff rates and provide intelligent recommendations to help users optimize energy usage and reduce costs. By combining predictive analytics with a user-friendly mobile interface, the application promotes energy efficiency and supports informed decision-making for household electricity management.

Functional Requirements:

I. User Authentication

- Users can create accounts and log in to save their data and predictions.

II. Input for Appliances

- Users can enter appliance details: name, power rating (Watts), daily usage hours.
- Multiple appliances can be added, edited, or removed.

III. Historical Data Input

- Users can enter past electricity consumption (monthly units or bills).
- The app can calculate trends from this historical data.

IV. Electricity Consumption Prediction

- The app predicts electricity usage using a pre-trained ML model.
- Predictions are adjusted using user historical data to improve accuracy.

V. Bill Estimation

- Calculates estimated electricity bills based on predicted consumption and local tariff rates.
- Provides total cost and appliance-wise breakdown.

VI. User Interface

- Interactive screens for data entry, prediction results, and visualization.
- Easy navigation between manual input, historical data, and results.

VII. Visualization

- Displays consumption trends using graphs/charts (past vs predicted).
- Highlights high-consumption appliances for user awareness.

VIII. Energy-Saving Recommendations

- Suggests tips to reduce electricity usage based on predicted and actual consumption.

IX. Data Persistence

- Stores appliance input and historical data locally (SQLite/Room DB).
- Allows cloud backup (e.g., Firebase Real Time Database / Cloud Firestore).

X. Error Handling & Validation

- Validates user input (numeric values for Watts, hours, units).
- Displays meaningful error messages for incorrect input.

Workflow:

The basic workflow of this project is as follows.

1. Project Setup

- Configure PyCharm for ML model training and Android Studio for app development.
- Use Firebase services for authentication and cloud storage.

2. Dataset Preparation

- Collect and preprocess public electricity consumption data.
- Handle missing values and split into training, validation, and test sets.

3. ML Model Development

- Train and fine-tune a machine learning model (Linear Regression / Random Forest).
- Export the model to TensorFlow Lite for Android deployment.

4. Android App Implementation

- Collect appliance details (power rating, usage hours) and user historical data.
- Integrate the pre-trained ML model and adjust predictions using historical trends.
- Display predicted consumption, estimated bill, graphs, and energy-saving suggestions.

Tools:

1. **IDE:** Android Studio & PyCharm
2. **Programming Language:** Java/Kotlin & Python
3. **Databases:** Firebase Real-Time/Cloud Firestore & SQLite/Room (for local storage)
4. **Dataset:** Kaggle Electricity Consumption Dataset
(e.g., <https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption>)
5. **ML Model:** Linear Regression / Random Forest
(e.g., https://scikit-learn.org/stable/modules/linear_model.html / <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>)
6. **Model Deployment:** LiteRT i.e., TensorFlow Lite
(<https://ai.google.dev/edge/litert>)

Note:

1. This is an **Android-only development project**, so tools like Flutter or Dart for cross-platform development are **NOT** allowed.
2. Any **relevant dataset and ML model** can be implemented depending on project requirements and availability.
3. VU will **NOT** pay for any software/library/toolkit/API etc. used in this project.

Supervisor:

Name: Muhammad Imran Afzal

Email ID: imran.afzal@vu.edu.pk

MS Teams ID: [jmranafzal126@outlook.com](https://www.outlook.com/teams/jmranafzal126)

“Answer Question” Community, Learning and Knowledge Platform
AQDesk

Project Domain: Web Application

Introduction

AQDesk (Answer Question Desk) is a digital community and knowledge platform designed to support learning, collaboration, and professional development in the fields of Emerging Technologies.

The objective of this project is to design and develop a full-featured web platform that integrates community discussion, project collaboration, consulting services, and opportunity sharing.

The platform will enable students, instructors, researchers, technology professionals, and policy experts to interact, share knowledge, and build collaborative projects.

Students will design and implement the **AQDesk portal** as a scalable web application that demonstrates modern web development, learning technology integration, and community platform design.

Focus

A fully accessible web-based application, ensuring equal participation and usability for all users.

Core Objectives

AQDesk Portal is an online community platform designed to enable students, educators, technology professionals, and experts to share knowledge, collaborate on projects, discuss emerging technologies, and access professional opportunities within a single digital ecosystem.

Functional Requirements

FR 1. User Management

Features:

- User registration and authentication
- Role-based access control
- Profile management

Authentication Methods:

- Email login
- Social login

FR 2. Community Q&A System

A discussion system similar to a technical knowledge community.

Features:

- Ask questions

- Post answers
- Upvote answers
- Tag topics

Categories may include:

- EdTech
- Artificial Intelligence
- Web Development
- Internet Governance

FR 3. Project Collaboration

Students and community members should be able to collaborate on projects.

Features:

- Create project spaces
- Invite collaborators
- Share files
- Provide feedback

FR 4. 5 Events and Podcast Management

The platform should support educational events and podcasts.

Features:

- Event creation
- Event registration
- Webinar integration
- Podcast publishing

FR 5. 6 Jobs and Opportunities Board

A dedicated space for career development.

Features:

- Job postings
- Internship listings
- Application submission

FR 6. 7 Consulting Management System

A module to manage consulting services.

Features:

- Client management
- Consultation booking
- Proposal tracking

FR 7. 8 Newsletter and Announcements

Features:

- Newsletter subscription
- Email campaign management
- Community updates

Technical Skills Required

Describe your experience and comfort level in the following areas:

Backend PHP (Laravel Framework preferred)

Frontend JavaScript Framework(Vue.js/Bootstrap)

Database MySQL

Cloud Deployment (AWS,Digital Ocean,Cpanel)

Version Control Git / GitHub

Deliverables

Final Report: Covering problem analysis, design, implementation, and evaluation.

User Manual/Guide: Instructions for form creators and respondents.

Testing Report: Usability testing with accessibility validation (WCAG compliance checks).

GitHub Repository: Complete source code with documentation.

Live Demo or Recorded Presentation: Demonstrating accessibility features in action.

Note

Students should consult with their supervisor before finalizing system architecture and implementation approach. The objective of this project is not only software development but also demonstrating the integration of education technology, community knowledge systems, and digital governance concepts.

Note: All official communication with the instructor should be carried out via **Google Chat** for timely coordination and feedback.

Instructor: Mr. Abdullah Qamar

Email: abdullah.qamar@vu.edu.pk

MS Teams ID: qamarabdullah@hotmail.com

Campus Ease - Smart Lost & Found System for University Campuses

Project Domain / Category

Web Application

[Abstract / Introduction]

Students and staff often lose personal items (ID cards, phones, keys, bottles, laptops) on campus. There is no efficient way to report lost items or return found ones. Current methods like WhatsApp groups, notice boards, and security registers are slow and scattered.

CampusEase is a web-based Smart Lost & Found System. It allows users to report lost items, log found items with photos and locations, and receive automatic match suggestions. After a potential match, users can chat securely to verify ownership.

[Actors & Their Roles]

1. Normal User (Students & Faculty)

- Register with university email. Report lost items. Browse/search found items. Submit claim requests with verification answers. Chat anonymously after claim approval. Flag fake reports. Mark resolved items. Renew expired lost reports.

2. Security / Admin

- Log found items. Update found item status. Approve/reject claims. Monitor all reports and dashboard (statistics, hotspots). Remove fake reports. Export PDF/Excel reports. Move unclaimed items to Donated/Disposed after 60 days.

3. Super Admin

- All Security/Admin permissions. Manage user roles. View system logs. Configure system settings (match sensitivity, expiration timelines). Override disputes. Access full analytics.

[Functional Requirements]

1. Authentication & Authorization

- University email-based registration.
- Three roles: Normal User, Security/Admin, Super Admin.

2. Lost Item Reporting

- Submit: item name, category, color, brand/model, date/time, location, optional photo, description.
- Status: Pending → Matched → Resolved → Closed.
- User can mark as resolved.

3. Found Item Logging

- Submit: item details, multiple photos, storage location.
- Status: Unclaimed → Claim Pending → Claimed → Transferred to Admin.

4. Smart Matching Engine

- Automatic matching by keywords, color, location proximity, date range.
- Displays match confidence (%).
- Manual search/filter by category, location, date.

5. Claim & Verification

- User submits claim request.
- Claimant answers verification questions.
- Security/Admin approves claim.
- Claim status: Pending Approval → Approved → Rejected → Completed.

6. Secure Communication

- Temporary anonymous chat after claim approval.
- Personal contact info hidden until both parties agree.

7. Notification System

- In-app alerts for matches, claim requests, status changes, daily/weekly digests.

8. Dashboard & Analytics (Admin)

- Overview: active reports, pending claims, resolution rate.
- Most commonly lost items.
- Hotspot analysis (by location).
- Export PDF/Excel reports.

9. Moderation & Safety

- Users can flag fake/inappropriate reports.
- Lost reports expire after 30 days (renewable).
- Unclaimed found items → Donated/Disposed after 60 days.

Tools:

HTML, CSS, JavaScript, Bootstrap 5 (Front-end)

MySQL / phpMyAdmin (Database)

PHP (Laravel Framework) or Python (Django/Flask) - Backend

XAMPP / WAMP (Web Application Server)

Supervisor:

Name: Ehsan ul Haq

Email ID: Ehsan.haq@vu.edu.pk

MS Teams ID: ehsan_mtn@outlook.com

Charity Management System for Donation and Request Handling

Project Domain / Category

Web Application

Abstract / Introduction

The Charity Management System (CMS) is a web-based application designed to streamline and digitize the process of charity distribution. In many societies, especially in developing regions, charity is often unmanaged, non-transparent, and inefficient. Donors are unable to find genuine needy individuals, while deserving people struggle to access available resources.

This system aims to bridge the gap between donors and needy individuals by providing a centralized, transparent, and user-friendly platform where:

- Donors can contribute money or goods (including old items)
- Needy individuals can request specific help directly
- Admin ensures authenticity and proper management

Functional Requirements:

Admin Module:

1. Admin can log in securely to the system.
2. Admin can register and manage donors (approve, block, update details).
3. Admin can register and verify needy individuals after checking their authenticity (CNIC, documents, etc.).
4. Admin can define and manage charity types (Financial aid, Food items, Clothes, Old household items).
5. Admin can view all donation requests and approve or reject them.
6. Admin can monitor all transactions and activities in the system.
7. Admin can assign donations to the most suitable needy individuals if needed.
8. Admin can resolve disputes or complaints between donors and recipients.
9. Admin can generate reports (Total donations, Active donors, Completed charity requests)
10. Admin can send notifications/alerts to users.

Donor Module:

1. Donor can register and create a profile.
2. Donor can log in/logout securely.
3. Donor can add donation details, including: Type of charity (money, clothes, food, old items) Description of items, Quantity, Location.
4. Donor can view requests from needy individuals.
5. Donor can accept or reject requests from needy persons.
6. Donor can track donation history.

7. Donor can schedule pickup or delivery of items.
8. Donor can communicate with needy individuals.
9. Donor can rate/feedback the experience.

Needy Person Module:

1. Needy person can register and create a profile. Must provide complete details (Name, Address, Income status, Family details, Supporting documents)
2. Needy person can browse available donations.
3. Needy person can request specific items or financial help.
4. Needy person can send direct requests to donors.
5. Needy person can track request status (pending, accepted, rejected).
6. Needy person can receive notifications when a donor responds.
7. Needy person can provide feedback after receiving help.

Donation Management Module:

1. System stores all donation records.
2. Categorization of donations based on: Type, Location, Urgency.
3. Matching system between: Donor offer and Needy requests.
4. Status tracking: Available, Requested, Approved, Delivered.
5. History of completed donations.

Tools:

Front-end

HTML, CSS, JavaScript, Bootstrap

Back-end

PHP, MySQL

Development Tools

Visual Studio Code, XAMPP/WAMP, Git & GitHub

Optional (Enhancements)

Chart.js, Leaflet.js

Supervisor:

Name: Ammara Rasheed

Email ID: ammara.rasheed@vu.edu.pk

MS Teams ID: ammara.rasheed@outlook.com

Citizen Issues Management System

Project Domain / Category

Web Application

Abstract / Introduction

This project is a web-based application that enables citizens to report public issues such as road damage, garbage, water shortages, electricity faults, and corruption complaints directly to government departments. The system allows users to submit complaints with location, images, and descriptions, while authorities can track, manage, and resolve these issues through an admin dashboard. It includes status tracking, prioritization, and analytics to improve response efficiency. The platform promotes transparency, accountability, and faster resolution of public problems using modern web technologies and database management.

Functional Requirements:

User (Citizen)

- Register, login, logout, and reset password
- Submit complaint with:
 - Title and description
 - Category (Roads, Electricity, Water, Sanitation, Others)
 - Location (text or map coordinates)
 - Optional image upload
- View and track complaint status:
 - Submitted → Under Review → Assigned → In Progress → Resolved / Rejected
- View complaint history with filters (status, category, date)

Super Admin (Central Authority)

- Create, update, and delete departments
- Create and manage Department Admin accounts
- View all complaints across the system
- Monitor overall system statistics (total, pending, resolved complaints)

Department Admin

- View complaints assigned to their department
- Verify complaints:
 - Approve (valid) → move to assignment
 - Reject (invalid) with reason
- Assign approved complaints to Field Officers with deadline
- Set or update priority (High / Medium / Low)
- Update complaint status (Under Review, Assigned, In Progress, Resolved)
- Monitor field officer performance (tasks completed, pending, time taken)

Field Officer

- View assigned tasks
- Accept or reject task (with reason)

- Update task progress:
 - Accepted → Work Started → In Progress → Completed
- Upload completion proof (image required, video optional)
- Add remarks/comments

Analytics Dashboard

- View complaints by category and area
- Track:
 - Total, pending, resolved complaints
 - Average resolution time
 - Most frequent issues

Tools:

- Python (Django or Flask framework) with PostgreSQL or MySQL database
- PHP (custom development or any framework such as Laravel) with MySQL database
- .NET Stack (ASP.NET C#) with Microsoft SQL Server database

Supervisor:

Name: Muhammad Ahmad Lodhi

Email ID: ahmadlodhi@vu.edu.pk

MS Teams ID: ahmadlodhi.vu@outlook.com

Collaborative Sharing of Bulk Items Purchasing System

Project Domain / Category

Web Application

Abstract / Introduction

The **Collaborative Sharing of Bulk Items Purchasing System** is a web-based platform designed to enable users to collaboratively purchase products in bulk and benefit from discounted pricing. In many markets, suppliers offer lower prices for large quantity purchases; however, individual buyers are often unable to meet these requirements. This system solves this problem by allowing multiple users to combine their purchasing needs into a single bulk order.

The platform organizes products into well-defined categories such as groceries, electronics, household items, personal care products, and office/stationery items etc. This categorization helps users easily browse, search, and participate in relevant bulk purchasing groups.

Users can register, create profiles, and browse categorized products. They can either create or join bulk purchasing groups where each participant contributes a specific quantity toward a predefined target. The system tracks the total quantity in real time and applies dynamic discounts once the required threshold is achieved. A secure individual payment system is integrated, where each user pays separately for their selected quantity using online payment methods. Payments are processed transparently, and users can track their payment status. The platform also includes features such as notifications, messaging between users, and order tracking. An admin panel allows management of products, categories, users, and discount rules, ensuring smooth system operation.

Functional Requirements

1. User Authentication
 - User registration, login, and profile management
2. Product Management
 - View product details
 - Admin can add, update, and delete products
3. Product Categories Management
 - Products are organized into categories (e.g., groceries, electronics, household items) and users can browse and filter products by category
 - Admin can manage categories
4. Bulk Group Creation
 - Users can create bulk purchase groups
 - Define required quantity and deadline
5. Join Bulk Orders
 - Users can join existing bulk groups
 - Add their required quantity
6. Dynamic Pricing System
 - Apply discounts based on total quantity

- Display updated prices in real time
- 7. Individual Payment System
 - Each user pays separately for their selected quantity using online payment methods
 - Payment status tracking (Pending, Paid, Refunded)
- 8. Order Management
 - Confirm orders after successful payments
 - Track order status and delivery
- 9. Notifications System
 - Alerts for group completion, payments, and deadlines
- 10. Messaging System
 - Communication among group members
- 11. Admin Panel
 - Manage users, products, categories, discounts, and transactions

Tools

Front End: HTML, CSS, JavaScript, bootstrap, jquery, ajax

Back End: PHP (Laravel or Codeigniter framework)

DBMS: MySQL

Supervisor:

Name: Umra Naeem

Email ID: umra.naeem@vu.edu.pk

MS Teams ID: umra.naeem@outlook.com

Community Food Wastage Reduction Portal

Project Domain / Category

Web Application

Abstract / Introduction

In urban environments, a significant amount of surplus food from restaurants, event planners, and households is discarded while local shelters and NGOs struggle to meet the needs of the underprivileged. This project aims to develop a centralized digital bridge that connects food donors with recipients in real-time. By automating the listing and claiming process, the portal eliminates the logistical delays that often lead to food spoilage. The system facilitates the efficient redistribution of surplus resources, turning potential waste into a valuable social asset through community collaboration.

Functional Requirements

The application should be able to:

- User Management: Provide secure registration and login for three distinct roles: Donors (Restaurants, Hotels, Individuals), Recipients (NGOs, Shelters, Volunteers), and System Admin.
- Donation Listing: Enable donors to upload surplus food details, including item names, food category (Veg/Non-Veg), quantity (kg or servings), and "Best Before" timestamps.
- Interactive Map/Search: Allow recipients to search for available food donations based on their current geographic location or city.
- Claiming System: Implement a "Claim" feature where an recipient can reserve a listed donation, preventing multiple organizations from attempting to pick up the same item.
- Status Tracking: Provide a real-time workflow to track the donation status from "Available" to "Claimed" and finally "Delivered".
- Notification Engine: Send automated alerts (email or dashboard notifications) to nearby NGOs whenever a new donation is posted in their area.
- Feedback & Rating: Allow recipients to provide feedback on the quality and hygiene of the food received to maintain system trust.

Inputs

- Donor Side: Food description, photos (via Firebase or local storage), pickup address (GPS-integrated), quantity, and expiry time.
- Recipient Side: Verification documents (for NGOs), pickup requests, and "delivered" status confirmations.
- Admin Side: Verification status for new donors/NGOs and management of food categories.

Expected Outputs

- Donor Dashboard: A summary of total food donated, impact metrics (e.g., "150 meals saved"), and current active listings.
- Recipient Dashboard: A prioritized list of available food nearby and a history of successful pickups.
- Admin Reports: Graphical analytics showing city-wise donation trends, total waste reduced, and number of people fed.
- Real-time Alerts: Instant dashboard notifications for matching requests or successful deliveries.

Tools

- Frontend: HTML5, CSS3 (Tailwind CSS or Bootstrap), React.js or Vue.js.
- Backend: Node.js (Express), Python (Flask/Django), or PHP (Laravel).
- Database: MySQL, PostgreSQL, or MongoDB (for flexible listing attributes).
- Additional APIs: Google Maps API (for location mapping) and Firebase (for real-time image storage and database sync).

Supervisor

Name: Hasnain Ahmed

Email ID: hasnain@vu.edu.pk

Microsoft Teams ID: tohasnain.bukhari@outlook.com

Smart Query Routing & Email Automation System

University Intelligent Communication Hub

Project Domain / Category

AI Automation / Full Stack Web Development

Abstract / Introduction

Universities receive hundreds of emails and queries daily from students, instructors, and departments. The current manual process of reading, sorting, and replying to these queries is time-consuming, error-prone, and inefficient. Students frequently do not know which department or person to contact, leading to misdirected emails, delayed responses, and frustration.

This project proposes the development of a Smart Query Routing and Email Automation System an AI-powered, role-based communication hub that automatically classifies incoming queries, routes them to the appropriate person or department, drafts intelligent replies using AI, and tracks resolution status from a unified dashboard. The system requires zero cost, using only free-tier tools and APIs, making it perfectly suitable for students with limited resources.

Functional Requirements:

- FR-01: Role-based login with Google OAuth2 authentication (All Roles)
- FR-02: Multi-channel query submission via email, web form, and WhatsApp (Student)
- FR-03: AI-powered query classification and intent detection (System)
- FR-04: Automatic routing to correct department or instructor (System)
- FR-05: AI-generated draft reply suggestions for staff (All Staff)
- FR-06: Real-time query status tracking for students (Student)
- FR-07: 24-hour auto-escalation for unresolved queries (System)
- FR-08: Email and WhatsApp notification system (All Roles)
- FR-09: HOD escalation and override capability (HOD)
- FR-10: Admin analytics dashboard with charts and reports (Admin)
- FR-11: Broadcast announcement to students (Admin)
- FR-12: Export query reports to PDF and Excel (Admin / HOD)
- FR-13: Instructor leave and auto-reply mode (Instructor)
- FR-14: Query history and audit log for all system activity (Admin)

Tools:

Development Environments / IDEs:

- VS Code — Primary development IDE for Python and JavaScript
- PyCharm Community Edition — Python backend development
- Postman — API testing and endpoint verification

Languages & Frameworks:

- Python 3.11+, JavaScript (ES6+), HTML5 / CSS3, SQL (PostgreSQL)
- FastAPI (backend), React.js (frontend), Tailwind CSS (styling)
- LangChain (AI orchestration), SQLAlchemy (ORM), Pydantic (validation)
- Recharts / Chart.js (analytics visualizations)

AI & Integration Tools:

- Google Gemini API (free tier) — query classification and reply drafting
- Claude API by Anthropic (free tier) — complex query understanding
- n8n (self-hosted) — visual workflow automation engine
- Gmail API — read and send emails via Google's official API
- Google OAuth2 — secure passwordless login
- Baileys Library (JS) — WhatsApp bot integration
- Google Sheets API — activity logging and reports

Database & Hosting:

- PostgreSQL / Supabase (free tier) — primary relational database
- Render.com (free tier) — FastAPI backend hosting
- Vercel (free) — React frontend deployment

Supervisor:

Name: Saima Jamil

Email ID: Saima.jamil@vu.edu.pk

Teams ID: saima.jamil1988@outlook.com

CyberNova Solutions

Domain / Category

Web Based

Introduction

The proposed project is CyberNova Solutions, a professional e-commerce platform for purchasing AI-driven cybersecurity software, tools, and subscription services. It provides a secure and intelligent shopping experience, enabling users to browse, filter, and purchase products such as network monitoring software, AI security modules, and digital protection subscriptions.

The platform emphasizes cybersecurity, offering secure user registration, encrypted transactions, optional two-factor authentication, and profile management. AI features enhance functionality through personalized product recommendations and a Cybersecurity Risk Analyzer, which provides a Digital Security Score to help users identify and mitigate risks. Users can also interact with a secure AI assistant or expert consultant for guidance. Administrators benefit from a comprehensive dashboard to manage users, products, orders, and AI module.

Functional Requirements

User Requirements:

- **Secure User Registration & Login** (email/phone verification, hashed passwords, optional 2FA)
- **Product Browsing & Search** (AI tools, network monitoring software, cybersecurity subscriptions)
- **Product Filtering & Sorting** (by type, license, AI capability, security level)
- **Shopping Cart & Checkout** (secure payment integration, optional crypto payments)
- **Order History & Tracking**
- **AI-based Product Recommendations** (Python AI module integrated via API)
- **Cybersecurity Risk Analyzer** (AI module calculates a **Digital Security Score**)
- **Secure Messaging System** (chat with AI assistant or expert consultants)
- **Profile Management** (edit personal details, manage licenses, subscriptions)

Admin Requirements:

- **Admin Dashboard** (manage users, products, orders, AI modules, alerts)
- **Product Management** (add/edit/delete software, AI modules, subscriptions)
- **Order Processing & Status Management**
- **Customer Support & AI Assistant Monitoring**
- **Sales & Analytics Dashboard** (top-selling products, AI-driven trends)
- **AI-based Fraud Detection** (monitor suspicious logins, payments, or fake orders)
- **Role-Based Access Control (RBAC)** for admins, support, and consultants

Cybersecurity Features

- Secure authentication & 2FA
- SQL Injection & XSS Prevention

- CSRF Protection
- SSL/TLS encryption for all data transmission
- RBAC for admin & support panel
- Regular security monitoring & logs
- AI-powered anomaly detection for transactions, subscriptions, and suspicious logins
- Automated security audit alerts for purchased software & licenses

AI Integration Features

- **AI Product Recommendations** (Collaborative & Content-Based Filtering)
- **Cybersecurity Risk Analyzer** (classification & scoring for digital networks and systems)
- **Fraud Detection System** (supervised/unsupervised anomaly detection)
- **AI Chatbot / Virtual Consultant** (NLP-based for guidance, setup, and threat prevention)
- **Predictive Threat Alerts** (detect network vulnerabilities or system risks before attacks)
- **Trend Analytics** (AI predicts which tools and subscriptions are most effective for different user types)

Tools & Technologies

- **Frontend:** HTML, CSS, Bootstrap, JavaScript
- **Backend:** PHP (Laravel recommended for secure development)
- **Database:** MySQL
- **Security Measures:** SSL/TLS, CSRF Protection, XSS & SQL Injection Prevention, Password Hashing, RBAC
- **AI Integration:** Python AI modules for recommendations, risk analysis, and fraud detection; Chatbot APIs (Dialogflow or ChatGPT API)

Helping Material:

Learn HTML and CSS basics:

<https://www.youtube.com/watch?v=HzczuZ7YJek&list=PLgnhbHE4H9fW4L1cW-V1VPISzKndo8lQR>

Learn PHP and SQL basics

https://www.youtube.com/watch?v=YijHzhBwUwg&list=PLgnhbHE4H9fX_idbehsbcn5_2o4HBcu1X

Cybersecurity Techniques & Implementation Guidance

1. **Secure User Registration & Password Hashing**
 - **OWASP:** [Password Storage Cheat Sheet](#)
(Guidelines for hashing algorithms like bcrypt, Argon2, PBKDF2 to store user passwords securely)
2. **SQL Injection & XSS Prevention (Input Validation & Secure Coding)**

- **OWASP:** [SQL Injection Prevention Cheat Sheet](#)
- **OWASP:** XSS Prevention Cheat Sheet
- 3. **CSRF Protection**
 - **OWASP:** [CSRF Prevention Cheat Sheet](#)
- 4. **SSL/TLS & Secure Transmission**
 - **Mozilla:** [TLS Configuration Generator](#)
(Best practices for HTTPS, TLS versions, and certificates)
- 5. **Role-Based Access Control (RBAC)**
 - **NIST:** [RBAC Reference](#)
(Guidance for designing secure access models for admins, support, and consultants)
- 6. **Security Logging & Monitoring**
 - **OWASP:** [Logging Cheat Sheet](#)
(How to log activities securely and detect suspicious behavior without exposing sensitive data)

AI Techniques & Implementation Guidance

1. **AI-Based Product Recommendations (Recommendation Systems)**
 - **Real Python:** [Build a Recommendation Engine with Collaborative Filtering](#)
 - **Analytics Vidhya:** Comprehensive Guide to Recommendation Engines in Python
2. **Fraud Detection with AI**
 - **Codementor:** Machine Learning in Fraud Detection – A Simple Guide
3. **AI Chatbots (NLP-based)**
 - **ProjectPro:** Python Chatbot Project – Build a Chatbot from Scratch
4. **Integrated Example: AI + Fraud Detection + Chatbot**
 - **Medium:** LLM-Powered Fraud Detection Chatbot with Python

Supervisor:

Name: Sohail Aamir

Email ID: sohailaamer@vu.edu.pk

MS Teams ID: [sohailaamir24@outlook.com](#)

Design and Development of the FashionHub Web Application

Project Domain / Category

Web Application

Abstract / Introduction

FashionHub is a modern web-based digital marketplace designed to connect fashion suppliers and customers on a centralized platform managed by an administrator. The system provides a seamless and user-friendly environment where suppliers can upload and manage their fashion products, while customers can explore, compare, and purchase items. This platform specifically focuses on fashion and lifestyle products, aiming to enhance online shopping experiences with advanced filtering, secure transactions, and interactive features such as reviews and wish list. The system includes three main roles: **Admin, Supplier, and Customer**, each having specific functionalities. The admin oversees the entire platform, suppliers manage their products and performance, and customers enjoy a smooth shopping journey.

Functional Requirements:

1. User Registration & Profile Management

- Users can create accounts and log in securely.
- Roles include **Admin, Supplier, and Customer**.
- Users can update profile information.

2. Product Management System

- Suppliers can **add, update, and delete products**.
- Each product includes:
 - Name
 - Category
 - Price
 - Description
 - Stock Availability
 - Images

3. Product Categories

- Products will be divided into categories such as:
 - Fashion (Clothing, Shoes, Accessories)
 - Beauty & Lifestyle
 - Seasonal Collections

4. Search & Filtering

- Users can search products by:
 - Name
 - Category
 - Price range
- Advanced filtering options for better user experience.

5. Order Management

- Customers can:
 - Place orders
 - View order history
 - Track order status (Pending, Shipped, Delivered)

6. Payment System

- Cash on Delivery (COD)

7. Shopping Cart & Wishlist

- Users can add/remove items from cart.
- Wishlist feature for saving favorite products.

8. Ratings & Reviews

- Customers can give ratings and reviews.
- Admin will approve reviews before publishing.

9. Reports & Analytics

- Admin and suppliers can view:
 - Sales reports
 - Product performance
 - Order statistics

10. Supplier Performance Tracking

- Suppliers can monitor:
 - Sales performance
 - Product engagement

11. Notifications & Alerts

- System will send notifications for:
 - Order confirmation
 - Status updates
 - Important alerts

Nonfunctional Requirements:

- **Security:** The solution will implement proper security measures to ensure that customer and bakery owner data is protected.
- **Performance:** The system will be designed to handle many concurrent users and transactions, ensuring that the platform is always available and responsive.
- **Scalability:** The system should be easily scalable to accommodate new features and functionalities as the business grows.
- **Usability:** The solution will be designed to be user-friendly and intuitive, ensuring that customers and bakery owners can easily navigate the system and complete their tasks.

Tools & Technologies

HTML, CSS, JavaScript, Bootstrap, PHP (Laravel) / Node.js / React / Angular

Database: MySQL / MongoDB/PostgreSQL/SQL Lite

Supervisor:

Name: Muhammad Qamar Usman

Email ID: qamar@vu.edu.pk

MS Teams ID: usman_qamar09@outlook.com

Digital Mandi – Smart Agricultural Marketplace Platform

Project Domain / Category

Web & Mobile Application Development

Abstract / Introduction

Agriculture is a major contributor to the economy of Pakistan, yet farmers often face challenges such as unfair pricing, dependence on middlemen, and limited access to broader markets. Traditional mandi systems require physical presence, making the process time-consuming, inefficient, and sometimes exploitative for farmers.

The proposed **Digital Mandi** platform aims to digitize agricultural trading by connecting farmers directly with buyers through an online system. The platform will allow farmers to list their products, set prices, and interact with buyers without intermediaries. Buyers will be able to search, compare, and purchase agricultural goods easily. This system will improve transparency, efficiency, and profitability in the agricultural supply chain.

Functional Requirements:

Provide a bulleted list of functional requirements

1. **FR1 – User Registration & Authentication**
 - Users (Farmers, Buyers) can register and log in securely
 - Role-based access control for different users
2. **FR2 – User Profile Management**
 - Users can update personal details (name, contact, location)
 - Farmers can add farm-related information
3. **FR3 – Product Listing Management (Farmer Side)**
 - Add new products with details (name, category, price, quantity, location)
 - Upload product images
 - Edit or delete existing listings
4. **FR4 – Product Browsing & Search (Buyer Side)**
 - Browse all available products
 - Search by keyword, category, or location
 - Apply filters (price range, availability, freshness)
5. **FR5 – Order Placement System**
 - Buyers can place orders for selected products
 - Select quantity and confirm purchase
 - Generate order summary
6. **FR6 – Order Management**
 - Farmers can view incoming orders
 - Accept/reject orders
 - Update order status (pending, confirmed, delivered)
7. **FR7 – Communication Module**
 - Chat or messaging system between buyer and farmer

- Option to share contact details
- 8. **FR8 – Payment Handling (Basic Level)**
 - Support Cash on Delivery (COD)
 - Optional integration with digital payment methods
- 9. **FR9 – Notification System**
 - Notify users about new orders, updates, or messages
 - Alerts for order confirmation and delivery
- 10. **FR10 – Admin Dashboard**
 - Manage users (approve/block accounts)
 - Monitor product listings
 - Handle complaints and disputes
- 11. **FR11 – Review & Rating System**
 - Buyers can rate products and sellers
 - Feedback system for service improvement
- 12. **FR12 – Location-Based Services**
 - Display nearby products using user location
 - Help buyers find local farmers easily
- 13. **FR13 – Data Storage & History**
 - Maintain order history
 - Store transaction records for users

Tools:

- **Programming Language:** JavaScript / Python / PHP
- **Frontend:** HTML, CSS, JavaScript, React.js (optional)
- **Backend:** Node.js / Django / PHP (Laravel)
- **Database:** MySQL / MongoDB
- **Development Environment / IDEs:** Visual Studio Code
- **Version Control:** Git
- **Deployment (preferred):** Firebase

Supervisor:

Name: Muhammad Tahir Jan

Email ID: tahir.jan@vu.edu.pk

MS Teams ID: mtahirjan@outlook.com

EduVault Library Management System

Project Domain / Category

Web Programming

Abstract / Introduction

EduVault is a web-based smart academic library management system designed to digitize traditional library operations. It provides an efficient platform for managing book records and user interactions, ensuring improved accessibility and operational control.

The system enables librarians to perform core functions such as managing book inventory, issuing books and tracking borrowing activities in real time. An automated fine calculation mechanism is integrated to handle overdue cases accurately, minimizing manual effort and reducing the chances of human error. The admin module provides supervisory control over the system by allowing monitoring of borrowing records, tracking overdue books, managing fine reports, and controlling user accounts. This ensures transparency, accountability, and effective system management.

From the user perspective, students can efficiently search for books, monitor borrowed books along with issue and due dates, and track their borrowing history. The system further enhances usability by incorporating a notification mechanism that alerts users about upcoming due dates and overdue books, along with fine status.

By combining tracking, administrative control, and user-centric features, EduVault transforms conventional library management into a more efficient system, ultimately improving resource utilization and overall user experience.

Functional Requirements:

1. Librarian

- Can securely log in to the system
- Can add, update, and delete books with details (title, author, category, and year)
- Can issue books to registered students
- Can record book returns
- Can track issued books and their status (issued / returned / overdue)
- System will automatically calculate fine based on overdue days

2. Admin

- Can view all borrowing records
- Can monitor overdue books and fine reports
- Can manage user accounts (activate/deactivate)

3. Student

- Can register and log in to the system
- Can search books by title, author, or category
- Can view borrowed books
- Can check issue date and expiry date
- Can view borrowing history

- Can check current fine for overdue books

Tools:

- Frontend: HTML, CSS, JavaScript, Bootstrap (User Interface)
- Backend: PHP (Server-Side Programming)
- Database: MySQL (phpMyAdmin)
- XAMPP: Web Application Server

You are advised not to switch the tools. If you do so, you will handle the technical side yourself.

Note:

- These are the basic requirements of the application. Students may add further functionalities to make the application more useful.
- Virtual University of Pakistan (VU) will not provide any kind of hardware for this project; a student has to arrange the required hardware by himself/herself.
- VU will not pay for any license of the software, the libraries /toolkits/APIs used in this project.

Supervisor

Name: Musaddiq Hussain

Email ID: musaddiq.hussain@vu.edu.pk

MS Teams ID: [kashaan.ali13](#)

Employee Salary Management System Using Python

Project Domain / Category

Desktop Application

Abstract/Introduction

The **Employee Salary Management System** is a desktop-based application designed to streamline the management of Employee Salary. Its core objective is the proper relation between all the department entities. Salary Management System is aimed at efficient management of employee information, emoluments, expenses, net pay-outs, calculation salary based on workdays and pay salary etc. All of these are managed through a database.

Key Features:

- Interactive GUI using Tkinter,
- Input fields for employee name, ID, and basic salary
- One-click salary computation
- Automatically calculates:
- House Rent Allowance – 20% of Basic Pay
- Conveyance Allowance – 10% of Basic Pay
- Utilities – 5% of Basic Pay
- Cost of Living Allowance - 8% of Basic Pay
- Calculates **Gross Pay** and **Net Salary**
- Displays results in a beautifully formatted digital payslip

Functional Requirements:

1 Employee

- Employees can view his/her Salary detail.
- Employees can view previous month Salary detail.

2 Accountants

- Add funds
- Add leaves
- Update funds

3 Admin

- Add New Employee
- Assign Salary
- Change Employee Post

Technical Implementation:

- **Programming Language:** Python (mandatory)
- **GUI Framework:** Tkinter
Tkinter is a lightweight and widely-used Python library for developing desktop applications. It enables the creation of interactive menus and buttons, and supports cross-platform compatibility.
- **Calendar Handling:**
Use Python's built-in calendar module to manage date-based operations such as generating time-based reports.

Automation Features:

- **Centralized Data Entry:**
Accountant/Admin need to enter data only once. The system will automatically propagate and manage this data across all relevant forms and modules.

Development Guidelines:

- Use Python standard libraries and modules (e.g., calendar, tkinter).
- **Do NOT** use drag-and-drop tools or external templates (e.g., Flask templates).
- Understand the project thoroughly before beginning development.
- To better capture real-world needs, **visit an Account office** for observation and requirement gathering.

Supervisor:

Name: Asadullah

Email ID: asad.ullah@vu.edu.pk

MS Teams ID: asad_ullah121@outlook.com

Project Proposal

Project Title

Event Discovery & Community Engagement Platform

Project Domain / Category

Web Application Development / Event Management System / Community Platform

Abstract / Introduction

The proposed project is a web-based Event Discovery and Community Engagement Platform designed to help users find, explore, and participate in local and online events such as workshops, concerts, sports meets, cultural festivals, networking sessions, and community volunteering drives. The system provides a dynamic and user-friendly interface where users can browse upcoming events displayed both on an interactive map and in a structured list view, helping them decide which events to attend based on location, interest, and date.

This application bridges the gap between event organizers and attendees by combining geographic visualization with structured event data. Users can filter events by category, view event details including venue, schedule, ticket information, and organizer contact, and RSVP or register directly through the platform. The system can be extended to include user reviews, social sharing, and personalized event recommendations, making it a comprehensive and scalable solution for real-world community engagement.

Functional Requirements

FR-01: Interactive Map Display

The system shall display an interactive map interface using a mapping library (such as Leaflet.js or Google Maps API). The map shall load all available events as markers upon page load. Each marker shall be clickable and shall visually distinguish between different event categories using color-coded or icon-based markers. The map shall support zooming, panning, and fitting bounds to show all markers within the viewport.

FR-02: List View of Events

The system shall provide a structured list view displaying all available events retrieved from the backend API. Each list item shall show the event name, category, thumbnail image, date/time, and a short description. The list shall be paginated or infinitely scrollable to handle large datasets efficiently. The list view and map view shall be synchronized so that hovering or clicking a list item highlights the corresponding marker on the map.

FR-03: Category-Based Filtering

The system shall provide a filtering mechanism that allows users to filter events by predefined categories such as Music, Sports, Technology, Art & Culture, Food & Drink, Health & Wellness, and Community Service. The filter shall be accessible via a dropdown menu, tab bar, or sidebar panel. Upon selecting a category, both the map markers and the list view shall update dynamically without requiring a full page reload. Multiple category selection shall be supported where applicable.

FR-04: Event Detail Page

The system shall provide a dedicated detail page for each event, accessible via a unique URL (e.g., /events/:id or /events/:slug). The detail page shall display all available information including the full event name, category, description, venue address, date and time, ticket pricing, organizer contact information, and an embedded mini-map showing the exact event location. The page shall also include a breadcrumb navigation for easy return to the listing page.

FR-05: Images and Descriptions Display

The system shall display one or more images for each event using an image gallery or carousel component. Images shall be lazily loaded to optimize page performance and reduce initial load time. Each event shall include a short description (used in list and featured views) and a full detailed description displayed on the event detail page. Image alt-text shall be included for accessibility compliance.

FR-06: Map Marker Highlighting

The system shall visually highlight the map marker of an event when the user selects it from the list view, search results, or featured section. The highlighted marker shall change its icon, size, or color to distinguish it from unselected markers. The map shall automatically pan and center on the selected marker if it is outside the current viewport. A popup or tooltip shall appear displaying the event name, date, and category.

FR-07: Featured Events Section

The system shall include a dedicated Featured Events section displayed prominently on the homepage or landing page. Featured events shall be manually selected and managed by the admin through the Strapi CMS dashboard. Each featured event card shall display the event name, category, thumbnail image, date, short description, and a "View Details" button linking to the full detail page. The section shall be displayed in a responsive card grid or horizontal scrollable carousel layout.

FR-08: Search Functionality

The system shall provide a search bar that allows users to search for events by name, category, organizer, or venue keyword. Search queries shall be sent to the backend API, which shall return matching results from the database. Search results shall be displayed in real time or upon submission, updating both the list view and the map markers accordingly. The search bar shall include a clear/reset button to

restore the default view. If no results are found, a user-friendly "No results found" message shall be displayed.

FR-09: Address and Basic Details Display

The system shall display the full address of each event venue including street, city, and region where available. Basic details such as event date, time, category label, ticket price (or 'Free'), and organizer contact shall be visible in both the list card and the detail page. The address shall be formatted consistently across all views and shall optionally include a "Get Directions" link that opens the venue in Google Maps or a native map application.

FR-10: Responsive Interface

The system shall provide a fully responsive user interface that adapts seamlessly to all screen sizes including mobile phones, tablets, and desktop monitors. The layout shall follow a mobile-first design approach using Tailwind CSS breakpoints. On mobile devices, the map and list view shall be toggled via a tab or floating button rather than displayed side by side. All interactive elements including buttons, filters, and search inputs shall be touch-friendly with appropriate tap target sizes as per accessibility guidelines.

The Three Core Roles

Role 1: Guest User (Unauthenticated)

This is any visitor who accesses the application without logging in.

What they CAN do:

- Browse the interactive map and view all event markers
- View the list of all available upcoming and past events
- Filter events by category, date range, or location
- Use the search functionality to find specific events
- View event detail pages (name, description, images, venue, schedule)
- View the featured events section on the homepage

What they CANNOT do:

- RSVP or register for events
- Submit reviews or ratings
- Create or submit new events
- Access any admin or management features

In Strapi: This maps to the Public role, which has read-only access to published event data via the API.

Role 2: Registered User (Authenticated)

This is a visitor who has created an account and logged in via NextAuth.js + Strapi authentication.

What they CAN do:

- Everything a Guest User can do
- RSVP or register for events and receive confirmations
- Save or bookmark favorite events to their profile
- Submit reviews and ratings for events they have attended
- View their personal profile, RSVP history, and saved events list
- Update their own account information (name, password, profile picture)
- Submit a new event for admin review and approval

What they CANNOT do:

- Edit or delete other users' data
- Publish or approve submitted events
- Access the admin dashboard
- Manage categories or system settings

In Strapi: This maps to the Authenticated role, which has read access to all public data plus write access to their own user-specific data such as RSVPs, favorites, and reviews.

Role 3: Administrator (Admin)

This is a trusted internal user who manages all content and system settings through the Strapi CMS admin dashboard.

What they CAN do:

- Full CRUD (Create, Read, Update, Delete) access to all events
- Add, edit, or remove event images, descriptions, venues, and schedules
- Create, edit, and delete event categories
- Mark or unmark events as 'Featured' for the homepage section
- Manage all registered user accounts (view, activate, deactivate, delete)
- Approve or reject user-submitted event listings
- Manage roles and permissions for other users
- Configure API access and content visibility settings
- Access usage analytics and system logs (if enabled)

What they CANNOT do:

- Actions are limited only by system-level server configurations outside Strapi

In Strapi: This maps to the Super Admin or a custom Admin role created inside the Strapi admin panel with full collection-level permissions.

Key Features

- Dynamic event content management via Headless CMS
- User authentication, authorization, and role-based access control

- Fully responsive design across all devices
- SEO optimization with meta tags and structured data
- Image optimization and lazy loading for performance
- Real-time data synchronization between map and list views
- RSVP and event registration with confirmation notifications
- Admin dashboard for complete content and user management

Tools & Technology Stack

Frontend Technology Stack

Component	Technology / Tool
Framework	Next.js 16+ (React-based)
Styling	Tailwind CSS / Styled Components
State Management	Context API / Redux
UI Components	Shadcn/ui / Material-UI
Authentication	NextAuth.js
Map Library	Leaflet.js
Deployment	Vercel / Netlify

Backend Technology Stack

Component	Technology / Tool
CMS	Strapi v5 (Headless CMS)
Database	PostgreSQL / MySQL
Authentication	Strapi built-in auth + JWT
File Storage	Cloudinary / AWS S3
API	RESTful API / GraphQL
Deployment	Railway / DigitalOcean / AWS

Supervisor

Name	Amjad Iqbal Khan
Email ID	amjad.iqbal@vu.edu.pk
Teams ID	amjadiqbalkhanniazi@outlook.com

Heart Disease Prediction Android App

Project Domain / Category

Mobile Application Development

Abstract / Introduction

Heart disease is one of the leading causes of death worldwide. Early detection and monitoring can significantly reduce the risk and help individuals take preventive measures. However, many people do not have easy access to quick health risk assessment tools. The proposed Android-based Heart Disease Prediction System uses machine learning techniques to analyze user health parameters and predict the likelihood of heart disease. The user enters medical information such as age, blood pressure, cholesterol level, and heart rate through a mobile application. A trained machine learning model processes the data and provides a prediction result indicating whether the person has a low or high risk of heart disease. This application can help users monitor their health condition and seek medical advice at an early stage.

Functional Requirements

1. **FR1:** The system shall allow users to enter personal health information such as age, gender, blood pressure, and cholesterol level.
2. **FR2:** The system shall process the entered health data using a trained machine learning model.
3. **FR3:** The system shall predict the risk level of heart disease based on the input parameters.
4. **FR4:** The system shall display prediction results to the user in an easy-to-understand format.
5. **FR5:** The system shall provide basic health recommendations based on prediction results.
6. **FR6:** The system shall allow users to view their previous prediction history.
7. **FR7:** The system shall ensure user-friendly interaction through a simple Android interface.
8. **FR8:** The system shall store prediction data for future reference.

Tools

The following tools and technologies will be used for the development of this project:

- **Android Studio** – for Android application development
- **Java / Kotlin** – programming language for mobile app development
- **Python** – for machine learning model development
- **Scikit-learn** – machine learning library
- **Jupyter Notebook / Google Colab** – model training environment
- **UCI Heart Disease Dataset** – dataset for model training
- **Firebase / SQLite** – database for storing user data (optional)

Supervisor:

Name: Hina Rafique

Email ID: hina.rafique@vu.edu.pk

MS Teams ID: hina.rafique@outlook.com

Project Domain / Category

Web Programming

Abstract / Introduction

Smart Tile Store & Inventory Management System is a web-based platform designed to efficiently manage the operations of a tile retail business dealing with multiple tile types, customers, outlets, and warehouses. The system will handle different categories of tiles such as floor tiles, washroom tiles, and front elevation tiles, each with varying sizes and measurement units.

The platform will support inventory tracking across multiple store outlets and warehouses using unique item IDs, ensuring accurate stock availability and location tracking. Additionally, the system will facilitate the management of orders, stock levels. Real-time updates on inventory, sales, and stock transfers will ensure smooth business operations and improved decision-making. This system aims to reduce manual errors, optimize inventory usage, and enhance customer service through automation and smart calculations.

Functional Requirements:

Admin:

1. **Login/Logout:** Admin authentication to access the system.
2. **Change Password/Retrieve Lost Password:** Secure credential management.
3. **Manage Tile Categories:** Add/Edit/Delete tile types (floor, washroom, elevation).
4. **Manage Tile Products:** Maintain tile details (size, design, price, company, unit type).
5. **Manage Inventory:** Track stock levels across outlets and warehouses using item IDs.
6. **Manage Warehouses & Outlets:** Add/Edit/Delete multiple store locations.
7. **Stock Transfer Management:** Transfer tiles between warehouses and outlets.
8. **Sales & Purchase Management:** Record customer purchase orders.
9. **Reports Generation:** Generate reports for sales, stock, and inventory status.

Customer:

1. **Login/Logout:** Customer account access.
2. **Register Account:** New customer registration.
3. **View Tile Catalog:** Browse available tiles by category and design.
4. **Search Tiles:** Filter tiles by type, size, price, or company.
5. **View Tile Details:** Check tile specifications and availability.
6. **Place Orders:** Purchase tiles online or request quotation.
7. **Track Orders:** View order history and status.

Visitor:

1. **View Static Pages:** About Us, Contact, Terms & Conditions.

2. **Browse Tile Catalog:** View available tiles without login.
3. **Search Tiles:** Basic search functionality.
4. **Register Account:** Sign up as a customer.

Tools:

HTML, CSS, JS, BootStrap, Node JS, Next Js, MongoDB, VS Code

Supervisor:

Name: Adnan Asif

Email ID: adnanasif@vu.edu.pk

MS Team ID: ch.adnanasif@hotmail.com

Intelligent Bug Hunting Platform Project

Domain:

Software Engineering/Cyber Security

Type: Software Product

Project Code: SP26_IBHP_AHR

Introduction

The rapid proliferation of Android-based systems across mobile devices, IoT, and embedded platforms has made them a prime target for cyber adversaries. Despite Google's continuous security enhancements, a significant gap remains between known vulnerabilities and their practical detection in real-world deployments. Android devices often ship with insecure firmware configurations, outdated kernel patches, misconfigured network stacks, and weak cryptographic implementations. Similarly, Android applications frequently expose sensitive data through insecure storage, broken authentication, improper network handling, and poor code obfuscation, all of which can be exploited by attackers with minimal effort.

Traditional security testing approaches rely heavily on manual penetration testing, isolated static analyzers, or off-device proxy-based dynamic analysis. These methods are time-consuming, require significant expertise, and often fail to detect device-level (systemic) bugs that span firmware, OS, kernel, and hardware-backed security features. Furthermore, existing tools do not integrate automated exploitation workflows, on-device traffic analysis (without proxy setup), or AI-driven vulnerability reporting.

Cybersecurity professionals and bug bounty hunters lack a unified, intelligent, and automated platform that can:

- Assess Android device integrity across firmware, OS, network, and cryptographic layers without requiring root access or off-device proxies.
- Analyze Android applications through both static code analysis and dynamic behavioral analysis to identify OWASP MASVS-class vulnerabilities.
- Automate exploitation of discovered bugs to validate risk and demonstrate impact.
- Generate structured bug reports with evidence, exploitation steps, and remediation recommendations.
- Leverage custom-trained LLMs to assist in vulnerability analysis, exploit generation, and contextual remediation advice.

Functional Requirements:

1. Android Device Bug Detection

1.1. Firmware & Boot Integrity

Unlocked Bootloader, Verified Boot / dm-verity Disabled, Weak or Missing Boot Chain Trust, Outdated Security Patch Level, Vulnerable Linux Kernel Version, Vendor-Specific Backdoors / Test Points

1.2. OS & Kernel Misconfigurations

SELinux Set to Permissive, USB Debugging Left Enabled, Exposed Kernel Interfaces, Insecure IPC Settings (System-Wide), Logging Sensitive System Information, Exposed JTAG / UART Interfaces

1.3. Network & Interface Exposure

Missing or Misconfigured Firewall Rules, Unsafe Default Network Configuration, Misconfigured Bluetooth & NFC Stack, Insecure OTA Update Mechanism, Exposed Diagnostic Interfaces

1.4. Cryptography & Key Management

Weak Cryptography for Device Encryption, Missing Hardware-Backed Security Features, Weak Entropy for Random Number Generation

2. Android App Bug Detection

2.1. Insecure Data Storage

Unencrypted Shared Preferences, World-Readable/Writeable Files, Sensitive Data in External Storage, Database Without Encryption, Logging Sensitive Information, Backups Allowing Data Extraction

2.2. Broken Authentication & Session Management

Hardcoded Credentials, Insecure Token Storage, No Token Expiry or Rotation, Weak Biometric Fallback, Local Authentication Bypass

2.3. Insecure Network Communication

Cleartext HTTP Traffic, Improper SSL/TLS Validation, Mixed Content Mode, Disabled Certificate Pinning, Ignoring SSL Errors, Insecure WebView Settings

2.4. Code Quality & Reverse Engineering Risks

Missing ProGuard/R8 Obfuscation, Debuggable App in Production, Exported Components Without Permissions, Deep Links Without Validation, Insecure Dynamic Code Loading, Hardcoded Cryptographic Keys, Weak Random Number Generation, Clipboard Data Leakage, Screenshot Capture Allowed on Sensitive Screens, Side-Channel Data Exposure

3. Exploitation

Development of Automated workflows for exploitation of identified bugs/issues.

4. On/Off Device Proxy

Any traffic interception should not be used on any device or off device proxy, instead of using ADB using stats.

5. Bug Report

Systematic workflow for bug detection and exploitation report.

6. AI Integration

Use Custom Trained LLM model for Analyses, Exploit Generation and Recommendations

Nonfunctional Requirements

1. Web based Response UI Design
2. Logging
3. Authentication and User management
4. Debugging
5. Support of latest Android SDKs
6. Modular Architecture/Plugin Based Feature Addition

Prototype Requirements

1. Response Automated Bug Hunting Framework
2. Static Analysis of device and Apps
3. Static Analysis using Open-Source AI Models (both device and apps)

Tools/Languages

There are no restrictions in terms of development language, framework, libraries and databases. Following is list of tools

1. Android Device
2. Kali Linux
3. Python/MERN Stack
4. MongoDB/SQL

Project Supervisor

Name: Dr. Arif Husen

Email ID: arif.husen@vu.edu.pk

MS Teams ID: [drahrpk@outlook.com](https://teams.microsoft.com/join/rahrpk)

Intelligent LMS for Automated Evaluation of OS Scheduling and Process Management Assignments

Project Domain / Category

AI Application

Abstract / Introduction

Operating Systems course typically includes assignments on process management and CPU scheduling having diagrams like: Gantt charts, process states, and scheduling algorithms. Manually grading these assignments is time-consuming and subject to human errors. With the advent of artificial intelligence in academia, the proposed system leverages algorithmic analysis and intelligent evaluation techniques to accurately assess submitted assignments. By automating the evaluation of scheduling outputs and process-related logic, it minimizes human error and significantly accelerates the grading process.

An Intelligent LMS for Automated Evaluation of OS Scheduling and Process Management Assignments offers a practical solution by analyzing and evaluating submissions with minimal human intervention.

Functional Requirements:

An Intelligent LMS for Automated Evaluation of OS Scheduling and Process Management Assignments is designed to analyze, classify, and evaluate student submissions including: scheduling problems, Gantt charts, process states, and algorithmic solutions. The system will automatically assess correctness of scheduling outputs such as waiting time, turnaround time, and execution order, and provide accurate grading with constructive feedback while minimizing human intervention.

The proposed system will have the following main users:

Admin, Teacher, and Student

1. Registration module: It will facilitate the registration process for students and teachers. Admin will approve and perform activation of the students and teachers accounts and registration requests.
2. Login Module: After successful registrations, all types of users will be able to login to the system by using their registered email and password.
3. The application will assist the teacher in scheduling quizzes, assignments, and evaluations related to OS course, specifically focusing on Process Management and CPU Scheduling including topics: FCFS, SJF, Priority Scheduling, and Round Robin algorithms.
4. The system can generate objective and subjective questions of different cognitive levels (Application and Analysis) for the OS course having domains like: Process Management, CPU Scheduling, Process Synchronization, and Performance Evaluation metrics (waiting time, turnaround time, and CPU utilization).

5. Teachers can validate a custom-labelled dataset of existing objective and subjective questions, along with their correct solutions and verifying correct outputs for scheduling problems (e.g., FCFS, SJF, Priority Scheduling, Round Robin) ensuring accuracy of expected results including metrics: waiting time, turnaround time, and execution order.
6. The system aligns assignment grading and evaluation with a predefined rubric focused on Process Management and CPU Scheduling.
7. The system leverages advanced computer vision and transformer-based models such as YOLOv8, Detectron2, and OpenCV, along with deep learning language models like BERT and RoBERTa, to automatically analyse and evaluate diagram-based numeric and textual solutions. It support detection and assessment of process scheduling representations.
8. The system can evaluate and interpret different process scheduling representations and convert them into different forms such as process states, scheduling sequences, and Gantt charts.
9. The system supports mapping between scheduling algorithms (FCFS, SJF, Priority, Round Robin) and their corresponding execution outputs. This enables bidirectional analysis.
10. For evaluating Process Management and CPU Scheduling assignment that include diagrammatic representations such as process state diagrams and Gantt charts. It also supports analyzing execution sequences and scheduling outputs to ensure correctness of applied algorithms.
11. For the Operating Systems course, the system can use advanced code-evaluation approaches and AI-assisted models to automatically compile and evaluate programming tasks related to Process Management and CPU Scheduling.
12. The system uses development and evaluation tools like: Python, Flask/Django, OpenCV, TensorFlow/PyTorch, and Scikit-learn to implement and support Process Management and CPU Scheduling assignment evaluation.
13. The system should allow teachers to export grades and performance reports in common formats such as CSV, Excel, and PDF for record-keeping and analysis. These reports include student performance in topics like: Process Management and CPU Scheduling assignments.
14. The system evaluates different Machine Learning (ML)/ Deep Learning (DL) models like YOLO, OpenCV, and BERT for scheduling assignment evaluation and compares their accuracy.
15. For validation purpose, metrics such as MSE, RMSE, MAE, and R^2 Score are used to measure the accuracy and reliability of different ML/DL models by evaluating Process Management and CPU Scheduling assignments.
16. The system collects student submissions for Process Management and CPU Scheduling assignments and compares model-predicted grades with instructor-assigned grades. It evaluates performance using MSE, RMSE, MAE, and R^2 Score to measure grading accuracy.

17. The system allows teachers to set threshold limits for MSE, RMSE, MAE, and R^2 Score, and triggers alerts for manual review when model performance exceeds these limits.
18. The system provides teachers with detailed question-level analytics, including average score, standard deviation, and difficulty index for Process Management and CPU Scheduling assignments. It also identifies and flags questions with high incorrect response rates to indicate difficulty or ambiguity. This supports deeper analysis and improvement of assessment quality.
19. The system allows teachers to generate detailed performance reports for each question and define custom rubrics for evaluating different types of subjective and diagram-based scheduling problems. This ensures flexible and structured assessment of student work.
20. The system provides a help centre with FAQs and live/chat support to assist students with technical issues and queries related to assignment submission and platform usage.

Tools: JSP, PHP, Python, JavaScript, HTML/CSS, MySQL, PyTorch, TensorFlow, Keras, Pandas, OpenCV, YOLO (including YOLOv8), Faster R-CNN, and transformer-based models such as BERT, RoBERTa, DistilBERT, and XLNet. The system may also utilize advanced AI coding and language models such as OpenAI Codex, GPT-4, LLaMA-3, Claude, and PolyCoder for intelligent evaluation and assistance.

Supervisor:

Name: Muhammad Umar Farooq

Email ID: umarfarooq@vu.edu.pk

Teams ID: [umar.vc@outlook.com](https://www.outlook.com/umar.vc)

IoT-Based Smart Parking Locator with Google Maps Integration

Project Domain / Category

Digital Logic Design / Internet of Things – IoT / Mobile Application Development

Abstract / Introduction

Urban centers and large campuses face increasing traffic congestion, a significant portion of which is caused by drivers actively searching for available parking spaces. This search process wastes time, increases fuel consumption, and causes unnecessary bottlenecks. To address this, we propose an advanced IoT-Based Smart Parking Locator. At the hardware level, this system utilizes a network of ultrasonic or infrared sensors installed at individual parking slots. These sensors act as edge devices to continuously monitor the physical occupancy status of each space. The raw sensor data is processed by a microcontroller unit (such as a NodeMCU or ESP32) and instantly transmitted to a centralized cloud database via a Wi-Fi connection. This ensures that the system's recorded state reflects real-world conditions with minimal latency.

On the software side, the project features a dedicated mobile application designed to interface directly with the real-time cloud database. The core feature of this application is the integration of the Google Maps API, which provides users with an interactive, geographical overview of the parking facility. Users can view the real-time availability of slots overlaid directly on the map, depicted through intuitive color-coded markers. Once a user selects an available spot, the application utilizes Google Maps routing capabilities to provide turn-by-turn navigation directly to the specific parking space. Additionally, the system includes an administrative module that aggregates this occupancy data, allowing facility managers to analyze peak usage times, monitor hardware sensor health, and optimize overall parking lot operations.

Functional Requirements:

1. FR1 - Hardware-Level Occupancy Detection: The system shall utilize ultrasonic or infrared sensors deployed at individual parking slots to continuously detect the physical presence or absence of a vehicle, updating its local state within milliseconds.
2. FR2 - Secure IoT Data Transmission: The embedded microcontroller shall establish a secure internet connection and transmit the updated occupancy status (available or occupied) to a centralized cloud database immediately upon detecting a state change from the sensors.
3. FR3 - Real-Time Cloud Synchronization: The centralized cloud database shall instantly push updated parking statuses to all connected mobile client applications, ensuring that the user interface always displays the most current availability data without requiring manual page refreshes.
4. FR4 - Google Maps API Integration: The mobile application shall seamlessly integrate the Google Maps API to render a geographic map of the parking

area, visually overlaying custom markers to clearly represent the real-time occupancy status of individual parking slots.

5. FR5 - Turn-by-Turn GPS Navigation: Upon the user selecting an available parking slot on the map interface, the mobile application shall generate an optimized travel route and provide real-time, turn-by-turn GPS navigation from the user's current location directly to the selected slot.
6. FR6 - User Authentication and Profiles: The mobile application shall allow users to register and securely log in to the system, enabling them to save vehicle details, view their parking history, and manage application preferences.
7. FR7 - Administrative Monitoring Dashboard: The system shall provide a separate administrative web interface that displays comprehensive metrics, including total facility utilization rates, peak parking hours, and the operational status of all deployed hardware sensors.

Tools:

Arduino IDE

NodeMCU (ESP8266) or ESP32

Ultrasonic/IR Sensors/GPS

Android Studio / Flutter

Google Maps API

Firebase Realtime Database

Supervisor:

Name: Waqar Ahmad

Email ID: waqar.ahmad@vu.edu.pk

Microsoft TEAMS ID: [engr.waqar.ahmad@outlook.com](https://teams.microsoft.com/join/engr.waqar.ahmad@outlook.com)

Project Title: Items Auction through online System

Project Domain / Category

Web based Application.

Abstract / Introduction

Nowadays, online trade of goods (items) by companies and organizations is a trend. In the same way many individuals are willing to sell and purchase items through online platforms. We need to develop an online application for an entrepreneur (individual) which will provide online facility for common users to sell and purchase their items. The application allows the sellers to register his/her item (s) for selling with complete description (properties/attributes) along with intended/initial price. The entrepreneur (Admin) may change the price of items as per their market values (if needed). The admin will further open the items for auction for a period of three to seven days (or even for a variable period) and will also incur his commission of three to five percent. The Buyer can auction for any open item(s) and the item will be sold to the buyer (user) who has maximum bid/offer for the item during the opening period.

Functional Requirements:

A set of functional requirements of the proposed system may include the following.

1. The users (seller and buyers) must be able to register themselves.
2. The users are able to edit/update their profiles.
3. There must be a login process for users/Admin to access the application.
4. The users (Sellers) must be able to register items for sale/auction
5. The users (Buyers) must be able to bid for any open item (s)
6. The Admin must be able to open the registered items for auction
7. The Admin must have interface to decide the buyer of an item as per the maximum bid.
8. The application enables the admins to generate various reports like, list of register users (Seller and Buyers separately), list of items sold/auctioned within a week, month or any period etc.

Note: Make sure that the above list is a sample of functional requirements; students do not need to be limited with these only. Students are supposed to think (or visit the problem domain) and implement some other functional requirements not mentioned in above list.

Tools:

MySQL, PHP etc.

Supervisor Information:

Name: Asif Hussain

Email ID: asifhussain@vu.edu.pk

MS Team ID: asifnoor1982@outlook.com

LIBRARY MANAGEMENT SYSTEM

Project Domain / Category

Web Application

Abstract / Introduction

The Library Management System is developed to automate and efficiently manage the operations of a library. It maintains records of books, members, and transactions such as issuing and returning books, thereby reducing manual effort and minimizing errors.

This system provides a user-friendly interface for both librarians and users, enabling quick book searches, tracking of issued and returned items, and effective record management. It enhances overall efficiency, accuracy, and accessibility of library services.

Functional Requirements:

1. Admin would be able to Add new books with details (title, author, ISBN, category)
2. Admin would be able to Update existing book information
3. Admin would be able to Delete book records
4. Admin would be able to Register new library members
5. Admin would be able to Update member information
6. Admin would be able to Delete member records
7. Admin would be able to Issue books to members
8. Admin would be able to Accept returned books
9. Admin would be able to Track due dates for issued books
10. Admin would be able to Calculate fines for late returns
11. Admin would be able to Maintain record of all transactions
12. Admin would be able to Generate reports for issued and returned books
13. Application will Automatically track due dates
14. Application will Automatically calculate fines
15. Application will Provide user authentication (login/logout)
16. User will be able to login and logout
17. User will be able to Search books by title, author, or category
18. User will be able to View availability status of books

Tools:

PHP, XAMP server, Notepad++, MySQL server.

Supervisor:

Name: Jibrán Khan

Email ID: Jibrankhan@vu.edu.pk

MS Teams ID: jibrankhanvu@outlook.com

Madaris Management System (MMS) - Android Application

Project Domain / Category

Android App

Abstract / Introduction

Many Madaris in Pakistan still manage records manually using registers. This creates difficulties in tracking students, monitoring Quran learning progress, managing fees, and maintaining transparency in financial records. The proposed **Madaris Management System (MMS)** is a comprehensive digital platform designed to automate administrative, academic, hostel, and financial operations of Islamic educational institutions.

The system will support multiple educational streams including:

- Noorani Qaida
- Nazra Quran
- Hifz-ul-Quran
- Dars-e-Nizami
- Asri Taleem (School education up to Class 10)

The application will also include:

- Hostel management
- Zakat and Sadaqat management
- Teacher and staff salary management
- Parent monitoring system
- Donation management
- Attendance and academic reporting

The system will be **bilingual (English and Urdu)** so that madrasa staff can easily use it.

Functional Requirements:

Admin should be able to:

- Add new students
- Edit student information
- Delete student records
- Assign program
- Assign class
- Upload student photo
- Search students

Student Information Fields

- Student Name
- Father Name
- Date of Birth
- Address
- Contact Number
- Admission Date
- Program Type
- Class
- Hostel Status

1- Teacher Management Module

Admin can:

- Add teacher
- Edit teacher profile
- Assign teacher to programs
- Assign teacher to classes
- View teacher workload

a. Teacher Information

- Name
- Qualification
- Experience
- Contact number
- Subjects assigned
- Salary

2- Attendance Management

Teachers must be able to:

- Mark daily attendance
- View attendance history
- Generate monthly reports

Attendance types:

- Present
- Absent
- Leave

3- Hifz & Nazra Module (Detailed)

Teachers must log **three daily values**.

a. Sabaq (New Lesson)

Fields:

- Juz number
- Surah number
- Ayah start
- Ayah end
- Pages memorized

Example:

Juz: 2

Surah: Baqarah

Ayah: 20-35

b. Sabaqi (Recent Revision)

Revision of **last 5–7 days lessons**.

Fields:

- Date
- Surah
- Ayah range
- Performance notes

c. Manzil (Old Revision)

Revision of previously memorized sections.

Fields:

- Juz number
- Surah
- Ayah range

d. Quality Grading (Mistake Counter)

Instead of marks, teachers record mistakes.

Mistakes	Grade
0	Mumtaz
1-2	Jayyid Jiddan
3-4	Jayyid
5+	Needs Improvement

4- Nazra Module

Teachers should record:

- Page number
- Lesson completion
- Fluency level
- Pronunciation errors
- Teacher remarks

Reports include:

- Daily reading log
- Weekly progress
- Completion percentage

5- Dars-e-Nizami Module

a. Kitab-Wise Progress

Since Dars-e-Nizami is book based, progress must be tracked per book.

Example:

Kitab	Progress
Hidayat-un-Nahw	40%
Mishkat	25%

Teachers must update:

- Chapters completed
- Pages covered
- Completion percentage

b. Takrar Attendance

Evening peer study sessions.

System must track:

- Takrar attendance
- Time
- Subject
- Teacher supervision

Separate from normal attendance.

6- Asri Taleem (School Module)

Features:

- Class management (1-10)

- Subject management
- Marks entry
- Result generation
- Student promotion

7- Hostel Management Module

Hostel module manages residential students.

a. Hostel Registration

Fields:

- Student ID
- Room number
- Floor
- Bed number
- Admission date

b. Room Management

Admin can:

- Create rooms
- Allocate rooms
- Track occupancy

c. Meal Management

Track:

- Breakfast
- Lunch
- Dinner

d. Visitor Management

Record:

- Visitor name
- Relation
- Visit time

8- Parent Monitoring Module

Parents should be able to view:

a. Attendance

- Daily
- Weekly
- Monthly

b. Hifz Progress

- Sabaq
- Sabaqi
- Manzil

c. Nazra Progress

- Pages completed.
- Teacher feedback

d. Fee Status (optional)

- Paid
- Pending

9- Zakat & Sadaqat Management

a. Donor Management

Fields:

- Donor name
- Contact number
- Address

b. Donation Recording

Types:

- Zakat
- Sadaqat
- General donation

Fields:

- Amount
- Date
- Purpose

c. Zakat Distribution

Track how funds are used:

- Student support
- Scholarships
- Food
- Madrasa expenses

10- Staff Salary Management

Admin can:

- Set salary
- Record payments
- Generate salary slips

a. Staff Types

- Teachers
- Administrative staff
- Hostel staff
- Support staff

11- Fee Management

Features:

- Record student fees
- Track unpaid fees
- Generate fee reports

12- Reports Module

System should generate reports:

- Student report
- Attendance report
- Hifz progress report
- Nazra report
- Dars-e-Nizami progress
- Hostel report
- Fee report

- Salary report
- Donation report

13- Bilingual System Requirement

The app must support:

English + Urdu

Example:

English	Urdu
Student	طالب علم
Teacher	استاد
Attendance	حاضری
Fee	فیس
Report	رپورٹ

Urdu interface must support **Right-to-Left (RTL)** layout.

Tools:

Student can use any of the development tool/programming language and database

a. Development Tools

- Android Studio
- Visual Studio Code
- Eclipse
- IntelliJ IDEA
- Flutter SDK

b. Programming Languages

- Java
- Kotlin
- Dart (for Flutter)
- JavaScript (for React Native)
- C# (for Xamarin)

c. Databases

- MySQL
- Firebase Realtime Database
- Firebase Firestore
- SQLite
- PostgreSQL

Supervisor:

Name: Saeed Nasir

Email ID: saeednasir@vu.edu.pk

Team ID: saeednasirvu@outlook.com

Make Your Own Platter – Android-Based Food Ordering System with Real-Time Alerts

Project Domain / Category

Mobile Application Development (Android)

Abstract / Introduction

With the increasing use of smartphones, mobile applications have become the preferred platform for food ordering and customization. Customers often face difficulty selecting meals that fit both their taste and budget from fixed menus.

This project proposes the development of an Android-based mobile application called “Make Your Own Platter”, which allows users to create their own food platter based on their budget. The app will enable users to select food items, track total cost in real time, and receive suggestions for optimal combinations within their budget. The system will also include a real-time alert mechanism to notify the website operator whenever a new order is placed, ensuring efficient order processing.

The system will provide a user-friendly interface, making food selection simple, interactive, and efficient.

Functional Requirements:

1. User Module

- User enters budget (e.g., Rs. 500)
- User browses food categories

2. Food Selection Module

- Display food items with images and prices
- Add/remove items to platter

3. Budget Management

- Real-time total cost display
- Alert if budget is exceeded
- Suggest alternative items within budget

4. Order Summary

- Show selected items and total cost
- Confirm or modify selection

5. Order Alert System

- Customer places an order through the mobile app
- Order is stored in the database
- System triggers alert to operator

- Operator receives notification and processes the order

6. Admin Panel

- Admin views and processes order
- Login dashboard
- View incoming orders
- Manage menu items
- Update order status
- Receive instant alerts for new orders

Tools and Techniques:

- Android Studio
- Java / Kotlin
- XML / Jetpack Compose
- SQLite / Firebase/ MySQL
- Firebase / REST API
- HTML, CSS, JavaScript
- Node.js / PHP

Supervisor:

Name: Dr. Said Nabi

Email ID: said.nabi@vu.edu.pk

MS Teams ID: saidnabi_115@outlook.com

Medicare System

Project Domain / Category

Web Application

Abstract / Introduction

In modern healthcare environments, efficient management of hospital operations is critical for delivering quality patient care. Traditional hospital systems often rely on manual processes for patient record handling, appointment scheduling, billing, and resource management. These manual approaches are time-consuming, error-prone, and lack scalability, resulting in poor patient experience and administrative inefficiencies.

With the rapid advancement in web technologies, there is a growing need for digital healthcare solutions that streamline hospital operations and improve service delivery. A centralized system can significantly enhance operational efficiency by integrating various hospital functions into a single platform.

This project aims to develop a web-based Medicare System, which will automate key hospital processes such as patient registration, appointment scheduling, doctor management, medical records handling, billing, and reporting. The system will provide role-based access for administrators, doctors, and patients, ensuring secure and organized data management.

The proposed system will reduce paperwork, improve data accuracy, enhance accessibility, and provide real-time insights into hospital operations.

Functional Requirements

1. User Roles

- Admin
- Doctor
- Patient

Patient Features

1. Registration & Login

- Create account (name, email, phone, password)
- Secure login system (session-based authentication)
- Password reset functionality

2. Appointment Booking

- View available doctors by specialization
- Select date & time slot
- Book, reschedule, or cancel appointments

3. View Medical Records

- Access previous diagnoses, prescriptions, reports
- Download medical history

4. Billing & Payments

- View invoices
 - Payment options (cash, card, online transfer)
-

Doctor Features

1. Dashboard

- View daily appointments
- Manage patient queue

2. Patient Management

- View patient details & history
- Add diagnosis and prescriptions

3. Appointment Handling

- Accept/reject appointments
- Update consultation status

4. Medical Records Management

- Upload reports (PDF, images)
 - Maintain treatment history
-

Admin Features

1. User Management

- Add/update/delete doctors and patients
- Assign roles and permissions

2. Doctor Management

- Manage doctor profiles (specialization, availability)

3. Appointment Management

- Monitor all appointments
- Handle scheduling conflicts

4. Billing System

- Generate invoices
- Track payments

5. Reports & Analytics

- Daily/monthly patient reports
- Revenue reports
- Doctor performance reports

6. System Management

- Backup & restore database
 - Manage hospital departments
-

Non-Functional Requirements

- **Security:** Data encryption, secure authentication
 - **Performance:** Fast response time
 - **Usability:** User-friendly interface
 - **Scalability:** Can handle increasing users
 - **Reliability:** Data consistency and backup
-

Tools & Technologies

- **Frontend:** React
- **Backend:** PHP
- **Database:** MySQL

- **Server:** XAMPP / WAMP
- **it is mandatory to use the mention languages. Any other framework/library or language will not be acceptable**

Note: VU will not provide any resources to purchase software, frameworks, or services. Students must arrange required tools on their own.

Supervisor:

Name: Muhammad Hashir Khan

Email ID: hashir.khan@vu.edu.pk

MS Team ID: hashir.khan9996@outlook.com

Medicine Recommendation System

Project Domain / Category

Machine learning, Deep learning

Abstract / Introduction

The Medicine Recommendation System is an advanced AI-powered virtual assistant designed to enhance patient care by providing accurate and personalized medicine suggestions. Whether you're a healthcare provider, pharmacist, or patient, our system offers intuitive and efficient solutions tailored to your medical needs. Receive personalized medicine recommendations based on your medical history, current conditions, and preferences.

Functional Requirements:

1. **Data Collection & Pre-processing:** Collect and pre-process medicine-related datasets (symptoms, diseases, prescriptions).
2. **Model Training & Testing:** Train and test multiple machine learning and deep learning models
3. **Model Fine-Tuning & Optimization:** Optimize models using hyper parameter tuning techniques. Improve prediction accuracy and reduce over fitting.
4. **Medicine Recommendation Engine:** Provide personalized medicine suggestions based on: (Patient symptoms, Medical history, and existing conditions). Rank recommendations based on relevance and effectiveness.
5. **Web-Based User Interface:** Develop interactive and user-friendly web interface. Enable users to input symptoms and receive recommendations easily.

Tools:

Colab, VS code, streamlite , Python , streamlite ,

Supervisor:

Name: Dr.Mushtaq Hussain

Email ID: mushtaq.hussain@vu.edu.pk

Team ID: themushtaq48

Mobile Accessories Stock Control app.

Project Domain / Category

Mobile Apps.

Abstract/Introduction

The objective is to develop a **mobile accessories inventory application** on the Android platform. This system is designed to help shop owners manage and monitor mobile accessories such as chargers, earphones, cases, and other items in real-time through a digital platform. The application allows users to **add, update, track, and control stock levels** efficiently without the need for manual record-keeping. It is a form of digital inventory management that improves accuracy and reduces errors. This project aims to provide the advantages of **easy stock management, quick access to data, and better organization** for shop owners. Using an Android device, the user can manage inventory anytime and anywhere. Thus, the system ensures **efficient inventory control, time-saving operations, and improved business management** for mobile accessories shops. It is a form of electronic commerce. This project is an attempt to provide the advantages of online shopping to customers of a real shop. It helps buying products in the shop anywhere through the internet by using an android device. Thus, the customer will get the service of online shopping and home delivery from his favorite shop.

Functional Requirements:

- Admin can add, modify, delete and search Products by first name, Last name and date of registration, category and subcategory.
- Users can create their log in and profile by first name, Last name, Gender, address, contact no and date of registration, can order, select courier and payment mode.
- Moderators can create their log in and profile by first name, Last name, Gender domain, address, contact no and date of registration, Approve the quality of vender product to sale, track order conformation and delivery,
- Admin can add, modify, delete and search announcements by date and Type.
- Admin can add, modify, delete and search insurance/payment receiving companies and courier for delivery, Manager payments record.
- Admin can add, modify, delete and search products of Moderators by different categories (head to toe wear.)
- Admin can add, modify, delete and search moderators' profiles by different product domains.
- Admin can add, modify, delete and search payments and transaction by users.
- Admin can add, modify, delete and search moderators by date, users type and product category.

- Admin/system can add, modify, delete and search employees by id, first name and Last name.

The mobile application must be Android-based, and its data is to be managed using SQLite. Also, the application should be easily accessible to users, secured, scalable and well performing. Those criteria are what would make the application stand up from a simple application to a product that can be delivered and used by real life clients. However, the focus is more on implementing the application using the new tools before considering these enterprise application features.

Tools:

- Android Studio (mainly uses **Kotlin and Java for coding**, and **XML for designing layouts**.)
- Database (SQLite or any modern database language)

Supervisor:

Name: Imran Akhtar,

Email: imran.akhtar@vu.edu.pk

Teams id: msisl.net@outlook.com

Multi-Turn AI Chatbot with LLaMA 3

Project Domain / Category

Artificial Intelligence / Natural Language Processing / Conversational AI

Abstract / Introduction

This project focuses on the development of a multi-turn AI chatbot using a local **LLaMA 3** that enables users to engage in long, dynamic conversations across multiple domains such as machine learning, healthcare, power systems, and e-commerce etc. The chatbot processes user queries in real time without prompt engineering and supports continuous dialogue through a structured session-based system.

In addition to core chatbot functionality, the system incorporates an intelligent context management mechanism that detects topic changes and dynamically manages the context window to improve response relevance. A mandatory feedback mechanism is integrated where users rate each response based on correctness, quality, and length. All interactions are stored in a structured database for further analysis.

The system performs multi-dimensional analytics including topic-wise performance, response accuracy, positional analysis (start, middle, and end of sessions), response time evaluation, and correctness per reply. These insights are presented through interactive tables and graphical dashboards, making the system a comprehensive AI-driven conversational platform combined with advanced behavioral and performance analysis capabilities.

Functional Requirements:

PHASE 1: Chatbot System Development (Frontend + Backend Setup)

FR1: System Initialization

- The system shall initialize all core modules at startup:
 - LLaMA 3 local model loading
 - Backend server initialization
 - Database connection setup
 - Analytics module initialization

FR2: Frontend Chat Interface Development

- The system shall provide a web-based chat UI including:
 - Google OAuth login page
 - Chat interface (user input + response display)
 - Feedback panel (rating, correctness, length type)

FR3: Backend Server Development

- The system shall implement a backend using Flask/FastAPI.
- The backend shall handle:
 - User authentication
 - Session creation and management
 - Message routing to model
 - Data storage operations

FR4: LLaMA 3 Model Integration

- The system shall integrate **LLaMA 3** as the core chatbot engine.
- The system shall process user queries locally through the model.

FR5: Service Connectivity

- The system shall connect:
 - Frontend UI
 - Backend API
 - LLaMA 3 model inference engine
 - Database
 - Analytics module

PHASE 2: Chat Execution + Data Storage System

FR6: User Authentication

- The system shall allow login via Google OAuth.
- The system shall create or retrieve user profiles after login.

FR7: Session Management

- The system shall generate a unique session ID for each chat session.
- Each session shall store conversation state independently.

FR8: Multi-Turn Chat Functionality

- The system shall allow continuous conversation within a session.
- The system shall support multiple topics in a single session.

FR9: LLaMA 3 Response Generation

- The system shall send user queries to LLaMA 3 model, receive and display generated responses in real time.

FR10: Response Time Tracking

- The system shall calculate response time using request and response timestamps.

FR11: Real-Time Data Storage

- The system shall store every interaction:
 - User message
 - Model response
 - Session ID
 - Timestamp
 - Topic label

FR12: Mandatory Feedback System

- The system shall freeze input after each response.
- The system shall require user feedback before next message:
 - Rating (1–4)
 - Correctness (Correct / Partial / Incorrect)
 - Length type (Short / To the Point / Lengthy)

FR13: Input Control Mechanism

- The system shall block new input until feedback is submitted.
- The system shall unlock chat after feedback completion.

PHASE 3: Analytics System

FR14: Data Retrieval for Analytics

- The system shall fetch stored session data from the database for analysis.

FR15: Topic Classification

- The system shall classify messages into domains:
 - Machine Learning
 - Deep Learning
 - Healthcare AI
 - Power Systems
 - E-commerce AI

FR16: Session Segmentation

- The system shall divide sessions into:
 - Start phase
 - Middle phase
 - End phase

FR17: Core Metric Computation

- The system shall compute:
 - Accuracy per reply
 - Accuracy per topic
 - Accuracy per phase
 - Response time statistics
 - Rating averages
 - Length distribution

FR18: Correctness Evaluation

- The system shall calculate:
 - Correct replies count
 - Partially correct replies count
 - Incorrect replies count
 - Overall correctness percentage

FR19: Table Generation

- The system shall generate analytical tables:
 - Topic vs accuracy
 - Reply correctness table
 - Phase-wise accuracy
 - Response time summary
 - Rating distribution
 - Length preference

FR20: Graph Generation

- The system shall generate visual graphs:
 - Topic distribution (pie chart)
 - Accuracy per topic (bar chart)
 - Accuracy vs phase (line graph)
 - Correctness per reply (bar chart)
 - Response time trend (line graph)
 - Rating distribution (bar chart)
 - Length preference (pie chart)

FR21: Dashboard Display

- The system shall display analytics dashboard including:

- Tables
- Graphs
- Session-level insights
- Topic-based performance

FR22: Session Termination

- The system shall end the session and store the final analytics summary permanently.

Tools:

- **Python** – Free and open-source programming language used for backend development, chatbot logic, and analytics.
- **LLaMA 3** – Free/open-source large language model used as the core chatbot engine for generating responses locally.
- **Flask / FastAPI** – Free and open-source Python web frameworks used to build backend APIs and manage chatbot requests.
- **HTML, CSS, JavaScript** – Standard web technologies for building the chatbot user interface.
- **MongoDB / Firebase** – Free-tier cloud databases used for storing chat logs, sessions, and feedback data.
- **pandas** – Open-source Python library used for data processing and analytics.
- **matplotlib / seaborn / plotly** – Open-source libraries used for generating graphs and visual analytics.
- **scikit-learn** – Open-source machine learning library used for topic classification and analytics support.
- **Ollama (optional)** – Free tool used to run LLaMA 3 locally and serve it as an API.
- **Google OAuth 2.0** – Free authentication system used for secure Gmail login integration.
- **VS Code / Jupyter Notebook** – Free development environments used for coding, testing, and experimentation.
- **Git & GitHub** – Free version control system used for project management and collaboration.

Supervisor:

Name: Neelam Alam

Email ID: neelam.alam@vu.edu.pk

MS Teams ID: neelam-cs

NGO Charity Management System

Project Domain / Category

Web programming

Abstract / Introduction

Charity is an act of kindness where a person who has more than enough of what he or she needs contributes part of his or her surplus income to help those who are less fortunate. Most NGOs have faced difficulties in securing funds or other necessary resources. Finding donors is very challenging, and sometimes meeting a donor's conditions can be a big hurdle for NGOs. This charity management system will assist NGOs in finding donors more easily.

Functional Requirements:

This system consists of three modules—Admin, NGO, and Donor—each of which is fully integrated and interconnected with the others.

1. The admin logs into the system using valid credentials.
2. The admin reviews all requests submitted by NGOs.
3. Each request is approved or rejected after verifying the NGO's uploaded documents.
4. The admin can generate and view reports of NGOs that have received donations.
5. NGOs must first register by providing their details and uploading the required documents.
6. NGOs can access the system only after receiving approval from the Admin.
7. Approved NGOs can submit donation requests to registered donors based on their needs.
8. NGOs can track their previous events and view detailed donation reports.
9. Donors can register, log in, and receive notifications about donation requests from NGOs.
10. Donors can respond to requests by providing donation details, view scheduled donation dates, and access their donation history.

Tools:

ASP.NET/C#, PHP, HTML, CSS, JavaScript, Crystal Reports, SQL Server, MySQL

Supervisor:

Name: Qaiser Shabbir

Email ID: qaiser.shabbir@vu.edu.pk

Microsoft Teams: qaiser_shabir@outlook.com

Project Title:

Online Construction Material Procurement and Delivery System

Project Domain / Category:

Web Application

Abstract / Introduction:

Construction projects often face delays and cost overruns due to inefficient procurement and delivery of construction materials. This project aims to develop an Online Construction Material Procurement and Delivery System using PHP and MySQL, enabling contractors to order materials from multiple vendors through a single platform.

The system will allow real-time tracking of deliveries using integrated maps, calculate delivery charges based on material type and weight, and provide transparency in inventory management. Suppliers can update stock availability, manage orders, and coordinate logistics efficiently. This platform will enhance operational efficiency, reduce delays, and ensure reliable material delivery for construction projects.

Functional Requirements:

1. User registration and login for contractors, suppliers, and admin.
2. Contractors can browse/search materials by type, vendor, and availability.
3. Suppliers can add, update, and manage material inventory with details like type, quantity, and weight.
4. Contractors can place orders from multiple vendors in a single request.
5. Delivery charges are calculated automatically based on material type, weight, and distance.
6. Integration with maps for real-time tracking of deliveries.
7. Notifications for order confirmation, dispatch, and delivery status updates.
8. Admin panel to manage users, vendors, orders, and system analytics.
9. Reporting module for order history, vendor performance, inventory status, and delivery efficiency.
10. Feedback and rating system for suppliers and materials.
11. Secure payment integration for online transactions.

Tools:

Development Environment / IDE: Visual Studio Code / PhpStorm

Frontend Tools: HTML, CSS, JavaScript, Bootstrap / Tailwind CSS

Backend Tools: PHP (Laravel or Core PHP)

Database: MySQL

Other Tools: Google Maps or Open Street Map API for delivery tracking, Git for version control, Postman for API testing

Note: VU will not pay for any license of the software, the libraries /toolkits/APIs used in this project.

Supervisor:

Name: Rehan Ahmed

Email ID: rehan@vu.edu.pk

MS Teams ID: rehan.vu@outlook.com

Online Exams App

Project Domain / Category

Web Application

Abstract / Introduction

The Online Exam Application will be a web-based platform designed to automate the traditional examination process for educational institutions. The system will facilitate the end-to-end management of assessments, from secure question bank creation and automated scheduling to real-time examination conduct and instant result generation. By using a centralized database, the application will allow instructors to schedule exams for specific student groups while providing students with a user-friendly interface to attempt tests within set time limits. The project will aim to eliminate manual paperwork, reduce human error in grading, and provide detailed performance analytics, making the evaluation process more efficient, transparent, and scalable.

Functional Requirements:

1. **User Authentication and Role Management**

The system must provide secure login portals for three distinct roles: **Admin** (system oversight), **Instructor** (exam creation), and **Student** (exam takers). It must manage profile data and ensure that users can only access features relevant to their specific role.

2. **Question Bank Management**

Instructors must be able to create, edit, and delete questions within a centralized repository. The system should support multiple question formats (e.g., Multiple Choice, True/False, Short Answer) and allow questions to be categorized by subject or difficulty level.

3. **Exam Scheduling and Configuration**

The system must allow instructors to schedule exams by setting a specific date, start time, and duration. Configuration options should include selecting questions from the bank, setting passing marks, and assigning the exam to specific student batches or classes.

4. **Live Examination Interface**

The application must provide a controlled environment for students to attempt exams. It should include a **countdown timer**, a navigation panel to skip or mark questions for review, and an automatic submission feature that triggers when the time expires.

5. **Automated Grading and Instant Results**

For objective-type questions (MCQs/True-False), the system must automatically calculate scores immediately upon submission. Students should be able to view their results and basic feedback, while instructors receive a consolidated grade sheet.

6. **Reporting and Performance Analytics**

The system must generate detailed reports for both individuals and groups. This includes graphical representations (like bar charts or pie charts) of class

averages, highest/lowest scores, and subject-wise performance trends to help instructors identify learning gaps.

Tools & Technologies:

- **Frontend Development:**
 - HTML, CSS, JavaScript
 - React or Angular (for a modern, component-based framework)
 - Material UI or Bootstrap (for responsive design and UI components)
- **Backend Development:**
 - Node.js or Python (for server-side logic and API development)
 - Express.js or Django (for web frameworks)
 - MongoDB or PostgreSQL (for database management)
- **Cloud Platform:**
 - AWS, GCP, or Azure (for hosting the application and scaling resources)
- **Additional Tools:**
 - Git (for version control)
 - NPM or Yarn (for package management)
 - Webpack or Parcel (for bundling and optimization)
 - Testing frameworks (e.g., Jest, Mocha)

Supervisor:

Name: Haseeb Akmal

Email ID: haseebakmal@vu.edu.pk

MS Teams ID: haseeb.akmal@outlook.com

Online Food Ordering and Delivery System

Project Domain / Category

Web Application

Abstract / Introduction

The food industry has rapidly shifted toward online ordering platforms, allowing customers to conveniently browse menus and place orders from home. Traditional restaurant systems often lack centralized order management and real-time updates. This project proposes an Online Food Ordering and Delivery System (OFODS) that connects customers with restaurants and provides efficient order processing, tracking, and management.

Functional Requirements:

User Side:

1. Users will be able to register and log in.
2. Users can browse restaurants and menus.
3. Users can search food items by category or price.
4. Users can add/remove items from cart.
5. Users can place orders and select delivery options.
6. Users can track order status.

Admin Side:

1. Admin will be able to log in.
2. Admin can manage restaurant and menu details.
3. Admin can update pricing and availability.
4. Admin can view and manage orders.
5. Admin can generate reports (sales, orders, delivery).

Tools:

PHP, MySQL, XAMPP, Notepad++ (or MERN stack)

Supervisor:

Name: Warda Fiaz.

Email ID: warda.fiaz@vu.edu.pk

MS TeamsID: wardafiaz30@gmail.com

OMRS: Online Medical Record System

Project Domain / Category

Web Application

Abstract/Introduction

The purpose of this software is to keep medical records of the patients. Medical record may include personal information, medical history, medication, laboratory test result, allergies, radiology images and vital signs.

This software maintains data accuracy and captures the patient information time to time. Proper use of this software will eliminate the need of paper based medical record. Therefore, the risk of loss of paper record will be finished. Single centralized data storage facility will eliminate data redundancy. By using the search facility of the software, user would be able to extract medical data for the examination of possible trends and long term changes in a patient. Population-based studies of medical records may also be facilitated by the widespread adoption of this software.

Functional Requirements:

Some of these functions include, but are not limited to:

- Maintain patient record
- Manage patient demographics
- Manage patient-specific problem lists
- Manage medication lists
- Manage medical procedural/surgical, family history including the capture of pertinent positive and negative histories, patient-reported or externally available patient clinical history.
- Create, addend, correct, authenticate and close, as needed, transcribed or directly-entered clinical documentation and notes.
- Incorporate clinical documentation from external sources.
- Present organizational guidelines for patient care as appropriate to support order entry and clinical documentation.
- Provide administrative tools for organizations to build care plans, guidelines and protocols for use during patient care planning and care.
- Generate and record patient-specific instructions related to pre- and post-procedural and post-discharge requirements.

Tools/Language:

You can use any language which supports the development of web applications.

Supervisor:

Supervisor:

Name: Dr. Syed Shah Muhammad

Email ID: syed@vu.edu.pk

MS Teams ID: dr.ssmuhammad@outlook.com

Online Parent-Teacher Meeting System

Project Domain / Category

Web Programming

Abstract / Introduction

The purpose of this project is to develop a web-based application called the Online Parent-Teacher Meeting System. This system will enable parents to attend parent-teacher meetings (PTMs) remotely without visiting the school physically.

It helps parents to stay updated on their child's academic performance, behaviour, and progress. Additionally, it allows smooth communication between teachers and parents, along with an easy way to provide feedback.

Admin panel features

6. Admin can log in and log out.
7. Admin can add, update, and delete teacher records.
8. Admin can create PTM events by setting the meeting date and duration.
9. Admin can restrict meeting access to only those parents whose dues are cleared.
10. Admin can manage (add, update, delete) available time slots.
11. Admin can verify parent registrations before granting access.
12. Admin can view the booked time slots and assigned the booked time slots to teachers.
13. Admin can view the parent's feedback.
14. Admin can manage expense reports.
15. Admin can generate and manage profit/loss reports.

Teacher Panel Features:

1. Teacher can log in and log out.
2. Teacher can update their profile information.
3. Teacher can view all the booked time slots information and the student id and name should be clearly visible on the booked time slots.
4. Teacher can add, update, and delete students' academic records (tests, exams, assignments, etc.) in digital form.
5. Teacher can manage detailed mark sheets of students.
6. Teacher can conduct online meetings with parents to discuss student performance.
7. Teacher can provide feedback on student behaviour and academic progress.
8. Teacher can view parent feedback and forward it to the admin if necessary.

Parent panel features:

1. Parent can register by providing valid student details such as registration number, name, class, and date of birth. Parent account must be verified by the admin before login access is granted.
2. After granting the access, the Parent can login the system.
3. After login, parents can view available time slots and book a suitable slot one or two days before the PTM.
4. Parents can join the meeting at the scheduled date and time.
5. The meeting link will only be visible if all dues are cleared.
6. If there are pending dues, parents must pay them online before joining.
7. During the meeting, parents can view student details such as:
 - Test results
 - Marked papers
 - DMC (Detailed Marks Certificate)
 - Behaviour and participation reports
 - Other relevant documents
8. Parents can discuss academic concerns directly with the teacher.
9. After the meeting, parents must submit feedback through an online form.
10. Parent can log out.

Tools:

Server: XAMPP Server

Database: MySQL

Backend: PHP

Frontend: HTML, CSS, JavaScript

Supervisor:

Name: Akmal Khan

Email ID: akmalkhan@vu.edu.pk

MS Teams ID: akmal_vu@outlook.com

Online Printing and Packaging Services System

Project Domain / Category:

Web Application

Abstract / Introduction:

Businesses and individuals often face challenges in finding reliable printing and packaging services, comparing prices, and tracking orders efficiently. This project aims to develop an Online Printing and Packaging Services System using PHP and MySQL, enabling customers to place customized printing and packaging orders from multiple vendors through a single platform. The system will allow users to select printing types (e.g., banners, brochures, business cards) and packaging options (e.g., boxes, labels), upload designs, and track order progress using integrated maps. It will also calculate service and delivery charges based on order specifications such as size, quantity, and distance. Vendors can manage their service offerings, update pricing, handle orders, and coordinate deliveries efficiently. This platform will enhance operational efficiency, improve customer convenience, and ensure timely delivery of printing and packaging services.

Functional Requirements:

- ❖ **User Authentication & Role Management:**
User registration and login for customers, vendors, and admin.
- ❖ **Service Browsing & Search Functionality**
Customers can browse/search printing and packaging services by category, vendor, price, and availability.
- ❖ **Vendor Service Management**
Vendors can add, update, and manage services including details like type, size, material, quantity, and pricing.
- ❖ **Multi-Vendor Order Placement**
Customers can place customized orders from multiple vendors in a single request.
- ❖ **Automated Cost Calculation**
System automatically calculates total cost based on service type, quantity, customization, and delivery distance.
- ❖ **Order Tracking**
Integration with maps for tracking of order delivery.
- ❖ **Order Notifications**
Notifications for order confirmation, processing, dispatch, and delivery updates.

❖ **Admin Dashboard & Control Panel**

Admin panel to manage users, vendors, orders, and system analytics.

❖ **Reporting & Analytics**

Reporting for order history, vendor performance, service demand, and delivery efficiency.

❖ **Feedback & Rating**

Customers can provide feedback and rate vendors and service quality.

❖ **Secure Payment Integration**

The system shall support safe transactions with flexible payment methods, allowing customers to choose Cash on Delivery as the default payment option. Online payment methods may be integrated optionally.

❖ **File Upload & Design Submission**

Customers can upload print designs (PDFs, images, etc.).

❖ **Global Search Interface**

All users can search vendors, designs, and order details using keywords.

❖ **Responsive & Cross-Platform Design**

System is accessible on both desktop and mobile browsers.

Tools:

❖ Development Environment / IDE: Visual Studio Code / PhpStorm

- XAMPP, Notepad++ / Sublime Text Editor

❖ Frontend: HTML, CSS, JavaScript, Bootstrap / Tailwind CSS

❖ Backend: PHP (Laravel framework or Core PHP)

❖ Database: MySQL

❖ Others:

- Google Maps API (for delivery tracking) ,Git (for version control),
Postman for API testing

Supervisor:

Name: Muhammad Saeed Amjad

Email ID: muhammad.saeed@vu.edu.pk

MS Teams ID: saeed.lro@outlook.com

Online Quick Pharmacy Market with AI-Assisted Medical Information

Project Domain / Category:

Web Application

Abstract / Introduction:

Access to medicines in urban and rural areas is often delayed due to unavailability at nearby pharmacies, lack of price transparency, and inefficient delivery tracking. This project proposes an Online Quick Pharmacy Market where multiple pharmacies can register and list their available medicines, allowing customers to search for medicines and instantly view availability, prices, and nearby pharmacy options.

If a specific medicine is unavailable, the system will intelligently suggest alternate medicines and show pharmacies where they are in stock. The platform will include real-time delivery tracking using maps for quick order monitoring. Additionally, an AI-assisted chat module will be integrated using APIs (GroqCloud, Cerebras, or OpenRouter) to provide users with general medical information based on symptoms. The system will also support discounts, promotional marketing campaigns, and a wallet system for both customers and pharmacies, all managed by the admin. The solution will be developed using PHP, MySQL, JavaScript, and CSS as a complete multi-vendor pharmacy marketplace.

Functional Requirements:

1. FR1: User registration and authentication for customers, pharmacies, delivery riders, and admin.
2. FR2: Pharmacies can add, update, and manage their medicine inventory with prices and stock status.
3. FR3: Customers can search medicines by name, category, or symptoms.
4. FR4: System displays medicine availability across multiple pharmacies with price comparison.
5. FR5: If a medicine is unavailable, the system suggests alternate medicines and available pharmacies.
6. FR6: Customers can place orders from selected pharmacies.
7. FR7: Integration of maps for real-time order tracking and delivery location monitoring.
8. FR8: AI-assisted chat for general medical information using GroqCloud, Cerebras, or OpenRouter APIs.
9. FR9: Discount and promotional deals management controlled by admin.
10. FR10: Wallet system for customers (to add funds and pay for orders).
11. FR11: Wallet system for pharmacies for marketing campaigns and promotions.
12. FR12: Order notifications and status updates (confirmed, dispatched, delivered).
13. FR13: Admin panel to manage users, pharmacies, medicines, campaigns, and reports.
14. FR14: Reporting module for sales, stock, wallet transactions, and delivery performance.
15. FR15: Feedback and rating system for pharmacies and services.

Tools:

- Development Environment / IDE: Visual Studio Code / PhpStorm
- Frontend: HTML, CSS, JavaScript, Bootstrap
- Backend: PHP (Core PHP or Laravel)
- Database: MySQL
- APIs & Integration:
 - Google Maps API for live tracking
 - GroqCloud / Cerebras / OpenRouter API for AI chat integration
- Other Tools: Git (version control), Postman (API testing)

Supervisor:

Name: Khaqan Khawer

Email ID: khaqan@vu.edu.pk

MS Teams ID: Khaqanme@gmail.com

Online Utility Store Management

Project Domain / Category

Web programming

Abstract / Introduction

Utility stores play a vital role in providing daily household items such as groceries, beverages, and essential goods to the public. However, traditional management of utility stores often involves manual record keeping, inefficient inventory handling, and delays in customer service. These challenges can lead to stock mismanagement, billing errors, and poor customer experience.

The proposed **Online Utility Store Management System** aims to automate and streamline store operations through a centralized web-based platform. This system will enable administrators to manage inventory, monitor sales, and oversee staff activities efficiently. Utility store staff will be able to handle billing, update stock, and manage customer orders. Customers will have the facility to browse available products, place orders, and track their purchases online.

The system will cater to three types of users including administrator, utility store staff, and customers, providing them with the necessary tools to manage store operations effectively and improve overall efficiency and customer satisfaction.

The users of this application are given below.

1. **Administrator:** Responsible for managing overall store operations, inventory control, and monitoring staff activities.
2. **Utility Store Staff:** Handles daily operations such as managing stock, processing orders, and updating product details.
3. **Customer:** Individuals who can browse products, place orders, and track their purchase history.

Functional Requirements:

1. **Administrator**
 - Manage product categories and items in the store.
 - Monitor and update inventory levels.
 - View sales reports and transaction history.
 - Manage user accounts (staff and customers).
2. **Utility Store Staff**
 - Add and update product details (price, quantity, description).
 - Process customer orders and generate bills.
 - Update stock levels after each transaction.
3. **Customer**
 - Register and create a customer profile.
 - Login to the system.
 - Browse/search products based on category.
 - Place orders and view order history.

Note:

- These are basic requirements of the application. Students may add further functionalities to make the application more useful.
- Virtual University of Pakistan (VU) will not provide any kind of hardware for this project; student has to arrange the required hardware by himself/herself.
- VU will not pay for any license of the software, the libraries /toolkits/APIs used in this project.

Tools:

Microsoft Visual Studio, SQL Server, ASP.net.

Note: You can switch languages or tools. However, if you do so, you will handle the technical side yourself.

Supervisor:

Name: Ghulam Abbas

Email ID: ghulam.abbas@vu.edu.pk

MS Teams ID: live49cf19b1f61ba126@outlook.com

Price Watchdog: Autonomous Multi-Platform E-commerce Agent

Project Domain / Category

Artificial Intelligence / Web Programming / Web Automation

Abstract / Introduction

The **Price Watchdog** is an intelligent AI agent designed to solve the problem of manual price tracking in the highly volatile e-commerce market of Pakistan. Unlike standard scraping scripts that break when a website layout changes, this project utilizes a "Reasoning Agent" that understands web content semantically. The agent can visit a URL, distinguish the main product's price from "Related Products" or "Advertisements," and compare it against a user-defined target.

The project follows a **ReAct (Reasoning + Acting)** architecture. The student will develop a system where a user provides a product link (e.g., from PriceOye or Sapphire) and a desired price. The agent autonomously monitors the site, identifies price drops, and provides a logic-based summary of why a notification was triggered. This project introduces students to the core pillars of AI development: **Prompt Engineering, Tool-Calling, and Agentic Workflows.**

Functional Requirements:

Provide a bulleted list of functional requirements

1. **FR1: URL Processing & Validation** – The system must accept a single product URL and verify its status (Up/Down) before proceeding.
2. **FR2: Intelligent Content Extraction** – The agent must use an LLM to extract the **Product Name**, **Current Price**, and **Currency** from raw web data, specifically ignoring sidebar or "similar" item prices.
3. **FR3: Variant Detection** – If a product has multiple versions (e.g., 128GB vs 256GB), the agent must identify which price it is tracking and log this choice.
4. **FR4: Autonomous Comparison Logic** – The system must compare the extracted price against the user's "Target Price" and determine if an alert is necessary.
5. **FR5: Availability Monitoring** – The agent must detect and report if a product goes "Out of Stock" rather than simply failing to find a price.
6. **FR6: Notification Trigger** – The system must generate a structured alert (Console message or Email) containing the price drop percentage and a direct link to the product.
7. **FR7: History Management** – The system must store previous price checks in a local file (JSON/CSV) to show a simple price trend to the user.

Tools:

1. **Programming Language:** Python 3.10 or higher.

2. **LLM API (The Brain): Google Gemini 2.5/3.0 Flash** (Free Tier via Google AI Studio).
3. **Agent Framework: CrewAI** (Open-source library for defining Agent roles and tasks).
4. **Scraping Tool: Firecrawl** (Free tier for turning websites into LLM-ready Markdown) or **BeautifulSoup4**.
5. **Database: SQLite** (Built-in Python database for tracking price history).
6. **Web Interface (Optional): Streamlit** (Free framework for building the user dashboard).
7. **Development Environment:** VS Code or PyCharm.

Supervisor:

Name: Faizan Tahir

Email ID: fazitahir@vu.edu.pk

MS Teams ID: faizan.vu@outlook.com

Prison Management System

Project Domain / Category:

Web Application

Abstract / Introduction

This project "Prison Management System" aims to maintain and save the records of prisoners, guards and jailors in an organized and efficient manner. The main objective of the proposed project is to efficiently manage and monitor prison operations through a fully automated system that requires minimal human effort, improves accuracy, and reduces time consumption. This Prison Management System (PMS) will be used to maintain comprehensive records of prisoners and guards, making it easier for authorized personnel to track, update, and manage relevant information efficiently.

This system can be used by two types of users: Jailor and Admin. The Jailor can log in as a user and perform the following tasks:

- Add the details of prisoners, including name, age, address, crime, and punishment.
- Add and manage details of different guards on duty.
- Modify attributes such as guard duty hours and shift schedules.
- Write and save First Information Reports (FIRs). Each FIR is assigned a unique ID and includes details like date, time, number, and incident information, accessible at any time by registered users. Every prisoner is also assigned a unique identification number.

Functional Requirements:

The system consists of two main panels: one for Admin and another for Jailor. Each user type has specific access levels and responsibilities:

Admin Panel

The Admin has full access to the system and can:

1. Login for Admin
2. Admin can access and view all records in the system, including prisoners, Jailors, guards, and FIRs.
3. Admin can add new Jailor or update record of existing jailor.
4. Admin can update information of any registered user (prisoner, Jailor, or guard).
5. Admin can mark transferred jailors, guards or prisoners as inactive instead of deleting their records.
6. Admin can Track prisoner punishments, guard shifts, and overall system operations.

Jailor Panel

The Jailor has limited access focused on day-to-day prison management and can:

1. Login for Jailor

2. Jailor can record details of convicted individuals, including name, age, address, criminal number, crime, and punishment. These records are permanent and cannot be deleted.
3. Jailor can modify prisoner details such as punishment status or personal information if necessary.
4. Jailor can add new guards, update their shift and duty hours, or mark guards as inactive if they are transferred.
5. Jailor can create First Information Reports, assign unique complaint number, and record details including date, time, and location.
6. The system shall restrict jailor to add or update other Jailors, and cannot delete any records.

Tools:

PHP/Laravel, Javascript/JQuery, MySQL, XAMPP, Visual Studio Code

Supervisor:

Name: Shams U Rehman

Email ID: shamsu.rehman@vu.edu.pk

MS Teams ID: shams.vu@outlook.com

Project Title

Smart Traffic Challan System Using Number Plate Recognition

Project Domain / Category

- Artificial Intelligence (AI)
- Computer Vision
- Web Application Development

With the rapid increase in vehicles on roads, traffic violations have become a major concern, making manual monitoring inefficient and time-consuming. To address this issue, this project proposes a **Smart Traffic Challan System** that automates violation detection using **Number Plate Recognition (NPR)**.

The system integrates **Computer Vision techniques** to capture and process images or video frames of vehicles. It then applies **Optical Character Recognition (OCR)** to extract the vehicle number plate characters from the detected region. After extraction, **Artificial Intelligence (AI)** is used to validate, match, and process the number plate information with the database to identify violations and generate automatic challans.

By combining Computer Vision for detection, OCR for text extraction, and AI for decision-making, the system improves accuracy, reduces manual workload, and enhances the efficiency of traffic management.

Functional Requirements

- **Input Stage:** Upload image or video as system input.
- **Vehicle Detection:** Identify vehicles using **AI-based computer vision techniques**.
- **Number Plate Detection:** Locate the number plate through image processing methods.
- **Text Extraction:** Extract number plate characters using **OCR (Optical Character Recognition)** powered by AI.
- **Data Cleaning and Verification:** Process and validate the extracted text to ensure accuracy.
- **Data Storage:** Store the verified information in a database for future reference.
- **Violation Detection:** Analyze vehicle data to identify traffic violations.
- **Challan Generation:** Automatically generate e-challans based on detected violations.
- **Result Display:** Present processed results to the user through the system interface.
- **Record Management:** Maintain a complete history of issued challans.
- **Administrative Control:** Provide an admin panel for system monitoring and management.

Tools

- **Programming Language:** Python
- **Libraries:** OpenCV, NumPy

- **OCR Tool:** Tesseract OCR
- **Framework:** Flask / Django
- **Database:** MySQL / SQLite
- **Frontend:** HTML, CSS, JavaScript

Supervisor

Name: [Huma Mumtaz]

Email: [huma.mumtaz@vu.edu.pk]

MS Teams ID: [[huma_yu@outlook.com](https://outlook.officeapps.com/teams/join/huma_yu@outlook.com)]

Real-Time University Transit Tracker System

Project Domain / Category

Digital Logic Design / Internet of Things – IoT / Mobile Application Development

Abstract / Introduction

University campuses often face logistical challenges with internal transit systems, resulting in unpredictable schedules, delayed commutes, and significant student frustration. To resolve these inefficiencies, this project proposes a comprehensive Real-Time University Transit Tracker System powered by Internet of Things (IoT) technologies and mobile application development. At the hardware tier, each campus bus will be equipped with a custom-built tracking module consisting of a high-precision GPS receiver (such as the Neo-6M) and a microcontroller unit (ESP32 or Raspberry Pi). These modules act as edge devices, continuously capturing the exact geographical coordinates of the vehicle in real-time. This spatial data is then securely transmitted over cellular or Wi-Fi networks to a centralized cloud server for immediate processing and storage.

The software tier of this project focuses on a dedicated, user-friendly mobile application designed specifically for the student body. By integrating directly with the cloud database, the application utilizes the Google Maps API to provide an interactive, live map of the campus. Students can view the exact, real-time locations of all active buses moving along their designated routes. Furthermore, the system incorporates predictive algorithms to calculate and display highly accurate Estimated Times of Arrival (ETA) for specific bus stops, taking into account current speeds and traffic conditions. This proactive approach empowers students to manage their time efficiently, eliminates the uncertainty of waiting at transit stops, and provides university transportation administrators with a powerful dashboard to monitor fleet performance, analyze route efficiency, and ensure optimal resource allocation.

Functional Requirements:

1. FR1 - Continuous Live Location Acquisition: The embedded hardware tracking module installed on each university bus shall continuously acquire precise geographical coordinates (latitude, longitude, and speed) using a high-accuracy GPS receiver module updated at least every three seconds.
2. FR2 - Secure Telemetry Data Transmission: The designated microcontroller shall establish a secure connection and transmit the acquired GPS telemetry data, along with a unique vehicle identifier, to a centralized cloud database via available cellular (GSM/LTE) or Wi-Fi networks to ensure continuous tracking.
3. FR3 - Cloud-to-Client Real-Time Synchronization: The central cloud infrastructure shall instantly broadcast incoming location updates to all active student mobile client applications, ensuring the geographic tracking map reflects the true physical position of the buses with near-zero latency.

4. FR4 - Interactive Map Integration: The mobile application shall seamlessly incorporate the Google Maps API to render a customized digital map of the university campus, dynamically overlaying moving vehicle markers that correspond to the real-time coordinates fetched from the cloud.
5. FR5 - Dynamic ETA Calculation: The system backend shall continuously compute the Estimated Time of Arrival (ETA) for each bus approaching upcoming designated campus stops, factoring in current vehicle speed, remaining distance, and historical route data, and display this metric within the mobile app.
6. FR6 - Automated Push Notifications: The student-facing mobile application shall feature a subscription system allowing users to opt-in for automated push notifications alerting them when a specific bus is approaching their chosen stop, or if significant delays or route diversions occur.
7. FR7 - Administrative Fleet Management Portal: The project shall include a secure, web-based administrative dashboard that aggregates all transit data, allowing campus transit managers to visualize the entire fleet's live status, review historical travel logs, manage driver assignments, and identify route bottlenecks.

Tools:

Arduino IDE

NodeMCU (ESP32) / Raspberry Pi

Neo-6M GPS Module

GSM/LTE Module

Android Studio / Flutter

Google Maps API

Firebase Realtime Database

Supervisor:

Name: Zaid Ismail

Email ID: zaid.ismail@vu.edu.pk

Microsoft TEAMS ID: m.zaid_1994_1@outlook.com

RetailFlow: A Cloud-Integrated Mobile E-Commerce Ecosystem

Project Domain / Category

Mobile App/Android Application

Abstract / Introduction

RetailFlow is a Cloud-Integrated Mobile E-Commerce Ecosystem that is designed to provide a scalable, secure, and user-friendly platform for online retail operations through Android devices. The application focuses on streamlining the process of browsing, selecting, and purchasing products by integrating modern mobile technologies with cloud-based infrastructure. By leveraging Firebase, the system ensures real-time data synchronization, efficient user authentication, and reliable storage, enabling seamless interaction between users and the platform. The application focuses on delivering an intuitive user experience while maintaining high performance and data security.

The proposed system incorporates essential e-commerce functionalities such as product management, shopping cart operations, secure checkout, and order tracking, along with integrated digital payment solutions like easypaisa or JazzCash. Additionally, the platform includes an administrative module to manage products, users, and transactions efficiently. The project emphasizes scalability, allowing the system to handle increasing user demands, and adaptability to support future enhancements such as recommendation systems and analytics. Overall, RetailFlow serves as a comprehensive solution for modern mobile commerce, aligning with current industry standards and technological advancements.

Functional Requirements:

Client Side (User) Requirements:

FR1: The system shall allow users to register and log in using email and password.

FR2: The system shall allow authenticated users to browse products retrieved from Firebase Firestore.

FR3: The system shall display detailed product information including name, price, description, and image.

FR4: The system shall enable users to add products to a shopping cart and manage cart items (update/remove).

FR5: The system shall allow users to place orders through a simulated checkout process (no real payment required).

FR6: The system shall allow users to view their order history stored in Firebase Firestore.

Admin Side Functional Requirements:

FR7: The system shall allow admin to log in through a separate login interface using predefined credentials stored in Firebase Firestore.

FR8: The system shall allow admin to add new products with details (name, price, description, image).

FR9: The system shall allow admin to update or delete existing products from the database.

FR10: The system shall allow admin to view and manage customer orders (view order details and status).

FR11: The system shall allow the admin to update order status (e.g., Pending, Completed), and the updated status shall be visible to users in real-time using Firebase Firestore.

Tools:

Recommended Programing Languages:

Android Java or Kotlin.

IDEs:

Android Studio or any other relevant IDE for android applications.

Database:

Firebase, MySQL or any other similar cloud based freely available database for storage

Supervisor:

Name: Irshad Nasir

Email ID: irshad.nasir@vu.edu.pk

MS Teams ID: irshad.nasir_vu@outlook.com

Pakistan Population Data Analysis System

Project Domain / Category: Data Science / Data Visualization / Demographic Analysis

Dataset: 7th Population & Housing Census 2023, Pakistan Bureau of Statistics (PBS)

Dataset Source: Official Government of Pakistan Website, (<https://www.pbos.gov.pk>)

Download Format: Excel (.xlsx) and PDF

Abstract:

Pakistan completed its 7th Population and Housing Census in 2023, and for the first time in the country's history, it was conducted digitally. This census was carried out by the Pakistan Bureau of Statistics (PBS), which is the official federal statistics body of the Government of Pakistan. The results cover all provinces, divisions, districts, and even Mouza-level data, making it the most detailed and up-to-date population dataset ever collected in Pakistan.

CensusInsight PK is a data science desktop application built in Python that takes this official census data and turns it into meaningful visual insights. The dataset is available in Excel format directly from the PBS government website, no third-party platform, no registration, and no cost. Students simply visit [pbos.gov.pk](https://www.pbos.gov.pk), go to the Population Census section, and download the Excel tables. The data includes district-wise population counts, urban vs. rural breakdown, male vs. female ratios, household sizes, population density, literacy rates, housing conditions, and year-over-year growth rates compared to the 2017 census.

The project applies a complete data science pipeline: cleaning and organizing the raw Excel tables, performing exploratory data analysis, building interactive visualizations and maps, training machine learning models to predict population growth trends, and generating an automated summary report. All of this is done in Python using standard data science libraries.

This project is divided into 8 modules that together demonstrate real, practical data science skills on a dataset that has direct relevance to Pakistan, making it both technically strong and meaningful for learning Data Science.

Functional Requirements:

Tools and Technologies:

Module 1. Data Loading, Cleaning & Preprocessing

The census data from PBS (Pakistan Bureau of Statistics) comes in multiple Excel files, one for each province and one national summary. Each file contains several tables covering different topics. This module handles loading all of those files and organizing them into a clean, unified dataset that the rest of the application can use.

- The application allows the user to browse and load the PBS (Pakistan Bureau of Statistics) Census Excel files from their computer using a file selection dialog.
- The system imports and integrates multiple Excel-based census datasets from all provinces into a single unified dataset.

- It detects and handles missing values, blank rows, and merged cells that are common in government Excel files.
- Column names are standardized across all provincial files (for example, “Total Population”, “Male Population”, “Female Population”, “District”, “Province”).
- Numerical columns are converted to proper integer or float types so calculations and charts work correctly.
- The system stores a cleaned and standardized version of the dataset for use across all other modules.
- A data summary panel shows the total number of districts loaded, total national population, number of provinces, and any missing or incomplete records found.

Module 2. National & Provincial Population Overview Dashboard

This module gives the student and user a high-level picture of Pakistan’s population as recorded in the 2023 census. It is the first thing a user sees after the data is loaded and is designed to be visually impactful.

- Four summary metric cards at the top display: Total National Population, Total Households, Urban Population Percentage, and Rural Population Percentage.
- A horizontal bar chart shows the population of each province side by side, making it easy to compare Punjab, Sindh, KPK (Khyber Pakhtunkhwa), Balochistan, and Islamabad Capital Territory at a glance.
- A pie chart shows the share of national population held by each province.
- A male vs. female population comparison bar chart is shown for each province.
- The user can click on any province in the chart to drill down and see district-level data for that province.
- The system displays all population figures in a human-readable format suitable for presentation.

Module 3. Urban vs. Rural Analysis

One of the most important findings of the 2023 census is the urban-rural population split across Pakistan. This module focuses entirely on that divide and what it looks like across different provinces and districts.

- A stacked bar chart shows the urban and rural population for every province in a single view.
- A district-level breakdown chart allows the user to select a province and see which of its districts are the most urbanized and which are the most rural.
- The top 10 most urbanized districts and top 10 most rural districts in Pakistan are shown in ranked tables.
- A comparison between the 2017 and 2023 census data shows which districts have urbanized the most over the six-year period .for example, districts around major cities like Lahore, Karachi, and Islamabad are likely to show the biggest shifts.

- A chart shows the urban growth rate vs. rural growth rate for each province, helping identify where migration from rural to urban areas is happening fastest.
- All charts can be filtered by province using a dropdown menu.

Module 4. Population Density & Growth Rate Analysis

Population density tells us how crowded different parts of Pakistan are, and the growth rate tells us how fast the population is expanding. Both are important for planning schools, hospitals, roads, and other public services. This module analyses both.

- A colour-coded bar chart shows population density (people per square kilometre) for every district in Pakistan, sorted from highest to lowest.
- The top 10 most densely populated districts and the 10 least densely populated districts are displayed in a ranked summary table.
- A growth rate comparison chart shows the annual population growth rate between the 2017 and 2023 census for every province and selected districts.
- Districts where the population has grown very fast (more than 3% per year) are highlighted in a separate 'High Growth Zones' list.
- Districts where the population has grown very slowly or even decreased are shown in a 'Low Growth Zones' list, which may indicate emigration or undercounting issues.
- An average household size chart shows how many people live in the average household in each province a key indicator of poverty and housing conditions.

Module 5. Gender Ratio & Demographic Breakdown

The male-to-female ratio in a population reveals a lot about social conditions, healthcare access, and cultural factors. This module analyses gender distribution across Pakistan at both the provincial and district level.

- A gender ratio chart shows the number of females per 100 males for every province and for the national average.
- Districts where the gender ratio is significantly unbalanced (either very high or very low) are highlighted and listed separately.
- The system displays a colour-coded grid showing gender ratio across all districts, arranged by province.
- Age group distribution charts show the breakdown of population into children (0–14), working age (15–64), and elderly (65+) for each province, where this data is available in the census tables.
- The system displays a comparison of male and female literacy rates by province using census education data. The map is built using the Folium library with Pakistan's district-level
- The user can hover over any bar to see the exact figures for that province or district.

Module 6. Geospatial Heatmap & District-Level Map Visualization

Pakistan's census data becomes much more powerful when it is shown on a map. This module uses the district names and provincial boundaries to create interactive geographic visualizations that make patterns immediately visible.

- An interactive choropleth map of Pakistan is displayed where each district is colour-coded based on its population size, darker means more people.
- The user can switch the map view to show different indicators: population density, growth rate, urban percentage, gender ratio, or household size, all using a dropdown menu.
- Hovering over any district on the map shows a tooltip with the district name, province, total population, and the selected indicator value.
- The system uses Pakistan's official district-level boundary data to render geographic visualizations. Boundary data is freely available from the Humanitarian Data Exchange (HDX) at data.humdata.org.
- The system provides an exportable static map image of any selected indicator for use in reports and documentation.
- A sidebar table shows the top 10 and bottom 10 districts for whichever indicator is selected on the map.

Module 7. Population Growth Prediction using Machine Learning

This is the machine learning module of the project. Using the population data from both the 2017 and 2023 census, this module trains a regression model to predict what each district's population might look like in future years, such as 2030 and 2035.

- The features used for prediction include: current population, growth rate, urban percentage, household size, area in square kilometres, and province (encoded as a number).
- The system trains and compares multiple machine learning regression models to identify the best-performing approach for population prediction.
- Model performance is measured using Mean Absolute Error (MAE) and R-Squared (R^2) score, which tell us how accurate the predictions are.
- The dataset is split into 80% training data and 20% testing data. A scatter plot shows predicted vs. actual population for the test set, making it easy to see how well the model performs.
- A feature importance chart (for the Random Forest model) shows which factors have the most influence on predicting population growth, for example, urban percentage or current growth rate.
- The system saves the best-performing trained model so it can be reloaded and reused without retraining.

Module 8. Population Projector (Interactive Prediction Tool)

This module makes the machine learning model from Module 7 useful in a practical, interactive way. A student, researcher, or policymaker can enter details about a district and get a projected population for a future year.

- A simple form allows the user to select a district from a dropdown list. The app automatically fills in the current 2023 census values for that district.

- The user can adjust the growth rate slider (for example, to simulate what happens if a new highway or industrial zone increases urbanization in that district).
- When the user clicks 'Project', the model predicts the population for 2030 and 2035 and displays the results with a bar chart comparing 2017, 2023, projected 2030, and projected 2035.
- A short written insight is auto-generated below the chart, for example: "At the current growth rate of 3.1%, District Sheikhpura is projected to reach 4.2 million people by 2035, which will require significant expansion of housing, schools, and water supply infrastructure."
- This module ties all the previous work together and makes for a very strong viva demonstration because it shows the full data science pipeline working end-to-end from government data to a live, interactive prediction.
- The student can demonstrate this tool live during the viva by selecting any district the supervisor asks about and showing the projection instantly.

Module 9. Automated Report Generation

After the analysis is complete, this module allows the user to generate a professional PDF summary report with a single button click. This is useful both for the project report submission and for the viva demonstration.

- The report includes: a dataset introduction, national population summary table, top 10 most/least dense districts, province-wise comparison charts, urban vs. rural breakdown, key gender ratio findings, ML (Machine Learning) model accuracy results, and two or three projected district populations.
- All charts generated during the current session are automatically embedded into the report.
- The system generates a professional PDF summary report and saves it to a user-specified location.
- The report header shows the project name, student name, and date of generation, making it directly usable as a project deliverable.

Programming Languages & Frameworks:

Category	Technology / Tool
Programming Language	Python 3.x
IDE / Environment	Google Colab / Jupyter Notebook / VS Code / PyCharm
Data Manipulation	Pandas, NumPy, OpenPyXL (for reading Excel files)
Data Visualization	Matplotlib, Seaborn, Plotly Express

Geospatial Mapping	Folium (interactive HTML maps), GeoPandas, GeoJSON boundaries from HDX
Machine Learning	Scikit-learn (Linear Regression, Random Forest Regression)
Model Saving	Joblib
GUI / Dashboard	Tkinter (desktop) or Streamlit (web-based dashboard)
Report Generation	ReportLab or FPDF2 (PDF export)
Dataset Source	7th Population & Housing Census 2023 — Pakistan Bureau of Statistics (pbos.gov.pk) — Free, Excel format, no login required

Supervisor:

- **Name:** A Q Mohsin
- **Email ID:** mohsin@vu.edu.pk
- **Microsoft Teams ID:** aqmohsin.vu (aqmohsin.vu@live.com)

Shopify-Based E-Commerce Store Development for Skincare Products

Project Category

E-commerce Website using Shopify

Introduction

This project gives students a great opportunity to build a practical skill alongside completing their degree. In 2026, Shopify development is a highly in-demand skill, so this project is not just about academics it can also help students start earning.

In this project, students will create a skincare e-commerce store using Shopify. By building the store themselves, they will gain hands-on experience in real-world e-commerce. This will help them understand how online businesses work and prepare them for freelancing or professional work.

Students are expected to complete this project on their own rather than outsourcing it, as the main goal is learning. The supervisor will provide guidance and support throughout the process whenever needed.

Project Objectives

- To develop a fully functional Shopify store for skincare products
- To implement Add-to-Cart and Cash on Delivery (COD) features
- To design a user-friendly and responsive interface
- To give students real-world experience in e-commerce
- To help students gain a valuable and in-demand skill

Functional Requirements

- User registration and login system
- Product listing with categories and detailed product pages
- Add-to-Cart functionality
- Cash on Delivery (COD) checkout system
- Order placement and tracking
- Customer order history
- Product reviews and ratings
- Admin dashboard for managing products and orders
- Responsive design for both mobile and desktop

Non-Functional Requirements

- Simple and easy-to-use interface
- Fast loading speed with optimized images
- Secure hosting through Shopify
- Reliable system performance
- Compatibility with modern web browsers
- Scalable structure for future growth

Learning Outcomes & Industry Relevance

The main goal of this project is for students to learn by doing. Shopify development is a highly valuable skill in today's digital market. After completing this project, students will be able to:

- Start freelancing
- Build Shopify stores for clients
- Begin earning while still studying

Technologies Used

- **Shopify** For building and managing the store
- **Shopify Themes (e.g., Dawn)** For design and layout
- **HTML5** Structure of web pages
- **CSS3** Styling and layout
- **JavaScript** Interactive features
- **Liquid** Shopify's template language
- **Shopify Database** For storing products, orders, and customer data

Supervisor Details

Name:Amber Shahzadi

Email: amber.shahzadi@vu.edu.pk

Teams ID: Amber Shahzadi VU (amber.vu.cs@outlook.com)

PROJECT TITLE: SKILLSWAP: A PEER-TO-PEER COMMUNITY SKILL EXCHANGE PLATFORM

Project Domain / Category: Web Application / Social Computing

Abstract / Introduction: The SkillSwap platform is designed to facilitate a "barter system" for knowledge, where users trade their expertise without monetary transactions. In a traditional learning environment, high costs can be a barrier; this platform removes that barrier by allowing a user to teach a skill they possess in exchange for learning a new one. The system aims to build a collaborative community where knowledge is the primary currency.

Functional Requirements:

- **FR1: User Registration and Authentication:** The system shall allow users to create accounts, log in securely, and reset passwords via email verification.
- **FR2: Profile Management:** Users shall be able to manage a profile that displays their bio, location, "Skills to Teach," and "Skills to Learn."
- **FR3: Skill Cataloging:** The system shall provide a categorized list of skills (e.g., IT, Arts, Music, Cooking) that users can select from when setting up their profiles.
- **FR4: Advanced Search Engine:** Users shall be able to search for potential partners based on specific skills, expertise levels, and geographic proximity.
- **FR5: Skill Matching Logic:** The system shall automatically suggest "Matches" where User A's teaching skill meets User B's learning skill and vice versa.
- **FR6: Exchange Request System:** Users shall be able to send, accept, or reject "Swap Requests" to initiate a learning partnership.
- **FR7: Real-time Chat Interface:** The system shall provide a messaging module for matched users to discuss lesson schedules and curriculum details.
- **FR8: Rating and Review System:** After an exchange is completed, users shall be able to rate each other and leave feedback to establish community trust.
- **FR9: Reputation Score Algorithm:** The system shall calculate and display a "Trust Score" on user profiles based on the average of their received ratings.
- **FR10: User Dashboard:** A centralized interface shall show the status of active swaps, pending requests, and recently viewed profiles.
- **FR11: Notification System:** The system shall provide real-time notifications (or email alerts) for new messages, swap requests, and feedback received.
- **FR12: Administrative Control Panel:** An admin module shall allow for the moderation of users, management of skill categories, and handling of reported content/users.

Tools:

- **IDE:** Visual Studio Code (VS Code)
- **Frontend:** React.js and Tailwind CSS
- **Backend:** Node.js with Express.js

- **Database:** MongoDB or PostgreSQL
- **Communication:** Socket.io for real-time chat
- **Documentation:** MS Word

Supervisor:

- **Name:** (Safid Ullah Shah)
- **Email ID:** (safid.ullah@vu.edu.pk)
- **Teams ID:** (a015194@gmail.com)

SkillSwap – Peer-to-Peer Learning & Skill Exchange Platform

Project Domain / Category:

Web Application / E-Learning & Social Collaboration

Abstract / Introduction:

Many individuals possess valuable skills but lack opportunities or platforms to share them, while others are eager to learn but cannot afford expensive courses.

Traditional e-learning platforms are mostly one-directional and paid.

SkillSwap is an innovative web-based platform that enables users to **exchange skills instead of paying money**. For example, a user can teach graphic design in exchange for learning programming or language skills. The platform promotes collaborative learning, builds community interaction, and makes education accessible to everyone.

Functional Requirements:

FR1: User registration and profile creation (skills offered & skills wanted).

FR2: Skill matching system (suggest users based on mutual interests).

FR3: Request and approval system for skill exchange sessions.

FR4: Scheduling system for online sessions.

FR5: Rating and feedback system after each session.

FR6: Chat system for communication between users.

FR7: Admin panel for monitoring users and resolving disputes.

FR8: Notification system for session updates and requests.

Tools:

- **Frontend:** HTML, CSS, JavaScript (React.js optional)
- **Backend:** Node.js / ASP.NET Core
- **Database:** MongoDB / SQL Server
- **IDE:** Visual Studio Code
- **Other Tools:** WebRTC (for live sessions), Firebase (for notifications/chat)

Supervisor:

Name: Nadia Tabassum

Email ID: nadiatabassum@vu.edu.pk

MS Teams ID: nadia.vu.cs1@outlook.com

Smart Cafeteria Management & Pre-Order System

Project Domain / Category

Web Application

Abstract / Introduction

Our goal is to develop an online platform for managing cafeteria operations in universities or workplaces, allowing users to browse menus, pre-order meals, and avoid long queues. The system will enable customers to place orders in advance, track order status, and receive notifications when their food is ready.

Additionally, cafeteria staff can manage menu items, update availability, and handle incoming orders efficiently. The platform will also provide insights into popular items and daily sales trends. With a focus on convenience, time-saving, and better resource management, this system aims to improve the overall cafeteria experience.

Users:

There will be four users of this website: Visitors, Customers, Admin, and Cafeteria Staff.

1. Visitors:

People who browse the website without logging in. They can view menus, prices, and cafeteria information.

2. Customers:

Students or employees who create accounts to place orders. They can browse menus, pre-order meals, track order status, and view order history.

3. Administrators:

System managers responsible for overseeing users, managing system settings, and monitoring overall operations.

4. Cafeteria Staff:

Staff members who manage menu items, prepare orders, and update order statuses.

Functional Requirements:

1. **Registration:** Customers and Cafeteria Staff must register to access the system.
2. **Login:** Admin, Customers, and Cafeteria Staff log in using their credentials.
3. **Manage Users:** Admin manages all users, including approvals and account status.
4. **Manage Menu:** Cafeteria Staff can:
 - Add food items with details (price, description, availability)
 - Update or remove items
 - Mark items as available/unavailable
5. **Place Orders:** Customers can:
 - Browse menu items
 - Add items to cart
 - Place pre-orders by selecting pickup time
6. **Manage Orders:** Cafeteria Staff can:
 - View incoming orders
 - Update order status (Pending, Preparing, Ready, Completed)

7. **Order Tracking:** Customers can track their order status in real time.
8. **View Menu:** All users can view menu items and details.
9. **Search & Filter:** Users can search food items by category, price, or popularity.
10. **Feedback System:** Customers can rate food items and provide feedback.
11. **Notifications:** Users receive notifications when orders are confirmed or ready.

Tools:

C#, .NET, Visual Studio, SQL, Tomcat

Supervisor:

Name: Manahil Hassan

Email ID: manahil.hassan@vu.edu.pk

MS Teams ID: manahil.hassan2@outlook.com

Smart Counselling & Appointment System

Project Domain / Category:

Web Application

Abstract / Introduction:

Student mental health and academic guidance are critical aspects of educational institutions. However, traditional counselling systems often rely on manual appointment booking, lack proper record management, and do not ensure privacy or efficient communication between students and counsellors.

The **Smart Counselling & Appointment System** is a web-based platform designed to streamline the counselling process by enabling students to book sessions online, counsellors to manage appointments and maintain session records, and administrators to oversee the entire system.

This system ensures **efficient scheduling, secure record-keeping, and improved communication**, while maintaining confidentiality of sensitive student information. It helps institutions provide timely support to students and manage counselling services effectively.

Objectives:

- To develop an online system for **booking and managing counselling appointments**.
- To maintain **secure and confidential records** of counselling sessions.
- To allow counsellors to manage schedules and track student cases.
- To provide administrators with tools to monitor counselling activities.
- To improve accessibility and efficiency of student support services.

Functional Requirements:

1. Admin Panel:

- Secure login/logout functionality.
- Manage student and counsellor accounts (add, edit, delete).
- Approve or reject counsellor registrations.
- View all appointments and session records.
- Generate reports (appointments, active cases, counsellor workload).

2. Student Functions:

- Student registration and login.
- View available counsellors and their schedules.
- Book counselling appointments (select date & time slot).
- View appointment status (Pending, Approved, Completed).
- Submit counselling requests (academic/personal).
- View session history (limited and secure).

3. Counsellor Functions:

- Secure login for counsellors.
- Manage availability (set available time slots).
- View and approve/reject appointment requests.
- Maintain session notes (confidential records).

- Update session status (Completed, Follow-up required).
- Track student counselling history.

4. Appointment Management:

- Time slot-based booking system.
- Avoid double booking of counsellors.
- Notification alerts for upcoming sessions.

5. Confidential Record Management:

- Secure storage of session notes.
- Restricted access (only counsellor & admin).
- Privacy-focused data handling.

6. Reports & Dashboard:

- **Admin Dashboard:** Total sessions, active cases, counsellor workload.
- **Counsellor Dashboard:** Upcoming sessions, past sessions, case notes.
- **Student Dashboard:** Booked sessions, status, history.

Tools:

- Programming Languages:
 - PHP (Backend Development)
 - HTML, CSS, JavaScript (Frontend Development)
 - SQL (Database – MySQL)
- Frameworks & Libraries:
 - PHP Framework (Laravel or CodeIgniter)
 - JavaScript Framework (React, Angular, or Vue.js)
 - Bootstrap or Tailwind CSS for UI design
- Tools & Editors:
 - Code Editor (VS Code, Sublime Text, PHPStorm)
 - Database Management (phpMyAdmin, MySQL Workbench)
 - API Integration (Google Maps API, Postman for testing)
 - Web Server (Apache, Nginx)
 - UI Design Tools (Adobe XD, Figma, Sketch)
 - Project Management Tools (Trello, Asana, Jira)

(Note: Students can use alternative tools as per their preference, as long as they align with the project's scope.)

Supervisor:

Name: Komal Saleem

Email ID: komal.saleem@vu.edu.pk

MS Teams ID: komalsaleem123@outlook.com

Smart Fleet & Logistics Management System

Project Domain / Category

Mobile Application

Abstract / Introduction

Develop a mobile application designed to streamline the booking and management of commercial logistics services, such as home shifting, freight transport, or last-mile delivery. Users can use this app to book transport vehicles (ranging from bikes to heavy-duty trucks) based on cargo type and volume. The app allows users to select value-added services like professional packing, labor for loading/unloading, and real-time transit insurance.

The system features a structured payment lifecycle: an initial booking fee, a pre-transit payment, and a final settlement upon successful delivery. Integrated GPS tracking allows users to monitor their cargo, while the admin dashboard enables fleet owners to assign drivers and logistics staff to specific "shipments."

Functional Requirements

1. **User Authentication:** Users (Customers and Drivers) must be able to register and log in via email or phone verification.
2. **Service Dashboard:** The app should display available vehicle categories based on cargo requirements (e.g., Small Van for furniture, Refrigerated Truck for perishables).
3. **Search & Filter:** Users can search for available fleets based on pickup location, destination, load capacity, and urgency (Standard vs. Express).
4. **Booking & Service Selection:** An interface to input pickup/drop-off points and select additional services like "Fragile Handling" or "Insurance Coverage."
5. **Load Customization:** Users can specify the type of goods, dimensions, and estimated weight to receive an automated price quote.
6. **Shipment Request Submission:** Once details are finalized, the user submits a formal "Logistics Request."
7. **Admin Review & Confirmation:** Logistics managers (Admin) review the route and load. Upon approval, a confirmation is sent via push notification.
8. **Initial Security Deposit:** Users pay a "Commitment Fee" (10% of the quote) through an integrated gateway to lock in the vehicle and driver.
9. **Operational Breakdown:** Once confirmed, the app displays the assigned team (Driver, Loader 1, Loader 2) and the vehicle registration details.
10. **Task Assignment:** Admins assign specific routes and cargo lists to drivers. These tasks are visible in the Driver-specific interface of the app.
11. **Auto-Cancellation Logic:** If the initial deposit is not paid within 24 hours of approval, the vehicle slot is released back into the available pool.
12. **Booking Modifications:** Users can reschedule or cancel bookings up to 12 hours before the scheduled pickup. Cancellation fees apply based on the proximity to the start time.
13. **Staged Payment Milestones:** * Phase 1: 50% of the remaining balance is paid when the driver arrives at the pickup location.
14. **Phase 2:** The final balance is settled once the digital "Proof of Delivery" (POD) is signed at the destination.

15. Status Closure: After delivery and payment, the Admin formally closes the "Trip" in the system to free up the driver and vehicle.
16. Automated Notifications: The app sends real-time updates regarding driver arrival, transit milestones, and payment reminders.

Tools & Technologies

Frontend: Android Studio (Java/Kotlin)

Backend/Database: Firebase (Firestore for real-time tracking and Authentication)

APIs: Google Maps Platform (for route calculation and GPS tracking)

Supervisor:

Name: Muhammad Anwar

Email ID: manwar@vu.edu.pk

MS Team Id: anwarvu@outlook.com

Smart Fuel Optimization & Resource Management System

Project Domain / Category

Web Application

Abstract / Introduction

In developing economies like Pakistan, fuel is a very essential but scarce commodity that is highly dependent on imports. When the supply chain is disrupted during times of crisis like economic instability, war conditions or any other factors that activate such crisis, the problem of fuel shortage can severely affect transportation, logistics and daily life. People and small organizations do not have mechanisms to control the consumption of fuel, which leads to the waste of fuel and financial burden.

This project proposes the construction of scalable web-based Smart Fuel Optimization and Resource Management System which will allow users to monitor, plan and optimize fuel consumption. The system will be inspired by such systems as Google Maps (to optimize the route), Mint (to gain an understanding of the budget), and Zoho Inventory (to track the resources), and will comprise a hybrid platform that will combine fuel tracking, smart trip planning, and consumption analytics.

The system will give the users some recommendations based on the data, including an identification of the fuel efficient routes, unnecessary travelling reduction, and effective fuel budget management. It will also feature a crisis mode, which will allow users to focus on the most important tasks and extend fuel availability during shortages. It will be available on web browsers and will be scalable, thus can serve individuals as well as small organizations.

Functional Requirements:

- ✚ The system shall allow users (individual, organizational, and admin) to register an account, log in securely, update their personal profile, and log out.
- ✚ The system shall allow users to add new vehicles with attributes including fuel efficiency and tank capacity.
- ✚ The system shall allow users to view, update, or delete vehicle information at any time.
- ✚ The system shall allow users to create, read, update, and delete fuel purchase records linked to vehicles, including date, quantity, and cost.
- ✚ The system shall allow users to create trips by specifying a starting point and destination.
- ✚ The system shall calculate the estimated fuel required and suggest optimized routes to minimize fuel consumption.

- ✚ The system shall provide a “crisis mode” that recommends strategies such as combining trips or avoiding non-essential travel to conserve fuel.
- ✚ The system shall allow users to set monthly fuel budgets, track fuel expenses automatically, and display alerts when budgets are exceeded through a user dashboard.
- ✚ The system shall provide visual charts and graphs showing total fuel consumption, costs over time, and per-vehicle statistics.
- ✚ The system shall allow users to filter and interact with dashboards to analyze fuel usage patterns.
- ✚ The system shall display planned and optimized routes on a map, including distance, estimated fuel consumption, and travel time.
- ✚ The system shall allow users to zoom, pan, and interact with the map for better trip planning.
- ✚ The system shall notify users of critical events, including budget limits exceeded, inefficient trips, or recommended actions during crisis mode.

Tools:

Development Technologies:

Front-end:

HTML5/CSS3/JavaScript/React.js

Back-end:

Node.js, Express.js

Database: MongoDB

Development Tools:

Visual Studio Code

Figma (for UI/UX design)

Postman (API testing)

GitHub (version control)

Supervisor:

Name: Muhammad Kamran Qureshi.

Email ID: kamran.gureshi@vu.edu.pk

MS Teams ID: [kamranqureshi99.vu@outlook.com](https://www.outlook.com/teams/join/kamranqureshi99.vu)

Smart Home Maintenance Services

Project Domain / Category

Web Programming

Abstract / Introduction

Due to modern lifestyles, urbanization, and time constraints faced by individuals and families, the need and importance for professional home service providers (e.g. for sanitary work, plumbing issues, electrical repairs etc.) has increased significantly. Many homeowners lack the technical skills or time to handle and complete these kind of essential home maintenance services. Keeping in view the above scenario, this project is focused to fulfil the needs of these people. Reliable home service providers will offer convenience, safety, and efficiency by delivering skilled services at the customer's doorstep.

This application will help customers to quickly find the expert service providers from nearby locations and according to the needs and requirements of the customers. The system will also be beneficial for service providers as they can find their work easily. This system will reduce time and cost of both customers and service providers. In this application, customers can register, login and view all the available nearby service providers. Customer can filter and select any service provider according to his experience, expertise and previous rating etc. Similarly, service providers can also get customers by registering and logging into the website and setting up their profile. Admin can login, manage service providers, approve new service providers and can remove service providers from the system after unsatisfactory remarks and extreme low rating by the customers. Admin can also check for the registered customers.

Functional Requirements:

There will be three modules of this web application: Admin, Service Provider and Customer. Each Customer and Service Provider should be registered on the website before booking any service. Administrator (Admin) is the super user of the website who can manage everything on the website.

1. Service Provider Module

- ❖ Service Provider can register and login to the system.
- ❖ Service Provider can make their profile including their name, work experience, expertise (checklist of sanitary work expertise, plumbing works, electrical appliances and repairing works on which concerned service provider has expert knowledge and work experience), qualification, profile picture and their location.
- ❖ Service Provider can view/update their profile and can update password.
- ❖ Service Provider can check the rating given by the customer.
- ❖ Service Provider can check any remarks (if given) by the customer.

2. Customers Module

- ❖ Customers can register and login to the system.

- ❖ Customers can view/ update their profile and can update password.
- ❖ Searching of Service Providers: Customers can search for all relevant Service Providers by search filters such as location, work experience, expertise and rating etc.
- ❖ View Service Provider: Customers can view all the details (work experience, expertise, rating, other customers' remarks, location etc.) of any available Service Provider in a proper layout on the website.
- ❖ Book any Service Provider: Customers can book any Service Provider according to his/her needs and choice by selecting a preferred date & time and type of required service.
- ❖ Rating of Service Provider: Customers can give rating to the concerned Service Provider after the completion of required work according to his/her satisfaction.
- ❖ Remarks about Service Provider: After the completion of the required work, customers can give any remarks and comments about the concerned Service Provider.
- ❖ Complaint about Service Provider: Customers can register complaint about the concerned Service Provider to the admin, in case of any serious issues such as service provider's misbehaviour etc.

3. Admin Module

- ❖ Admin can login to the system.
- ❖ Admin approves the registered service providers.
- ❖ Admin can manage the service providers (add/ view/ update / delete).
- ❖ Admin can update the service provider details and manage login password.
- ❖ Admin can manage the customers (add/ view/ update / delete).
- ❖ Admin can check the rating given by customers about each service provider.
- ❖ Admin can check average rating for each service provider.
- ❖ Admin can check the remarks given by customers about each service provider.
- ❖ Admin can block any service provider in case of continuous poor rating.
- ❖ Admin can view the customer complaints and take appropriate actions.
- ❖ Admin can generate a report on a monthly basis for the total number of bookings and their successful handling.

4. Payment Module:

Upon successful completion of any service from the service provider, customers can either pay the amount to the concerned service provider on the spot or can transfer the amount to the concerned service provider's account number available on the website. The responsibility of completing the required service will be on the service provider and the responsibility of verifying the completed service will be on the customer respectively.

[**Note:** Student can add/enhance requirements as per need and keeping the time span and scope in view.]

Tools:

ASP.NET, C#, React JS, Node JS, HTML, CSS, JavaScript, Ajax, jQuery, Bootstrap, MS SQL Server

Supervisor:

Name: Syed Hassan Ali Shah

Email ID: hassan.ali@vu.edu.pk

MS-Teams ID: syed.hassan.ali.sha@outlook.com

Smart Package & Subscription Reminder App

Project Domain / Category

Mobile Application Development

Abstract / Introduction

Managing multiple subscriptions such as mobile packages, streaming services, gym memberships, and utility bills can be challenging. Users often forget renewal dates or continue paying for unused services. This project proposes a Smart Package & Subscription Reminder App that allows users to track all their subscriptions in one place and receive timely reminders before expiry or renewal. The system will provide a centralized dashboard, notification system, and history tracking to help users manage their expenses efficiently.

Functional Requirements

The main functional requirements of the project are:

1. Develop a system to manually track subscriptions and packages.
2. Implement reminder notifications for expiry and renewal.
3. Create a dashboard for overview of active subscriptions.
4. Maintain a history of past subscriptions and payments.
5. Provide smart recommendations based on usage patterns.

Prerequisites (what you must know / install)

1. Basic knowledge of mobile app development
2. Understanding of databases
3. Basic programming concepts
4. Understanding of APIs

Tools & Technologies

- Frontend: Flutter / React Native
- Backend: Firebase / Node.js + MongoDB
- Notifications: Firebase Cloud Messaging
- APIs (Optional): Carrier or subscription service APIs
- Development Tools: Android Studio or Visual Studio Code (VS Code)

Supervisor:

Name: Fizzah

Email ID: fizzah@vu.edu.pk

MS Team ID:

fizzahbhatti2020@outlook.com

fizzah.vu@outlook.com

Smart Plant Care System with AI-Based Monitoring and Mobile App

Project Domain / Category

Internet of Things (IoT) + Mobile Application + Artificial Intelligence

Abstract / Introduction

With the advancement of IoT technologies, everyday tasks can be automated and optimized through smart systems. However, plant care remains a common challenge, as many individuals lack the knowledge or consistency needed to maintain proper watering and environmental conditions, often leading to unhealthy plants.

This project proposes a Smart Plant Care System that combines IoT sensors, a mobile application, and a simple machine learning model to assist users in maintaining plant health. The system uses sensors to monitor key parameters such as soil moisture, temperature, and light intensity, and transmits this data to a mobile app in real time via a WiFi-enabled microcontroller (e.g., ESP32).

The mobile application allows users to monitor plant conditions, view historical data, and receive alerts when action is needed. Additionally, a lightweight machine learning model analyzes past data to provide personalized recommendations, such as optimal watering schedules. An optional automated watering mechanism can also be included to enhance convenience.

This solution aims to simplify plant care, improve plant health, and promote efficient resource usage, offering a practical and user-friendly smart gardening system.

Functional Requirements:

1. **Sensor Data Collection (IoT Module)**
 - Collect soil moisture, temperature, and light data using sensors.
2. **Real-Time Monitoring**
 - Send sensor data to the mobile app via WiFi/Bluetooth.
3. **User Registration and Login**
 - Secure access to user-specific plant data.
4. **Plant Profile Management**
 - Users can add different plants and set preferences.
5. **Live Dashboard**
 - Display real-time sensor readings in the mobile app.
6. **Automated Alerts**
 - Notify users when plants need watering or attention.
7. **AI-Based Recommendation System**
 - Suggest optimal watering schedules based on past data.
8. **Watering Prediction (ML Component)**
 - Predict when the plant will need water next.
9. **Manual Override Control (Optional Hardware Control)**
 - User can trigger a water pump via the app.
10. **Data History & Visualization**

- Show trends using graphs (moisture vs time, etc.)

Tools:

Hardware (IoT):

- Arduino / ESP8266 / ESP32
- Soil Moisture Sensor
- Temperature & Humidity Sensor (DHT11/DHT22)
- Camera Module for Live Tracking
- Light Sensor (LDR)
- Water Pump (optional)
- Relay Module

Software:

- **Mobile App:** Flutter / Android Studio
- **Backend:** Firebase / Node.js
- **Machine Learning:** Python (Scikit-learn)
- **Database:** Firebase / SQLite

Other Tools:

- Arduino IDE
- Git (Version Control)
- Figma (UI Design)

Supervisor:

Name: Waqas Ahmad

Email ID: waqas.ahmad@vu.edu.pk

MS Teams ID: waqas.ahmad657@gmail.com

Smart Resume Screener & Job Board

Project Domain:

Web Programming

Abstract:

The Smart Resume Screener is a web-based application designed to streamline the recruitment process by bridging the gap between employers and job applicants. In traditional hiring, reviewing hundreds of resumes is a manual, time-consuming process prone to human oversight. This system provides a platform where employers can post job openings and applicants can apply by uploading their resumes in PDF format. The core innovation of this project is its automated Python-based screening engine. Upon submission, the backend extracts text from the applicant's resume, processes it, and calculates a similarity score against the target job description using basic Natural Language Processing (NLP) techniques. Employers are then presented with a dashboard where candidates are automatically ranked by their match percentage, significantly reducing the time required for initial candidate shortlisting.

Functional Requirements:

User Accounts & Security:

- Users need to be able to sign up as either an "Employer" or an "Applicant."
- To register, they just need to provide their full name, a unique email address, and a password.
- For security, the backend must hash all passwords before saving them to the database—no plain-text passwords allowed!
- Once registered, users can log in to start a secure session, which should instantly redirect them to their specific dashboard (Employers go to the employer view, Applicants go to the job board).
- A standard logout button is required to safely end the session.

Employer Tools (Job Management):

- Employers need a straightforward way to manage their hiring. From their dashboard, they can create new job postings by filling out a Job Title, a detailed Job Description, and a comma-separated list of Required Skills.
- They must have full control to edit these details later or toggle a job's status between "Open" (visible to applicants) and "Closed" (hidden/inactive).
- The main dashboard will act as their home base, cleanly displaying a list of all the jobs they've created.

Applicant Experience (Job Board):

- When applicants log in, they should see a feed of all currently "Open" jobs.
- Clicking on a job title should expand it to show the full description and the skills the employer is looking for.
- If it's a good fit, the applicant can apply right there by uploading their resume. To keep things standardized for the text extractor, the upload form must strictly only accept .pdf files.
- To prevent duplicate data, the backend needs to block an applicant from applying to the exact same job twice.

- Applicants should also have a handy "My Applications" page so they can track the history of the jobs they've applied for.

The "Smart" Engine (Backend Resume Parsing):

- This is the core logic of the project. As soon as an applicant submits their resume, the backend immediately goes to work.
- First, it extracts all the readable text from the uploaded PDF.
- Next, it pulls the target "Job Description" and "Required Skills" from the database and combines them into one reference text.
- To ensure a fair comparison, the Python logic needs to clean both the resume text and the reference text by making everything lowercase and stripping away basic punctuation.
- Finally, it calculates a similarity score (a match percentage from 0% to 100%) based on how many keywords and skills overlap, and saves that score to the database.

Employer Dashboard & Applicant Tracking:

- When an employer clicks on one of their job postings, they shouldn't just see a random list of people. Instead, the page should display all the applicants neatly ranked, with the highest match percentage right at the top.
- The list should clearly show the applicant's name, email address, and their calculated score.
- Finally, next to each applicant's name, there needs to be a "Download Resume" button so the employer can easily grab the original PDF if they want to review it themselves.

Admin Oversight & Moderation:

- The system should have a specialized "Admin" role. Because this is a high-level account, people shouldn't be able to just register as an admin. This account can be manually created in the database or assigned via a special backend script.
- When the admin logs in, they should be taken to a unique "System Dashboard" that gives them a bird's-eye view of the entire platform's activity (e.g., total number of users, total job postings, and total resumes processed).
- **User Management:** The admin needs the power to view a list of all registered Employers and Applicants. If someone is abusing the system (like a fake employer account), the admin must have a button to permanently delete or suspend that user.
- **Content Moderation:** The admin must be able to view every job posting on the platform, regardless of who created it. If a job posting contains inappropriate content or is spam, the admin needs the authority to delete it immediately to keep the platform clean.

Required Tools:

Front-End

- **HTML5 & CSS3:** For standard page structure and styling.
- **JavaScript:** For basic client-side form validation (e.g., ensuring a file is selected before hitting submit).

- **Bootstrap 5:** For rapidly building a responsive, clean, and modern User Interface (navigation bars, tables, forms, and grid layouts).

Back-End

- **Python (3.x):** The core programming language for backend logic.
- **Flask:** The lightweight web framework used to handle routing, HTTP requests, and server-side logic.
- **Jinja2:** Flask's default templating engine for rendering dynamic HTML pages based on backend data.

Database

- **SQLite:** A lightweight, file-based database perfect for a single-semester project. Requires no separate server setup.

Specific Python Libraries (Dependencies)

- **Flask-SQLAlchemy:** An Object Relational Mapper (ORM) to handle database queries using Python classes instead of raw SQL.
- **Flask-Login:** To manage user sessions, login state, and role-based route protection.
- **Werkzeug.security:** For password hashing (generate_password_hash, check_password_hash).
- **PyPDF2 (or pdfplumber):** To extract raw text strings from the uploaded PDF documents.
- **scikit-learn:** Specifically, the **TfidfVectorizer** and **cosine_similarity** modules to calculate the mathematical text match percentage between the resume and the job description.

Supervisor:

Name: Muhammad Ilyas

Email ID: Muhammad.ilyas@vu.edu.pk

MS Teams ID: ilyas.vu@outlook.com

Smart Student Performance & Prediction System

Project Domain / Category

Machine Learning, Web Application Development

Abstract / Introduction

The Smart Student Performance & Prediction System is a web-based application designed to manage student academic data and predict future performance using machine learning techniques. The system allows teachers and administrators to efficiently manage student marks, attendance, and course details while providing intelligent insights into student performance.

The core feature of the system is its ability to analyze historical academic data and predict student outcomes such as final grades and risk of failure. This helps educators identify struggling students early and take corrective actions. The system integrates a user-friendly web interface, backend logic, database storage, and a machine learning model to provide actionable insights for improving student success rates.

Functional Requirements:

1. User Authentication

The system will allow teachers and administrators to register and login securely using email and password.

2. Student Management

The system will allow adding, updating, deleting, and viewing student records including personal information (name, roll number, email, contact).

3. Marks Entry

Teachers can enter marks for quizzes, assignments, midterm exams, and final exams for each student.

4. Attendance Management

Teachers can record and update student attendance on a daily, weekly or monthly basis (present/absent with percentage calculation).

5. Grade Calculation

The system will automatically calculate total marks, percentages, and grades (A, B, C, D, F) based on predefined criteria.

6. Performance Prediction (Machine Learning)

i. Training Process: The machine learning model will be trained using a pre-collected dataset in CSV format containing following student features:

- Attendance percentage
- Quiz scores
- Assignment marks
- Midterm exam scores
- Final exam results (target variable)

The training process includes:

- Data preprocessing (handling missing values, normalization)
- Feature selection

- Model training using algorithms such as Random Forest or Decision Tree
- Model evaluation using accuracy, precision, and recall metrics

ii. Prediction Process: During runtime, the trained model will be loaded into the backend (Django/Flask). When a teacher enters student marks and attendance through the frontend form, the backend sends this data to the ML model, which predicts:

- Expected final grade
- Risk level (Low, Medium, High)

The system will classify students into three risk levels based on predicted performance:

- Low: Expected to pass with good grades
- Medium: May need additional support
- High: At significant risk of failing

The predicted result is then displayed to the teacher and stored in the database for future reference.

7. Prediction History

The system will store all prediction results in the database, allowing teachers to track student progress over time and verify prediction accuracy against actual results.

Tools and languages:

Programming Language: Python

Framework: Django / Flask

Database: MySQL / SQLite

Machine Learning Library: scikit-learn, pandas, numpy

Frontend: HTML, CSS, Bootstrap

IDE: VS Code / PyCharm

Other Tools: Git

Supervisor:

Name: Humaira Naeem

Email ID: humairanaeem@vu.edu.pk

Teams ID: humera_naeem@outlook.com

Smart University Complaint System

Project Domain / Category Web Application

Abstract / Introduction Physical complaint boxes are slow, and emails often get lost. This project replaces them with a smart, web-based complaint routing system designed to make issue reporting fast, transparent, and fair.

By choosing this project, you will build a dynamic web application that solves a real-world administrative problem. You will develop a system with distinct role-based access (Students, Admins, HODs) and implement backend logic for user quotas, automated priority sorting, and sensitive data routing. It is an excellent project for students looking to strengthen their skills in full-stack web development and database management.

Functional Requirements:

- **FR1 - Role-Based Dashboards:** The system must provide separate, secure login portals for Students, Administrators, and the Head of Department (HOD).
- **FR2 - Categorized Submissions:** When submitting a complaint, students must explicitly select the category of the issue and specify against whom or which department the complaint is directed.
- **FR3 - Monthly Quota:** To prevent system overload and spam, each student account is strictly limited to submitting 5 complaints per month.
- **FR4 - Admin Workflow:** Admins manage the standard complaint queue. They can take two primary actions: mark a complaint as "Solved" or forward it to the HOD if it requires higher-level intervention.
- **FR5 - Automated Priority Sorting:** Complaints on the Admin dashboard must be automatically sorted by urgency (e.g., High, Medium, Low) ensuring critical issues are handled first.
- **FR6 - Conflict of Interest Filter:** If a student submits a complaint directed "Against Admin," the system must automatically hide this ticket from the Admin dashboard and route it directly to the HOD.
- **FR7 - Emergency Override:** Students can use an "Emergency" button for urgent, immediate hazards. This pushes the ticket to the top of the Admin's queue in red, overriding standard sorting.
- **FR8 - Smart Grouping:** The system must allow Admins to group identical complaints together (e.g., merging 10 complaints about the same broken projector) to resolve them with a single action.

Tools:

- **Programming Language:** Python
- **Framework:** Flask, Django, or FastAPI
- **Database:** SQLite or MySQL
- **Frontend:** HTML, CSS, and JS

Supervisor:

- **Name:** Mohammad Raheel
- **Email ID:** mohammad.raheel@vu.edu.pk
- **MS Teams ID:** mraheel.cs@outlook.com

SmartGrader – AI Powered Short Answer Evaluation System

Project Domain / Category

Machine Learning / Natural Language Processing (NLP)

Abstract / Introduction

SmartGrader is an intelligent automated evaluation system designed to assess short-answer responses using advanced Natural Language Processing (NLP) and machine learning techniques. The system implements dual evaluation approaches: a rule-based keyword matching model for precise term coverage and a neural sentence-transformers model for semantic understanding.

Using a real-world “Automatic Short Answer Grading” dataset containing over 4000 training records and structured evaluation fields (questions, model answers, student answers, teacher marks, and total marks), the system processes textual responses and generates predicted scores. The application ensures accurate, consistent, and scalable evaluation by analyzing both keyword relevance and semantic similarity, enabling fair comparison between predicted and actual teacher-assigned marks.

Functional Requirements:

The functional requirements of this project are given below:

16. Professional Web Interface Design

Requirement: Design a complete multi-page web application interface for user interaction.

Details:

- Home Page:
Display project overview, purpose, and navigation to evaluation system.
- Evaluation Page:
 - Provide input fields for question, model answer, and student answer
 - Allow model selection between Rule-Based and Neural evaluation models
 - Display results including predicted marks, similarity score, and feedback
- Dataset Page:
Upload and view dataset records in tabular format
- Dashboard:
Show evaluation history and comparison of predicted vs teacher marks

17. Flask Backend API with Processing Logic

Requirement: Implement a backend system with REST APIs for evaluation and data handling.

Details:

- POST /api/evaluate: Accept student answer and model answer and return predicted marks
- POST /api/upload: Upload dataset and validate required columns
- GET /api/history: Retrieve stored evaluation results
- Process user input in real-time and return immediate evaluation response
- Ensure backend handles both Rule-Based and Neural model processing

18. Data Collection

Requirement: Load and utilize a structured dataset for short-answer grading.

Details:

- Use Automatic [Short Answer Grading dataset](#) containing:
 - questions
 - model_answer
 - student_answer
 - teacher_marks
 - total_marks
- Ensure dataset contains 4000+ training records and test samples
- Verify dataset completeness and correctness before processing

19. Data Preparation

Requirement: Clean and standardize dataset before model processing.

Details:

- Remove missing or incomplete records from dataset
- Ensure consistent column naming and formatting
- Normalize marks using teacher_marks and total_marks
- Convert dataset into structured CSV format for easy processing
- Also, you can perform any further necessary improvements to improve the model's accuracy.

20. Data Pre-Processing

Requirement: Prepare textual data for machine learning models.

Details:

- Remove punctuation, special characters, and unwanted symbols
- Convert all text into lowercase for consistency
- Perform tokenization to split sentences into words
- Remove stopwords to reduce noise in data
- Apply lemmatization using spaCy to normalize words

21. Feature Extraction

Requirement: Convert text into numerical representations for model input.

Details:

- For Rule-Based Model: Extract important keywords and phrases from model answers and compare them with student answers to measure coverage and relevance

- For Neural Model: Generate dense embeddings using sentence-transformers (all-MiniLM-L6-v2) to capture semantic meaning of answers
- Convert both model answers and student answers into vector representations
- Ensure consistent feature generation pipeline for both training and evaluation phases

22. Train & Test Data Preparation

Requirement: Prepare dataset for model training and evaluation.

Details:

- Use the provided dataset containing first 4000 records for training and learning patterns
- Utilize last 275 records as test samples to evaluate model performance. Don't use them in training.
- Normalize marks for consistent scoring comparison
- Ensure proper separation of training and testing data to avoid data leakage

23. Model Development – Rule-Based Evaluation Model

Requirement: Build a classical keyword-based evaluation model.

Details:

- Extract key terms and important phrases from model answers
- Match these keywords with student answers to determine coverage
- Calculate score based on percentage of matched keywords
- Assign marks proportionally according to total_marks

24. Model Development – Neural Embedding Model

Requirement: Build a deep learning evaluation model using semantic embeddings.

Details:

- Use sentence-transformers to generate embeddings for both model and student answers
- Calculate cosine similarity between embeddings to measure semantic similarity
- Map similarity score to marks based on predefined thresholds
- Ensure model captures conceptual similarity even if exact keywords are different

25. Performance Evaluation

Requirement: Evaluate accuracy and effectiveness of both evaluation models.

Details:

- Compare predicted marks with teacher_marks from dataset

- Calculate evaluation metrics such as Mean Absolute Error (MAE) and accuracy
- Analyze performance differences between Rule-Based and Neural models
- Identify strengths and limitations of each approach

Tools & Technologies:

Backend: Python, Flask

ML/NLP: spaCy, Sentence-Transformers, Scikit-learn

Frontend: HTML5, Bootstrap5, JavaScript

Database: SQLite

Development: PyCharm with virtual environment, Git for version control

Helping Material:

	Resource Title	Link
	Python Official Website	https://www.python.org/
	Python Tutorial - W3Schools	https://www.w3schools.com/python/
	Python Tutorial - TutorialsPoint	https://www.tutorialspoint.com/python/index.htm
	Kaggle Python Course	https://www.kaggle.com/learn/python
	Intro to Machine Learning - Kaggle	https://www.kaggle.com/learn/intro-to-machine-learning
	Google Machine Learning Crash Course	https://developers.google.com/machine-learning/crash-course
	Intro to Deep Learning - Kaggle	https://www.kaggle.com/learn/intro-to-deep-learning
	Python Deep Learning Tutorial	https://www.tutorialspoint.com/python_deep_learning/index.htm
	Deep Learning Tutorials	https://www.tutorialspoint.com/deep-learning-tutorials/index.asp
	NLP with Python - YouTube	https://www.youtube.com/watch?v=VyWAvY2CF9c
	Machine Learning Tutorial - YouTube	https://www.youtube.com/watch?v=6M5VXKlf4D4
	Natural Language Processing Specialization	Coursera NLP courses
	spaCy v3 NER Tutorial	YouTube spaCy tutorials
	Flask Official Documentation	https://flask.palletsprojects.com/

	Bootstrap 5 Documentation	https://getbootstrap.com/docs/5.0/
	spaCy Documentation	https://spacy.io/usage
	Sentence Transformers Docs	https://www.sbert.net/
	scikit-learn Documentation	https://scikit-learn.org/stable/
	Pandas Documentation	https://pandas.pydata.org/docs/
	SQLite with Python	https://docs.python.org/3/library/sqlite3.html
	JavaScript Fetch API	MDN Web Docs
	Git Version Control	https://git-scm.com/doc
	PyCharm IDE Guide	JetBrains documentation
	Virtual Environments Guide	Python venv documentation

Supervisor:

Name: Muhammad Bilal

Email ID: bilal.saleem@vu.edu.pk

MS Teams ID: bilalsaleemteams@outlook.com

Smart Household and Home Services Management System

Project Domain / Category	Web Based Application
Abstract / Introduction	Modern households often rely on multiple people and external service providers to complete daily chores and maintenance work. Managing these tasks manually can be difficult, time-consuming, and confusing. This project proposes a centralized web-based system that helps users organize household tasks and book home services such as cleaning, plumbing, electrician support, gardening, appliance repair, laundry, and grocery reminders. The system improves convenience, tracking, transparency, and communication between homeowners, service providers, and administrators.
Types of Users	Homeowner / Customer, Service Provider, Admin
Tools & Technologies	Frontend: HTML, CSS, JavaScript, bootstrap, AJAX, JQuery Backend: PHP farmwork (Laravel), simple PHP not allowed Database: MySQL

Problem Statement

In many homes, daily tasks and service requests are handled through phone calls, messages, or informal arrangements. This often leads to missed appointments, poor coordination, delayed responses, and no proper record of completed work. A smart digital system can simplify these activities and make household management more organized and reliable.

Project Objectives

- To provide a single web platform for managing household tasks and home service bookings.
- To help users schedule, track, and review domestic services in an organized way.
- To allow service providers to manage their availability, requests, and service history.
- To give administrators control over users, service categories, bookings, and reports.

Functional Requirements

1. Homeowner / Customer Functionalities

- User registration, login, and profile management.
- Create household tasks and set reminders for routine activities.
- Book home services such as cleaning, plumbing, electrician, pest control, gardening, and appliance repair.
- Track booking status, assigned provider, and service history.
- View invoices, payment history, and service ratings.

- Send complaints, requests, or feedback through the system.

2. Service Provider Functionalities

- Register and maintain service profile and offered categories.
- Set availability, working hours, and service locations.
- Accept or reject service requests.
- Update task progress and completion status.
- View job history, customer feedback, and ratings.

3. Admin Functionalities

- Manage customer accounts, service provider accounts, and login access.
- Create, update, and delete service categories.
- Monitor all bookings, task records, and complaints.
- Assign or reassign service requests when needed.
- Manage payments, reports, and system analytics.
- Approve providers and maintain platform quality.

Non-Functional Requirements

- The system should be secure and protect user login and payment information.
- The interface should be simple, mobile-friendly, and easy to navigate.
- The application should respond quickly and support multiple users.
- The platform should allow future extension for more home service categories.

Proposed Database Tables

Table Name	Purpose	Key Fields	Notes
Users	Stores customer, provider, and admin accounts	user_id, name, email, role	Authentication
Services	Stores available home service categories	service_id, title, price	Service catalog
Bookings	Stores service requests and task bookings	booking_id, user_id, service_id	Core transaction
Payments	Stores payment records and invoice details	payment_id, booking_id, amount	Billing
Complaints	Stores feedback and issue reports	complaint_id, booking_id, status	Support

Suggested Modules

- Authentication and user management
- Household task scheduling
- Home service booking and assignment
- Payment and invoice management
- Notification and reminder system

- Feedback, rating, and complaint module
- Admin dashboard and reports

Expected Outcome

The final system will provide a practical solution for smart household management and home service booking. It will improve convenience for users, help service providers manage requests efficiently, and give administrators proper control over the entire process.

Supervisor

Name: Asma Batool

Email ID: asmabatool@vu.edu.pk

Teams ID: [xpsc00@gmail.com](https://www.teams.microsoft.com/join/00000000-0000-0000-0000-000000000000)

Remote Computer Network Monitoring

Project Domain / Category:

Desktop application

Abstract/Introduction

With the evolution of Internet the systems has become more advanced, old manual activities have revolutionized with the advanced automatic techniques. Innovative techniques have been adopted to remove security breaches. This system is designed to reduce the manual interaction with the client/users to authenticate access, session management of multiple users on different PCs in a client server environment, like multiple stories Internet cafes, shopping malls etc.

The aim of the project is to properly monitor the user's session, billing and termination in an autonomous way. Client will login the PC with login credentials. After login the camera will capture picture and send it to admin. At the start of the session the user will have to give information about the time duration to use the system using GUI interface of the proposed application. This information will be displayed at server side for administrator information. Illegal activities like proxy breaching attempt will automatically generate a warning alert.

Functional Requirement

- This proposed application should have two modules, an Admin Module & a User Module.
- The admin module will have the dashboard showing status of every PC on the network, start time, Duration.
- The request will be displayed at admin side which will get approval by the admin.
- User will access the system using valid login credentials.
- Camera will capture the picture and that should be sent to admin side.
- The user will select the time frame during which he will use the system for that duration. The user can extend the session when needed.
- Application will generate billing at the screen at user side as well as at admin side once after the selected time frame gets completed.
- The user will have the options to pay the bill through different online transactions. The user side will have multiple online transaction options available.

Tools:

C# / VB.net

SQL server

Supervisor:

Name: Asim Mehmood

Email ID: asimmehmood@vu.edu.pk

MS Teams ID: sardar-asim

Stroke Prediction System using Linear Regression

Project Domain / Category

Data Science

Abstract / Introduction

A stroke is defined as an acute neurological disorder of the blood vessels in the brain that occurs when the blood supply to an area of the brain stops and the brain cells are deprived of the necessary oxygen. According to the World Stroke Organization, 13 million people get a stroke each year, and approximately 5.5 million people will die as a result. It is the leading cause of death and disability worldwide, and that is why its imprint is serious in all aspects of life. Stroke not only affects the patient but also affects the patient's social environment, family, and workplace. In addition, contrary to popular belief, it can happen to anyone, at any age, regardless of gender or physical condition.

This project focuses on a binary classification problem: predicting whether a patient will experience a stroke (1) or not (0) based on input health features.

- **Algorithm Chosen:** Logistic Regression (LR) is selected for its high interpretability in healthcare, as its coefficients directly relate to the odds of a stroke.
- **Significance:** It serves as a strong, efficient, and transparent benchmark model for clinical decision support systems.

Objectives

- To build a machine learning model that predicts stroke risk with high accuracy.
- To analyze the impact of various modifiable risk factors (e.g., BMI, smoking status) on stroke occurrence.
- To implement a preprocessing pipeline to handle imbalanced data (SMOTE) and missing values (imputation)

Methodology

The project will follow the KDD (Knowledge Discovery in Databases) process:

1. **Data Acquisition:** Use the Kaggle "Healthcare dataset stroke data" (<https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>).
2. **Data Preprocessing:**
 1. Handle missing values (especially BMI).
 2. Encoding categorical variables (Gender, Smoking Status).
 3. Handling Class Imbalance using SMOTE (Synthetic Minority Over-sampling Technique).
3. **Model Training:** Develop the Logistic Regression model using libraries like Scikit-Learn.

4. **Model Evaluation:** Evaluate using the Confusion Matrix, Accuracy, Precision, and Recall.
5. **Deployment:** Create a simple GUI using Python (tkinter or Streamlit)

Tools:

Python

Supervisor:

Name: Tayyaba Sehar

Email ID: Tayyaba.sehar@vu.edu.pk

MS Teams ID: Tayyaba.sehar13@outlook.com

StudyBuddy: Smart Study Planner & Reminder System

Project Domain / Category

Web Programming

Abstract / Introduction

Effective time management is a major challenge for students, often leading to missed deadlines and poor academic performance. StudyBuddy is a web-based application designed to help students plan, organize, and track their study activities efficiently.

The system allows users to create personalized study schedules, set reminders, and monitor their progress through an interactive dashboard. By providing timely notifications and simple analytics, StudyBuddy encourages consistent study habits and improves productivity.

The main objective of this project is to develop a user-friendly and responsive platform that assists students in managing their academic tasks effectively.

Moreover, objective includes:

- Help students manage study time efficiently
- Provide reminders for tasks and deadlines
- Track academic progress
- Improve productivity and consistency
- Offer simple analytics for better decision-making

The complete project consists of following modules:

1. User Module
2. Task Management Module
3. Notification Module
4. Analytics Module
5. Dashboard Module

Functional Requirements:

1. User Management

- User registration (email & password)
- Secure login/logout
- Profile management (update info, change password)
- Forgot password functionality

2. Study Planning

- Create daily, weekly, and monthly study plans
- Add subjects and topics
- Assign priority levels (High, Medium, Low)
- Set start and end time for each task
- Edit and delete tasks

3. Reminder and Notification System

- Set reminders for each study task
- Email notifications for upcoming deadlines
- Browser notifications (optional)
- Daily summary notification

4. Progress Tracking

- Mark tasks as completed or pending
- View completion percentage
- Track study hours per day/week
- Visual progress using charts/graphs

5. Dashboard

Student View:

- Overview of tasks (completed/pending)
- Upcoming deadlines
- Study progress charts

Admin View

- View all users
- Monitor system usage

6. Search and Filtering

- Search tasks by subject or keyword
- Filter tasks by date, priority, or status

7. Notes Section

- Add personal study notes
- Edit and delete notes

Note:

- **More Functional requirements can be added to each deliverable.**
- **Prior knowledge of related concepts is required. Please thoroughly study the proposal and then opt for the project.**

Tools:

- Python (Flask)
- Database: SQLite
- Libraries: Pandas, Matplotlib (for analytics)

Note: Before selecting project make sure you have enough knowledge about mentioned tools. After that it will not be changed in any case.

Supervisor:

Name: Rizwana Noor

Email ID: rizwana.noor@vu.edu.pk

MS Teams ID: rizwana.noor77

Vehicle Service Management System using MVC-based Framework

Project Domain / Category

Web Application/MVC-based Framework Project.

Abstract / Introduction

This web application project is built on an MVC-based framework. This project follows a three-layered architecture. The first layer is the presentation layer, which includes views. The second layer is the controller (business) layer, which includes services that mediate between the presentation and data access layers. The third layer is the application data access layer (model), which directly interacts with the database server. In this web application design and development process, the student manages vehicle service records, customer details, service schedules, and payment details. This project helps students to understand CRUD (Create-Read-Update-Delete) operations, business logic implementation, and application data flow across MVC layers.

Functional Requirements:

Provide a bulleted list of functional requirements:

Admin:

1. Admin registration.
2. Admin logs in to the application.
3. Admin forgets password.
4. Admin manages (Create-Read-Update-Delete) vehicle in service details.
5. Admin manages (Read-Update-Delete) customer details.
6. Admin manages (Create-Read-Update-Delete) vehicle service schedules.
7. Admin manages (Read-Update-Delete) payment details.
8. Admin logs out of the application.

Customer:

1. Customer registration.
2. Customer logs in to the application.
3. Customer forgets password.
4. Customer views the vehicle details.
5. Customer views the vehicle service schedule.
6. The customer hires the specific vehicle for travelling.
7. The customer pays for the specific vehicle.
8. Customer logs out of the application.

Tools:

MVC-based Framework, Database Server, HTML, CSS.

Supervisor:

Name: Sheikh Israr Ahmad

Email ID: israr.ahmad@vu.edu.pk

Teams ID: israr.ahmad.vu@outlooks.com

Web-based Stock, Inventory and Sales Management System

Project Domain / Category

Web Development / Business Management System

Abstract / Introduction

The proposed project is a web-based Stock, Inventory, and Sales Management System designed to help businesses efficiently manage their products, track inventory levels, and monitor sales activities in real time. The system aims to replace manual record-keeping with an automated, centralized platform that improves accuracy, reduces human error, and enhances decision-making.

This application will provide an intuitive interface for managing stock entries, updating inventory, generating invoices, and analyzing sales reports. It will support multiple users with role-based access and ensure secure data handling. The system is particularly useful for small and medium-sized enterprises (SMEs) seeking to digitize their operations and streamline business processes.

Functional Requirements (Module-wise):

1. User Management Module

- **FR1.1:** User registration and login system
- **FR1.2:** Role-based access (Admin / Manager / Salesman)
- **FR1.3:** Logout functionality

2. Inventory Management Module

- **FR2.1:** Add new inventory items (product name, category, price, quantity)
- **FR2.2:** Update existing inventory details
- **FR2.3:** Delete inventory items
- **FR2.4:** View complete inventory list
- **FR2.5:** View remaining stock of each product

3. Stock Management Module

- **FR3.1:** Add stock (stock in / purchase entry)
- **FR3.2:** Automatically update stock after purchases
- **FR3.3:** Low stock alert/notification
- **FR3.4:** Stock history tracking (in/out logs)

4. Sales Management Module

- **FR4.1:** Create sales invoice
- **FR4.2:** Add multiple items to a single invoice
- **FR4.3:** Automatic stock deduction after sale
- **FR4.4:** Calculate total bill (including taxes/discounts if applicable)
- **FR4.5:** Print/download invoice

5. Return Management Module

- **FR5.1:** Process returned items from customers
- **FR5.2:** Automatically add returned items back to stock
- **FR5.3:** Maintain return history

6. Customer & Supplier Module

- **FR6.1:** Add and manage customer details
- **FR6.2:** Add and manage supplier details
- **FR6.3:** View customer/supplier transaction history

7. Reports Module

- **FR7.1:** Generate daily sales report
- **FR7.2:** Generate monthly sales report
- **FR7.3:** Generate yearly sales report
- **FR7.4:** Inventory report (available stock)
- **FR7.5:** Profit/loss report

8. Dashboard Module

- **FR8.1:** Display total sales summary
- **FR8.2:** Show low stock items
- **FR8.3:** Show recent transactions
- **FR8.4:** Graphical representation of sales (charts)

9. Search & Filter Module

- **FR9.1:** Search products by name/category
- **FR9.2:** Filter sales by date, customer, or product
- **FR9.3:** Quick lookup for invoices

10. System & Security Module

- **FR10.1:** Data backup functionality
- **FR10.2:** Secure data storage (authentication & authorization)
- **FR10.3:** Activity logs (user actions tracking)
- **FR10.4:** Responsive design for mobile and desktop

Tools: (You can use any of the mentioned and unmentioned tools of your choice)

- **Frontend:** HTML, CSS, JavaScript (Bootstrap for UI design)
- **Backend:** Python (Django) *or* PHP (Laravel) *or* JavaScript (Node.js)
- **Database:** MySQL / PostgreSQL
- **IDE/Editor:** VS Code / PyCharm
- **Version Control:** Git & GitHub
- **Web Server:** Apache / Nginx
- **Other Tools:** Postman (API testing), Figma (UI/UX design)

Supervisor:

Name: Abdur Rafay

Email ID: abdur.rafay@vu.edu.pk

MS Teams ID: [abdurrafay94@outlook.com](https://www.linkedin.com/in/abdurrafay94)

Fake News Detection System Using Python and Django

Project Domain / Category:

Data Science / Web Application Development

Introduction:

With the rapid growth of social media and online news platforms, the spread of misinformation and fake news has become a serious global issue. Users often find it difficult to distinguish between credible information and misleading or fabricated content. Fake news can influence public opinion, create social unrest, and spread false narratives, making it essential to develop automated systems that can identify and flag such content. In this project, a web-based fake news detection system will be developed using Python and Django. The system will analyze news articles, headlines, or social media posts and classify them as real or fake using machine learning and Natural Language Processing (NLP) techniques. The platform will allow users to input text or URLs, and the system will provide predictions along with confidence scores. This solution aims to provide users with a reliable tool to verify information and promote digital literacy.

Functional Requirements:

1. User Management & Authentication

- Users can register, login, and logout.
- Basic role management (User & Admin).

2. Dataset Handling & Preprocessing

- Use labeled fake/real news datasets.
- Perform preprocessing:
 - Stopword removal
 - Tokenization
 - Normalization

3. Feature Extraction & Model Development

- Extract features using TF-IDF or Bag of Words.
- Train ML models (e.g., Logistic Regression, Naive Bayes).
- Evaluate using accuracy, precision, recall, F1-score.

4. News Input & Classification

- Users can input:
 - News text
 - URL
- System classifies news as **Real or Fake** with confidence score.

5. Explainability & Basic Verification

- Show prediction confidence.
- Highlight important words (basic explanation).
- (Optional) Simple source checking for URLs.

6. Admin Panel & Model Management

- Admin can:
 - Upload dataset
 - Retrain model

- View system logs

7. System, Testing & Security

- Build web app using Django.
- Perform basic testing with unseen data.
- Ensure simple security (authentication, input validation).

Actors in the System:

Admin (System Manager):

Manages datasets and retrains machine learning models.

Monitors system performance and user activity.

Reviews flagged or controversial news content.

User (End User):

Inputs news articles, headlines, or URLs.

Receives classification results (real or fake).

Views confidence scores and explanations.

Expected System Modules:

1. User Registration and Profile Management:

Users can register and log in to the system.

2. News Input and Classification Module (User):

Users can input text or URLs.

The system processes and classifies the content.

3. Result Display and Visualization:

Show classification results with confidence scores.

Display charts or indicators for reliability.

4. Performance Monitoring and Analytics (Admin):

Monitor model accuracy and user interactions.

Analyze trends in fake news detection.

5. Feedback and Reporting Module:

Users can report incorrect predictions.

Admin can review feedback to improve the model.

Dataset:

The dataset will consist of labeled news articles categorized as real or fake. Data may include headlines, full text, authors, and publication sources.

Example datasets:

<https://www.kaggle.com/datasets/clmentbisailon/fake-and-real-news-dataset>

<https://www.kaggle.com/datasets/jruvika/fake-news-detection>

Tools and Technologies:

Python (Django): Backend web framework for system development.

Jupyter Notebook: For data analysis and model training.

NLTK / SpaCy / Scikit-learn: For NLP processing and machine learning models.

TensorFlow / PyTorch: For deep learning models (optional).

Matplotlib / Seaborn / Plotly: For data visualization and analytics.

General Instructions:

Please add your supervisor soon after selecting the project and introduce yourself with basic details such as your project name and student ID.

You must use your own Teams ID for communication.

Avoid sending unnecessary messages or emails.

Feel free to ask anything relevant to your project using Teams.

Supervisor:

Name: Muhammad Kaleemullah

Email ID: m.kaleem@vu.edu.pk

MS Teams ID: kaleembhatti561@outlook.com

Ship Classification from SAR Images Using the NASTaR Dataset

Project Domain Image Processing

Introduction

Maritime surveillance plays an important role in navigation safety, environmental monitoring, and defense applications. Synthetic Aperture Radar (SAR) imagery is widely used for ship detection because it works in all weather conditions and during day or night. This project focuses on developing an automated system for classifying ship types using SAR image patches from the NASTaR dataset. The system will use deep learning techniques, mainly Convolutional Neural Networks (CNNs), to learn discriminative features and classify different ship categories. Students will also explore transfer learning using Swin Transformer for performance comparison.

Functional Requirements

1. Dataset Loading & Management

- The system shall load the NASTaR dataset for training and testing.
 - The system shall support proper dataset splitting (training, validation, testing).
-

2. SAR Image Preprocessing

- The system shall preprocess SAR images before training.
 - Preprocessing may include resizing, normalization, and data augmentation.
-

3. Ship Classification Engine

- The system shall classify input SAR ship images into predefined ship categories.
 - The system shall use a CNN-based deep learning model as the baseline approach.
-

4. Model Training & Evaluation

- The system shall support training of the CNN model using the dataset.
 - The system shall evaluate performance using accuracy, precision, recall, F1-score, and confusion matrix.
 - The system may support Swin Transformer for comparison.
-

5. Explainability & Visualization

- The system may generate heatmaps (e.g., Grad-CAM) to visualize important regions used in classification.
-

6. Comparative Analysis

- The system may compare performance between CNN and advanced models such as Swin Transformer.
-

7. Results Display & Reporting

- The system shall display predicted ship class with confidence scores.
 - The system shall generate a simple performance report including evaluation metrics and visualizations.
-

Tools & Technologies

- **Programming Language:** Python
 - **Libraries:** PyTorch / TensorFlow, NumPy, OpenCV, Matplotlib, Scikit-learn
 - **IDE:** Jupyter Notebook / Google Colab / Spyder
 - **Dataset:** NASTaR (NovaSAR Automated Ship Target Recognition Dataset)
-

Hardware Requirements

- CPU-based system for CNN training (basic experiments)

- GPU recommended (Google Colab / Kaggle / local GPU) for faster training and advanced models
-

Expected Outcomes

- A working deep learning model for ship classification from SAR images
 - Accurate prediction of ship types with confidence scores
 - Evaluation results including accuracy and F1-score
 - Visualization of model decisions using explainability methods
 - Comparative results between baseline CNN and advanced models
-

Supervisor:

Name: Noor Rahman

Email ID: noor.rahman@vu.edu.pk

MS Teams ID: [mahsud-cs619@outlook.com](https://teams.microsoft.com/join/mahsud-cs619)

Online Task Scheduling in Edge Computing using Reinforcement Learning Techniques

Project Domain

Edge Computing

Introduction

Edge computing complements the cloud infrastructure by providing services at the network edge. It executes workloads in close proximity instead of being transmitted to distant cloud servers. This reduces delay, network traffic, and energy consumption of IoT devices while improving Quality of Service (QoS). This project aims to compare the reinforcement learning-based scheduling algorithms for various task arrival rates.

Functional requirements:

1. A simulation environment will create and maintain ten Virtual Machines (VM) representing ten edge nodes with Millions of Instructions Per Second (MIPS) in the range of [100 MIPS, 200MIPS].
2. Incoming tasks should be generated according to a Poisson distribution with an arrival rate of 0.3, 0.4, and 0.5.
3. Each generated task must be assigned a computational length in Millions of Instructions (MI) based on a uniform distribution with a mean of 200 MI.
4. Each task needs to have its data size uniformly distributed between 10Kb and 40Kb.
5. A scheduler will dynamically assign incoming tasks across the available VMs using:
 - Q-learning
 - Deep Neural Network (DNN)
 - Deep Q-Network (DQN)
6. Develop a Graphical User Interface (GUI) to run the scheduling algorithms for task arrival rates (λ) of 0.3, 0.4, and 0.5.
7. The GUI need to set a total simulation time, where the system automatically allocates the first 30% of the duration to the training phase and the remaining 70% to the testing phase.
8. During the testing phase, the system should evaluate the scheduling outcomes using metrics such as makespan, resource utilization, and response time.
9. Generate line charts of the obtained results for easier interpretation.
10. The computed results in step-6 should be stored in excel file for detailed analysis.
11. The environment can be reset to its initial state, allowing users to rerun simulations with different configurations.

Tools and language:

Languages: Python

IDE: No restriction

Supervisor:

Name: Dr. Zulfiqar Ali Khan

Email ID: zulfiqarmrd@vu.edu.pk

MS Teams ID: zulfiqar.vu@outlook.com

Automated Fire Detection System

Project Domain / Category

Image Processing – Deep Learning / Computer Vision / [Web Application](#)

Abstract

Fire detection is the process of identifying the presence of fire or flames in an environment using sensors, cameras, or other monitoring systems. Traditionally, fire detection relied on smoke detectors, heat sensors, or flame sensors, which activate alarms when they detect smoke, high temperature, or light emissions from fire. However, these conventional systems often have limitations, such as delayed detection, false alarms, or inability to detect fire in open areas like forests. With advances in computer vision and deep learning, fire detection systems can now use image and video analysis to detect fire and smoke in real-time. These systems analyze visual cues such as flame color, flickering patterns, smoke shape, and motion, enabling faster and more accurate detection compared to traditional sensors. This project aims to develop an automated Fire Detection System using deep learning techniques. The system will detect fire and smoke in images or real-time video streams. A Convolutional Neural Network (CNN) model will be trained on a dataset containing fire and non-fire images. The model will be integrated with a real-time detection interface to monitor live camera feeds or uploaded images. This technology can be applied in forest monitoring, smart buildings, industrial safety, and surveillance systems to detect fire at an early stage and reduce damage.

Functional Requirements

- 1. User Management & Authentication**
Manages user registration, login, and access control for authorized system usage.
- 2. Video Input & Camera Integration**
Integrates webcams or CCTV cameras to capture real-time video streams for monitoring.
- 3. Fire Detection Engine (AI-Based)**
Uses deep learning models to automatically detect fire in images and video feeds.
- 4. Smoke Detection Module**
Identifies smoke patterns in visual data to detect early signs of fire.
- 5. Flame Detection Module**
Detects visible flames based on color, shape, and motion characteristics.
- 6. Multi-Sensor Integration (Optional)**
Combines data from sensors like temperature or gas detectors for improved accuracy.
- 7. Real-Time Alert & Notification System**
Sends instant alerts via alarms, messages, or emails when fire is detected.
- 8. Visualization & Monitoring Dashboard**
Displays live video, detection results, and system status in a user-friendly interface.
- 9. Recording & Evidence Management**
Stores detected events, images, and video clips for future reference and analysis.
- 10. False Positive Reduction & Calibration**
Minimizes incorrect detections through model tuning and environmental calibration.

11. **Emergency Response Integration**

Connects with emergency services or systems for quick response in case of fire.

12. **Report Generation & Analytics**

Generates reports and statistics based on detection history and system performance.

13. **Admin & Model Management**

Allows administrators to manage users, update models, and configure system settings.

14. **Performance & Real-Time Requirements**

Ensures fast processing and real-time detection with minimal delay.

15. **Database Integration**

Store detection results including timestamps, image paths, and predictions using SQLite or MySQL. Allow administrators to view and analyze detection statistics.

16. **Security & Compliance**

Protects data privacy and ensures secure access following safety standards.

17. **Environmental Adaptation**

Adapts detection performance under different lighting, weather, and environmental conditions.

18. **User Interface Development**

Provide a web-based interface for image uploads or live camera feed detection

Tools and Technologies Required

- Programming Language: Python
- Libraries & Frameworks:
 - Deep Learning: TensorFlow / Keras or PyTorch
 - Image Processing: OpenCV, Pillow, dlib
 - Backend: Flask / Django
 - GUI: Tkinter / PyQt

Dataset:

- Kaggle Fire Detection Dataset (<https://www.kaggle.com/search?q=fire+detection>)
- Roboflow Fire Dataset (<https://universe.roboflow.com/>)
- Custom images (optional for fine-tuning)
- Database: SQLite / MySQL
- IDE: PyCharm, VS Code, or Jupyter Notebook
-

Hardware:

- CPU: Intel i5 or AMD Ryzen 5 and above
- RAM: Minimum 4 GB
- GPU (optional for faster training)
- Storage: A few GBs for dataset and model weights

Supervisor

Name: Madiha Faqir Hussain

Email ID: madiha.hussain@vu.edu.pk

MS Teams ID: madiha.akhtar74

Automated Helmet detection system using deep learning

Project Domain / Category

Image Processing – Deep Learning / Computer Vision

Abstract

Helmet detection is an important application of computer vision aimed at improving road safety by ensuring that motorcyclists wear helmets. Manual monitoring by traffic authorities can be inefficient and limited in coverage. Automated systems based on deep learning can provide real-time monitoring and enforcement support.

This project proposes the development of a **Helmet Detection System** using advanced object detection models, specifically YOLOv8. The system will be trained on a helmet detection dataset to identify whether a person is wearing a helmet or not.

The trained model will be integrated with a real-time webcam system where **helmet images or actual helmets shown in front of the camera will be detected instantly**. The system will analyze live video frames, draw bounding boxes around detected objects, and classify them as "Helmet" or "No Helmet." This solution can be applied in traffic monitoring, smart surveillance, and safety compliance systems.

Functional Requirements

1. Data Collection & Preprocessing

- Use a helmet detection dataset (Kaggle or Roboflow).

- Include images of people wearing helmets and not wearing helmets.

- Convert dataset into YOLO format (images + label files).

- Resize and normalize images.

- Apply data augmentation (flip, rotation, brightness, scaling).

2. Model Development

- Train object detection models using YOLOv8.

- Use pretrained weights for transfer learning.

- Configure training parameters (epochs, batch size, learning rate).

- Compare both models based on performance metrics.

- Evaluate using mAP, precision, recall, and F1-score.

3. Notebook Implementation

- Implement complete workflow in Jupyter Notebook:

 - Data loading and visualization

 - Model training and validation

 - Performance evaluation

 - Save trained model weights for deployment.

4. Real-Time Detection Using Webcam

- Use OpenCV to access webcam.

- Load trained YOLO model.

- Capture live video frames.

Detect helmets when a person or helmet image is shown in front of the camera.

Draw bounding boxes with labels: "Helmet" or "No Helmet."

Display confidence score for each detection.

5. Alert System (Optional Enhancement)

Display warning message when "No Helmet" is detected.

Optional: Capture and save violation images.

Optional: Sound alert for violations.

Tools and Technologies Required

Programming Language

- Python

Libraries & Frameworks

- PyTorch
- Ultralytics YOLO (v8)
- OpenCV
- NumPy, Pandas
- Matplotlib

Dataset

- Helmet Detection Dataset <https://www.kaggle.com/datasets/andrewmvd/helmet-detection>
- Custom dataset (optional)

Development Environment

- Jupyter Notebook / Google Colab

Hardware Requirements

- CPU: Intel i5 or equivalent
- RAM: Minimum 4–8 GB
- GPU: Recommended for training
- Webcam for real-time detection
- Storage: Few GBs

Supervisor

Name: Laraib Sana

Email ID: laraib.sana@vu.edu.pk

MS Teams ID: [laraibsana08@outlook.com](https://outlook.officeapps.com/teams/08-laraibsana08)

Brain Tumor Segmentation using Res-Net with Flask-Based Visualization

Project Domain / Category

Deep Learning based Web Application

Abstract / Introduction

Brain tumor segmentation is a critical task in medical image analysis, enabling precise detection of tumor regions for diagnosis and treatment planning. In this project, you will develop a **deep learning-based segmentation model** using **Res-Net**, a **U-Net based model** to automatically segment brain tumors from MRI scans in the **BraTS 2023 dataset**.

In addition to training and evaluating the segmentation model, you will develop a **Flask-based web application** that allows users to upload MRI images and visualize the predicted segmentation results alongside ground truth masks.

Functional Requirements:

The following are basic requirements:

- Implement **Res-Net** for brain tumor segmentation using **PyTorch**.
- Preprocess the **BraTS 2023 dataset** (multi-modal MRI images).
- Train the model using **Dice Loss and F1 score** and evaluate its performance.
- Display segmentation results as **images with ground truth comparison**.
- Develop a **Flask web application** where users can upload an MRI scan and receive the predicted segmentation mask.

Note: Understating and learning is the basic part of this project. Just writing the required code is not sufficient for this project.

For your better understanding the guidelines that how to start and study/code the details of all above points is as follows:

Task 1: Understanding the Dataset

- Study the U-Net model and Res-Net model.
- Study the BraTS 2023 dataset (FLAIR, T1, T1ce, T2 modalities).
- Understand tumor segmentation labels (Whole Tumor, Tumor Core, Enhancing Tumor).
- Code to visualize sample MRI images and corresponding masks.

Task 2: Preprocessing & Data Augmentation

- Load and preprocess MRI scans using Nibabel.
- Use 250 patients dataset (minimum)
- Normalize images, apply z-score and apply augmentations (flipping, rotation, resizing).
- Convert segmentation masks into one-hot encoded format.
- Display the effect of augmentation of 3 patients from both angles (sagittal, coaxial).

Task 3: Implementing Res-Net for Brain Tumor Segmentation

- Implement the **Res-Net model** using **PyTorch**.
- Define loss function (Dice Loss) and optimizer (AdamW).
- Train the model and monitor performance metrics.

Task 4: Model Evaluation & Results Visualization

- Evaluate the model using Dice Score, IoU.
- Compare Res-Net vs U-Net segmentation performance.

- Display the performance effect of both models in the form of image segmentation.
- Display segmentation results alongside ground truth masks.
- Display the graph of performance evaluation using Res-Net and U-Net separately.
- Generate a comparison graph of both models performance.

Task 5: Developing a Flask Web App for Segmentation Visualization

- Build a Flask-based web interface for MRI scan uploads.
- Integrate the trained Res-Net model for segmentation predictions.
- Display input image, predicted segmentation mask, and ground truth mask.

Bonus Tasks (Optional, if successfully completed any of the following, 5 extra marks will be awarded)

- Optimize model performance using hyperparameter tuning.
- Improve segmentation quality using post-processing (CRF, morphological operations).

Tools:

- Operating System: Window 7 and above
- RAM 8 GB or more (Dataset size is 3 GB so it cannot be executed on small size RAM)
- Anaconda OR jupyter notebook OR Google Colab (Python)

Download sources:

<https://anaconda.org/>

<https://jupyter.org/>

Language of the Project:

Python

Note: You can write the Names of Functions of your own choice.

Do not use random datasets.

Dataset will be provided through email to the enrolled students.

Supervisor:

Name: Shafaq Nisar

Email ID: shafaq.nisar@vu.edu.pk

MS Teams ID: shafaqnisar1

DeepDerm with Grad-CAM Visualization

Project Domain / Category

Artificial Intelligence (AI) / Computer Vision / Explainable AI (XAI) / Web app

Abstract / Introduction

This project focuses on the development of a deep learning-based system for skin image classification using convolutional neural networks (CNNs). The system aims to categorize skin conditions, particularly acne severity, into multiple classes based on visual features extracted from input images.

In addition, the project incorporates explainable AI techniques, specifically Gradient-weighted Class Activation Mapping (Grad-CAM), to visualize the regions of interest influencing the model's decision. Students are expected to analyze model performance, interpret feature maps, and evaluate the effectiveness of explainability methods in understanding deep learning predictions.

Functional Requirements:

1. Image Upload & Input Handling

The system shall allow users to upload skin/acne images for analysis.

2. Image Preprocessing Module

The system shall preprocess input images (resizing, normalization) before analysis.

3. Acne Classification Engine

The system shall classify the input image into predefined categories (e.g., normal, mild, moderate, severe acne) using a deep learning model.

4. Grad-CAM Visualization & Explainability

The system shall generate a Grad-CAM heatmap to highlight the regions influencing the model's prediction.

5. Result Display Module

The system shall display the predicted class along with confidence score and corresponding heatmap.

6. Performance Evaluation Module

The system shall evaluate and report model performance using metrics such as accuracy, precision, and recall.

7. Model Experimentation Support

The system shall support experimentation with different CNN architectures for performance comparison.

8. Ethical Consideration Module

The system shall provide general guidance only and avoid direct medical or drug recommendations.

Tools:

- **Programming Language:** Python
- **Libraries & Frameworks:** TensorFlow / Keras / PyTorch, OpenCV, NumPy, Matplotlib
- **Models:** CNN (for classification)
- **Explainability Tool:** Grad-CAM
- **Dataset:** Acne/skin image datasets (Kaggle or public sources)
- **Development Environment:** Google Colab / Jupyter Notebook / VS Code
- **Frontend:** Django

Supervisor:

Name: Sonia Salman

Email ID: sonia.salman@vu.edu.pk

Microsoft Teams ID: sonia_salman@outlook.com

Automated Detection, Multi-Stage Severity Grading and Explanation of Diabetic Retinopathy Disease

Disclaimer: This project is primarily recommended for serious students; those without sufficient background or interest in advanced AI concepts may find it challenging.

Project Domain / Category

AI/Machine Learning/Image Processing

Abstract / Introduction

Diabetic Retinopathy (DR) is a progressive eye disease caused by prolonged diabetes. It may lead to damage of retinal blood vessels and potential vision loss. According to the World Health Organization, DR is one of the main causes of preventable blindness worldwide.

Early detection and accurate severity grading are crucial for timely intervention. However, manual screening by ophthalmologists is time-consuming, subjective, and resource-intensive, especially in developing countries. Advances in Artificial Intelligence (AI), particularly deep learning, offer promising solutions for automated, scalable, and accurate DR diagnosis using retinal fundus images.

Functional Requirements:

Develop an AI enabled framework/system that:

- Accepts retinal eye images as input
- Checks image quality before analysis
- Identifies whether the image is normal or abnormal
- Classifies the image into DR stages, such as:
 - No DR
 - Mild Non-Proliferative DR
 - Moderate Non-Proliferative DR
 - Severe DR
- Displays the predicted class with confidence score
- Highlights suspicious regions in the retina image
- Interprets the results using Explainable AI techniques (very important)
- Generates Automated Diagnostic Report containing:
 - Prediction results
 - Severity level
 - Visual explanation (heatmaps)
 - Confidence scores
- Stores patient image and prediction result in the database
- Allows the doctor to review the results and report
- Supports search and retrieval of past results

Tools:

Programming: Python, CSS, Javascript, HTML, PHP etc.

Frameworks: TensorFlow / PyTorch

Deep Learning Models: Yolov11, autoencoder, CNN, Transformer etc.

Libraries: OpenCV, NumPy, Scikit-learn

Visualization: Matplotlib, Seaborn

Explanation tools: Local Interpretable Model-agnostic Explanations (LIME) and Shapley Additive Explanations (SHAP)

IDE: Visual Studio Code, Jupyter notebook, Google Colab, Kaggle

Dataset: [Indian Diabetic Retinopathy Image Dataset \(IDRiD\)](#)

Deployment: Flask, FastAPI, Streamlit

Important Instructions:

- Each group will be assigned a specific set of models and datasets for their project work.
- Students' progress will be monitored on bi-monthly basis through online meetings and proper record will be maintained.
- Student will submit the progress report through LMS/Turnitin/Google Drive.
- Plagiarism and AI Writing report will be strictly checked through **Turnitin**.
- **FINAL REPORT WILL BE DIFFERENT FROM THE CONVENTIONAL FINAL DELIVERABLE** report and will have following sections:
 1. Abstract – 5 Marks (Semester-1, Assignment-1)
 2. Introduction – 15 Marks (Semester-1, Assignment-2)
 3. Literature Review – 15 Marks (Semester-1, Assignment-3)
 4. Methodology – 15 Marks (Semester-2, Assignment-1)
 5. Results and discussion – 15 Marks (Semester-2, Assignment-2)
 6. Conclusion – 5 Marks
 7. References – 5 Marks

(Semester-2, Assignment-3)

Main Tasks:

- Collection of datasets
- Data augmentation (optional)
- Pre-processing
- Splitting of dataset into train, validation, test datasets
- Training of models
- Model evaluation
- Analysis of results
- Writing the reports

Supervisor:

Name: Muhammad Kamran

Email ID: m.kamran@vu.edu.pk

Skype ID: Kamran.rao.vu@outlook.com

Fake News Detection using Machine Learning

Project Domain / Category

Natural Language Processing / Machine Learning

Abstract / Introduction

With the rapid growth of digital journalism and social media, the spread of fake news, misinformation or fabrications presented as legitimate news, has become a global concern. Fake news can manipulate public opinion, influence elections, and cause social unrest. Identifying false information manually is a monumental task that cannot be scaled with the speed of internet sharing.

This project aims to develop a machine learning, based system to automatically classify news articles or social media posts as "Real" or "Fake." By leveraging advanced text analysis, the system will help users verify the credibility of information before they consume or share it.

Problem Statement

Online information ecosystems are currently plagued by misinformation. Existing verification methods, such as manual facts, checking websites, are slow and cannot keep up with the viral nature of digital content. Furthermore, simple keyword filters often fail to capture the nuanced linguistic patterns used in deceptive writing.

An automated, intelligent classification system is required to provide real-time credibility scores for news content to enhance digital literacy and information integrity.

Functional Requirements

The admin (Student) will design and develop a system capable of performing the following tasks:

1. To design and implement a text classification model for detecting Fake News.
2. To preprocess and clean raw text data (remove noise, normalize language, handle slang).
3. To extract meaningful features using techniques such as TF-IDF and word embeddings.
4. To train and optimize various classification algorithms (e.g., Passive Aggressive Classifier, Naive Bayes, Random Forest, or Transformer,based models like RoBERTa).
5. To measure performance using metrics such as Accuracy, Precision, Recall, and the ROC, AUC curve.
6. To build a web-based prototype where users can paste a news URL or text snippet to receive an instant authenticity report.

Dataset:

https://drive.google.com/drive/folders/1_QTQ5pzhe4-EeMlkg161BLDu-Zcukidl?usp=sharing

**You must use your VU email id to access/download the dataset.*

Tools

- **Language:** Python
- **Libraries:** Scikit,learn, Pandas, NumPy, NLTK, or TextBlob.
- **Environment:** Jupyter Notebook, Google Colab, or PyCharm.
- **Web Framework:** Streamlit or Flask (for the prototype interface).

Prerequisite

- Foundational knowledge of Python programming and Data Structures.
- Understanding of Machine Learning workflows (Training/Testing split, Overfitting).
- Familiarity with Natural Language Processing (NLP) concepts.
- The student will complete relevant short courses and prepare initial documentation (SRS and Design Document).

Helping Material

Topic #	Weblink
1	https://www.python.org/
2	Official Python Documentation
3	https://www.w3schools.com/python/
4	https://www.tutorialspoint.com/python/index.htm
5	https://www.kaggle.com/learn/python
6	https://www.kaggle.com/learn/intro-to-machine-learning
7	https://developers.google.com/machine-learning/crash-course
8	https://www.kaggle.com/learn/intro-to-deep-learning
9	Kaggle: NLP Guide
10	NLTK Documentation
11	https://www.tutorialspoint.com/python_deep_learning/index.htm
12	https://www.tutorialspoint.com/deep-learning-tutorials/index.asp
13	https://www.youtube.com/watch?v=VyWAvY2CF9c
14	https://www.youtube.com/watch?v=6M5VXKlf4D4
15	Natural Language Processing Specialization
16	Fake News Detection Research Paper Repository (arXiv)

Additional Tips

- Use keywords like "Misinformation detection," "Stance detection," and "Linguistic features" for deeper research.
- Look for websites that specialize in NLP education and resources.
- Check MOOC platforms such as Coursera, edX, and Udacity for free courses and tutorials.
- Read blog posts and articles written by experts in the field.
- Participate in Kaggle competitions related to NLP to sharpen your feature engineering skills.

Supervisor:

Name: Umair Ali

Email ID: umairali@vu.edu.pk

MS Teams ID: umair.ali.hamid@outlook.com

Fork After Withholding (FAW) Attack Detection Using Machine Learning

Project Domain / Category

Blockchain–Machine Learning.

Abstract / Introduction

Fork After Withholding (FAW) is a sophisticated attack on Proof-of-Work blockchain networks where malicious miners withhold blocks and strategically release them to create forks, earning rewards from multiple pools. This project proposes an ML-based detection system using ensemble methods (Random Forest, XGBoost, and AdaBoost) to analyze miner behavioral patterns such as submission timing, fork participation, work ratios, and reward anomalies, identifying FAW attackers in real-time with high accuracy.

Functional Requirements:

1. Attack Simulation Module

S. No.	Requirement	Description
FR-01	Blockchain Network Simulator	Simulate PoW blockchain with 50-100 miners (honest + attackers)
FR-02	FAW Attack Logic	Implement block withholding, strategic submission timing, and fork triggering
FR-03	Honest Miner Behavior	Poisson process for block discovery; immediate block submission
FR-04	Dataset Generation	Produce labeled dataset with 10,000+ mining rounds
FR-05	Feature Extraction	Extract 15+ behavioral features per miner (timing variance, PPoW/FPoW ratio, fork count, orphan rate, reward efficiency)

2. ML Detection Module

S.No.	Requirement
FR-06	Preprocess data: SMOTE for imbalance, feature scaling
FR-07	Train Random Forest classifier (100 trees, balanced weights)
FR-08	Train XGBoost classifier (scale_pos_weight=10)
FR-09	Train AdaBoost classifier (50 decision stumps)

S.No.	Requirement
FR-10	Implement soft voting ensemble combining all three
FR-11	Evaluate: precision, recall, F1-score, AUC-ROC

3. Real-Time Detection

ID	Requirement
FR-12	Stream monitoring with <5 min detection latency
FR-13	Alert system (console/email) for detected attackers
FR-14	Dynamic credit scoring (0-100) per miner
FR-15	Streamlit dashboard with real-time visualizations

4. Expected Deliverables

- FAW Attack Simulator: Python-based simulator generating labeled mining data
- Labeled Dataset: 10,000+ mining rounds with 15+ features per miner
- Three Trained Models: Random Forest, XGBoost, AdaBoost
- Ensemble Classifier: Voting ensemble combining all three models
- Real-Time Detection Script: Stream processor with alerting capability
- Streamlit Dashboard: Interactive visualization of miner risks and alerts
- Final Results Evaluate: precision, recall, F1-score, AUC-ROC metrics, experimental results, and comparative analysis

Attack Simulation Tools

Tool	Purpose
Python 3.9+	Primary language
SimPy	Discrete event simulation
NumPy	Poisson processes, random numbers
Pandas	Data collection & storage

ML Implementation Tools

Tool	Purpose
Scikit-learn	Random Forest, AdaBoost, preprocessing, metrics
XGBoost	XGBoost classifier
Imbalanced-learn	SMOTE for class imbalance
GridSearchCV	Hyperparameter tuning
Matplotlib/Seaborn	Visualization

Real-Time & Dashboard

<u>Tool</u>	<u>Purpose</u>
Streamlit	Interactive dashboard
Plotly	Real-time charts
Joblib	Model serialization

Supervisor:

Name: Fouzia Jumani

Email ID: fouziajumani@vu.edu.pk

MS Teams ID: [fouziajumani@outlook.com](https://teams.microsoft.com/join/fouziajumani@outlook.com)

GRAZE SMASHER PRODUCT HUNTING TOOL

Project Domain / Category

Artificial Intelligence / Machine Learning / Web App

Abstract / Introduction

Graze Smasher is a web-based product hunting tool designed to detect and filter fake reviews in e-commerce platforms using machine learning algorithms. The system validates user submitted reviews in real time by applying Sentiment Classification, Support Vector Machine (SVM), and Decision Tree J-48 algorithms, combined with IP detection, to accurately distinguish genuine reviews from spam empowering buyers to make informed purchasing decisions.

In the modern digital market, fake reviews are a type of consumer fraud that is widespread, and product owners employ a community of spammers to unnaturally inflate the ratings of their products and deceive actual customers. To solve this issue, Graze Smasher creates a smart validation layer that examines the linguistic patterns, emotional tones and origin metadata of each review. The application is developed based on the Django framework, offering a refined user experience and a robust multi algorithm detection pipeline running in the background, and eventually establishing a more open and reliable online shopping experience.

Functional Requirements:

- **User Management & Authentication:** The system shall provide secure user registration, login, and role-based access control to differentiate between general users, reviewers, and administrators.
- **Product Discovery & Hunting:** The system shall allow users to search, browse, and submit products for review analysis, enabling discovery of trending or suspicious products across e-commerce platforms.
- **Graze Smasher Scoring Algorithm:** The system shall apply a composite scoring mechanism combining SVM, Decision Tree J-48, and Sentiment Classification results to generate a final authenticity score for each reviewed product.
- **Product Performance Analytics:** The system shall track and display product level analytics including total reviews received, percentage of fake vs genuine reviews detected, and overall authenticity rating over time.
- **Product Monitoring & Alerts:** The system shall continuously monitor submitted products and trigger automated alerts to users or admins when a suspicious spike in fake reviews is detected for a particular product.

- **User Dashboard & Personalization:** The system shall provide each user with a personalized dashboard displaying their review history, saved products, received alerts, and authenticity scores of monitored products.
- **Social Proof & Community Insights:** The system shall aggregate genuine verified reviews and display community-driven trust ratings, helping buyers make decisions based on authenticated social proof.
- **Visualization & Reporting:** The system shall generate visual reports including charts and graphs representing fake review trends, detection accuracy, and product authenticity scores for both users and admins.
- **Admin & Platform Management:** The system shall provide administrators with a dedicated panel to manage users, review flagged content, oversee ML model performance, and configure platform wide detection settings.
- **API & Integration Capabilities:** The system shall expose RESTful API endpoints to allow integration with third party e-commerce platforms, enabling external product data to be fed into the fake review detection pipeline.
- **Ethical & Legal Compliance:** The system shall ensure that all user data collection, IP detection, and review analysis processes comply with applicable data privacy laws and ethical guidelines, with transparent disclosure to users regarding how their data is used and stored.

Tools:

- **VS Code**
Primary code editor used for writing and managing Python and Django project files.
- **Python 3.10**
Core programming language used for backend logic and machine learning model development.
- **Django**
Python-based web framework used to build and deploy the web application frontend and backend.
- **Scikit-learn**
Python ML library used to implement SVM and Decision Tree J-48 classification algorithms.
- **NLTK / TextBlob**
Natural language processing libraries used for sentiment classification and text preprocessing.

- **Jupyter Notebook**

Used for ML model training, testing, and result visualization during the development phase.

- **MySQL / SQLite**

Relational database used to store submitted reviews, classification results, and user data.

Supervisor:

Name: Muhammad Wasif Mairaj

Email ID: wasif.mairaj@vu.edu.pk

MS Teams ID: wasif.vu@outlook.com

Image Analysis and Tagging Web Application

Project Domain / Category

Web Application

1. Introduction

With the rapid growth of digital content, there is a strong need for automated image understanding systems. This project aims to develop a web-based application that utilizes Azure Cognitive Services (Computer Vision API) to analyze uploaded images and generate descriptive tags along with confidence scores.

2. Objectives

- Develop a user-friendly web application
- Integrate AI-based image recognition
- Display tags with confidence levels
- Ensure scalability and performance

3. Scope

- Upload images (JPG, PNG)
- Process images using Azure APIs
- Display tags and captions
- Optional history feature

4. Functional Requirements

- Image upload with validation
- Image processing via API
- Display results (tags, captions, confidence)
- Error handling
- Optional user accounts and history

5. Non-Functional Requirements

- Performance: 3–5 seconds response time
- Scalability: Cloud-based support
- Security: HTTPS, secure API keys
- Usability: Simple UI/UX
- Reliability: Minimal downtime
- Maintainability: Modular code
- Compatibility: Cross-browser support

6. Tools & Technologies

- Frontend: HTML, CSS, JavaScript / React
- Backend: Node.js / Python
- API: Azure Cognitive Services
- Database: MongoDB / MySQL
- Cloud: Microsoft Azure

7. Expected Outcomes

Functional web app

Accurate tagging system

User-friendly interface

Supervisor:

Name: Dr. Nida Anwar

Email ID: nidaanwar@vu.edu.pk

MS Teams ID: nida_vu@outlook.com

Intelligent Diabetes Risk Predictor

Project Domain / Category

Machine learning, data science.

Abstract / Introduction

Diabetes is a chronic disease that requires early detection and continuous monitoring to prevent serious health complications. This project, Intelligent Diabetes Risk Predictor, presents a machine learning-based system designed to assist both patients and healthcare providers in predicting the likelihood of diabetes using clinical and lifestyle data.

The system utilizes a publicly available dataset to train multiple machine learning models, including Neural Networks, Support Vector Machine (SVM), Decision Tree, and Logistic Regression. The data undergoes preprocessing, exploratory data analysis, and is divided into training and testing sets to ensure accurate model evaluation. Performance is assessed using metrics such as accuracy, precision, recall, F1-score, and confusion matrix, enabling comparison and selection of the best-performing model.

In addition to prediction, the system provides risk factor analysis and interpretable explanations to enhance transparency. It also offers personalized recommendations based on prediction results, along with visualization dashboards for monitoring health trends over time. The system supports report generation, model persistence, and continuous improvement through feedback mechanisms.

Overall, this project aims to deliver an intelligent, user-friendly, and efficient decision-support tool that enhances early diagnosis and promotes preventive healthcare practices for diabetes management.

Functional Requirements:

Admin (Student) will perform all of these tasks (Functional Requirements).

1. User Management & Authentication

- The system shall allow users (patients, healthcare providers, and administrators) to register, log in, and manage their accounts securely.
- The system shall enforce role-based access control.

2. Health Data Input Collection

The system shall allow users to input health-related data required for diabetes prediction. The system shall also support dataset import by administrators for model training. (first download data set from

<https://www.kaggle.com/datasets/akhilalexander/diabeticprediction>)

3. Data Analysis & Visualization

The system shall perform Exploratory Data Analysis (EDA) on datasets.

The system shall display:

- Summary statistics
- Trends and patterns
- Visual insights using graphs

4. Data Preprocessing

The system shall preprocess data by:

- Cleaning and transforming data
- Splitting into training (70%) and testing (30%) datasets

5. Machine Learning Prediction Engine

The system shall train machine learning models including:

- Neural Networks
- Support Vector Machine (SVM)
- Decision Tree
- Logistic Regression

The system shall use trained models to predict whether a patient has diabetes.

6. Model Evaluation & Performance Analysis

The system shall evaluate models using:

- Accuracy
- Precision
- Recall
- F1-score

The system shall generate a confusion matrix for performance analysis.

7. Model Comparison & Selection

The system shall compare multiple models and identify the best-performing model based on accuracy and evaluation metrics.

8. Model Persistence

The system shall save trained models for future use without retraining.

9. Risk Factor Analysis & Explanation

The system shall identify key features contributing to predictions and provide interpretable explanations.

10. Personalized Recommendations

The system shall provide recommendations based on prediction results. (e.g., lifestyle changes, medical consultation).

11. Longitudinal Tracking & Progress Monitoring

- The system shall store user health records over time.
- The system shall allow users and healthcare providers to track changes in health status and prediction results.

12. Visualization & Dashboard

The system shall provide dashboards to display:

- Prediction results
- Risk Levels
- Model performance
- Trends over time

13. Report Generation & Export

The system shall generate reports containing:

- User Data
- Prediction results and analysis.
- Model Evaluation Metrics

Reports shall be downloadable (PDF/CSV).

14. User Education & Resources

The system shall provide educational content related to:

- Diabetes risk factors
- Prevention strategies
- Healthy lifestyle guidelines

15. Clinical Decision Support

- The system shall assist healthcare providers by presenting ML-based insights and risk analysis.
- The system shall support clinical decision-making but shall not replace professional judgment.

16. Feedback & Model Improvement

- The system shall collect user and clinician feedback.
- The system shall support continuous model improvement through retraining.

17. Admin & Model Management

The system shall allow administrators to:

- Manage datasets
- Train/update models
- Monitor system performance

18. Performance & Real-Time Requirements

- The system shall generate predictions within an acceptable response time.
- The system shall efficiently handle multiple users.

19. Security & Compliance

- The system shall ensure secure storage and transmission of user data.
- The system shall maintain data privacy and comply with relevant standards.

Pre-requisite:

Note: In order to completely understand the machine learning and data science algorithms, watch given tutorials and also google for better understanding.

- <https://www.javatpoint.com/machine-learning>
- <https://ocw.vu.edu.pk/Videos.aspx?cat=Computer+Science%2fInformation+Technology+&course=CS607>
- https://www.youtube.com/watch?v=u-PaJCpwiU&list=PLu0W_9lI9ai6fAMHp-acBmJONT7Y4BSG&index=1
- https://vulms.vu.edu.pk/Courses/CS607/Downloads/AI_Complete_handouts_for_Printing.pdf

Dataset:

<https://www.kaggle.com/datasets/akhilalexander/diabeticprediction>)

Python Tutorials:

- <https://www.programiz.com/python-programming>
- <https://www.tutorialspoint.com/python/index.htm>
- <https://www.w3schools.com/python/>

-

Machine Learning Tutorials:

- <https://machinelearningmastery.com/machine-learning-in-python-step-by-step/>
- <https://www.geeksforgeeks.org/machine-learning-with-python/>
- <https://www.youtube.com/watch?v=Zftl2fEz0Fw>
- <https://www.simplilearn.com/10-algorithms-machine-learning-engineers-need-to-know-article>
- <https://www.analyticsvidhya.com/blog/2017/09/common-machine-learning-algorithms/>

Tools:

Language: Python (Only python language)

Framework: Anaconda

Supervisor:

Name: Komal Khawer

Email ID: komal.khawer@vu.edu.pk

Teams ID: kom.kk@outlook.com

Multilingual AI-Based Framework for Real-Time Semantic Plagiarism and Code Similarity Detection

Project Domain / Category

Web Application

Abstract / Introduction

This project aims to design and develop an advanced real-time, multilingual plagiarism detection system capable of identifying both textual and logical similarities in academic submissions across diverse languages and formats.

Unlike traditional plagiarism detection tools that rely primarily on keyword matching, this system leverages advanced Natural Language Processing (NLP) and Machine Learning (ML) techniques to detect semantic similarity, enabling it to identify paraphrased, translated, or contextually similar content with high accuracy.

The system is designed to operate in real time, allowing instant analysis of submissions as they are uploaded. This ensures immediate feedback for educators, learners or programmers enhancing the efficiency of their evaluation workflows.

To support multilingual environments, the system incorporates cross-lingual embeddings and translation-aware models, enabling it to detect similarities between content written in different languages (e.g., English, Urdu, Pashto, Punjabi, Balochi, Saraiki and Arabic) without losing contextual meaning.

In addition to textual analysis, the system will evaluate programming assignments by transforming source code into Abstract Syntax Trees (ASTs). This enables the detection of logical and structural similarities, even when the code has been altered through variable renaming, reordering, or syntactic changes.

Functional Requirements:

1. **Upload Documents and Source Code Files:** Allow uploads in PDF, DOCX, TXT, and code formats with preprocessing.
2. **Semantic Plagiarism Detection Engine:** Analyse meaning using embeddings/transformers.
3. **Detection of AI generated content:** Real time detection of AI generated content (For example Chat GPT and DeepSeek like text).
4. **Code Similarity Detection Engine:** Compare logical structures of code.
5. **Generate Detailed Similarity Reports with Visualization:** Provide scores, highlights, and graphs.
6. **Multilingual Text Input & Submission:** Extendable support for languages like English, Urdu, Pashto, Punjabi, Balochi, Saraiki and Arabic.
7. **Cross-Lingual Plagiarism Detection:** Ability to detect plagiarism for English, Urdu, Pashto, Punjabi, Balochi, Saraiki and Arabic.
8. **Admin Dashboard:** Manage users, Visualization and reporting dashboard.

Tools:

You are free to use any tool(s) provided that ALL the functional requirements of the project are FULLY implemented.

Supervisor:

Name: Mehboob Ali

Email ID: mehboob.ali@vu.edu.pk

MS Teams ID: mehboobali.vu@outlook.com

Smart Food Inspection System for Meat Quality Analysis Using Artificial Intelligence

Project Domain / Category

Artificial Intelligence / Computer Vision / Food Quality Inspection / Web Application

Abstract / Introduction

The proposed project aims to develop an AI-powered system for detecting and classifying fresh meat from various animals, including chicken, beef, mutton, and fish. The system will utilize computer vision and deep learning techniques to analyze meat images and determine both the type and freshness level.

Ensuring meat quality is critical for food safety, consumer health, and regulatory compliance. Traditional methods rely on manual inspection, which can be subjective, time-consuming, and prone to human error. This project addresses these limitations by providing an automated, accurate and scalable AI-based solution.

The system will classify meat type and assess freshness based on visual features such as colour, texture and moisture. It can assist vendors and consumers by providing recommendations regarding consumption suitability.

By leveraging convolutional neural networks (CNNs) and image processing techniques, the platform will contribute to smart food quality monitoring and safety assurance.

Functional Requirements:

FR1: User registration and authentication (admin and users).

FR2: Upload or capture meat images using a web/mobile interface.

FR3: AI-based classification of meat type (chicken, beef, mutton, fish, etc.).

FR4: Freshness detection (fresh, moderate, spoiled) using trained deep learning models.

FR5: Display confidence score and classification results.

FR6: Recommendation system suggesting whether the meat is safe for consumption.

FR7: History tracking of analyzed images for users.

FR8: Admin dashboard for managing dataset, users, and system monitoring.

FR9: Model retraining module to improve accuracy with new data.

FR10: Reporting and analytics (accuracy, usage statistics, classification trends).

Tools:

Frontend: React.js / HTML, CSS, JavaScript

Backend: Flask / Django (Python)

Database: MongoDB

AI/ML Libraries:

- TensorFlow / PyTorch
- OpenCV (image processing)
- Scikit-learn

Development Environment:

- VS Code
- Jupyter Notebook

Other Tools:

- GitHub (version control)
- Docker (deployment)
- Kaggle / Custom Dataset (for training images)

Note: *Attendance at Google Meet sessions is mandatory for discussing the project with the supervisor; failure to attend may result in the project not being accepted.*

Prerequisite:

To help students grasp the project problem concepts, they must be required to complete a free course certification along with initial project documentation. Students must have basic knowledge of:

- Python programming
- Machine Learning & Deep Learning concepts
- Image processing fundamentals

A relevant free course on **Computer Vision or Deep Learning** is recommended before implementation. Guidance and datasets will be provided during supervision sessions. Relevant course materials and sources will be shared during Google Meet sessions for further guidance.

Supervisor:

Name: Dr. Saima Munawar

Email ID: saima.munawar@vu.edu.pk

MS Teams ID: saima.vu1@outlook.com